

University of Dhaka
Institute of Information Technology (IIT)

Project
On

**“Nexis: A Multi-Agent Autonomous Engineering Platform
with Closed-Loop Fault Recovery”**

Submitted By

Name: Ratul Sikder
Exam Roll: 2506102

A project submitted to the Institute of Information Technology (IIT), DU
for the partial fulfillment of the degree requirements.

Supervised By

Institute of Information Technology (IIT)
University of Dhaka

Date: 11th August, 2026

Acknowledgement

I would like to express my sincere gratitude to the faculty of the Institute of Information Technology (IIT), University of Dhaka, for the academic environment and guidance that made this project possible. Nexis grew out of a genuine, recurring frustration familiar to every engineering team I have worked with or studied: the gap between an AI tool that writes a function and a system that actually closes the loop on a production failure. I am grateful for the freedom to pursue that problem in depth, across nine cooperating agents, a real multi-tenant control plane, and a working closed-loop deployment engine, rather than a narrower proof of concept. I also thank my family for their patience during the long build-and-verify cycles this project demanded.

Declaration

This is to declare that this project is my original work. No part of this work has been submitted elsewhere, partially or entirely, for the award of any other degree or diploma. All project-related information will remain confidential and shall not be disclosed without the formal consent of the project supervisor. The plagiarism policy, as stated by the supervisor, has been maintained.

Student's Name: Ratul Sikder

Student's Signature: _____

Signature Page

Project : Nexis: A Multi-Agent Autonomous Engineering Platform
with Closed-Loop Fault Recovery

Student Name : Ratul Sikder

Exam Roll : 2506102

Institute : Institute of Information Technology (IIT), University of
Dhaka

Date of Submission : 11th August, 2026

Supervised By : _____

Supervisor's Approval : _____

Abstract

Modern software engineering teams spend an estimated 40–60% of their time on maintenance, debugging, and manual incident response rather than building new capability. Existing AI-assisted developer tools – code completion, automated test generation, deployment dashboards – address individual tasks in isolation; none model the collaborative structure of a real engineering team, and none close the loop between detecting a production failure and autonomously repairing it. **Nexis** addresses both gaps at once. It models a software engineering organisation as nine role-specialised autonomous agents across two cooperating layers – a five-agent Execution Team (Architect, Backend, QA, DevOps, Data Engineer) and a four-agent Self-Healing Loop (Sentinel, Pathfinder, Synthesiser, Validator) – durably orchestrated by Temporal and gated by a severity-routed human-in-the-loop Approval Gate that supports Approve, Reject, and Modify (RLHF) decisions. This report documents the complete, working system: a multi-tenant control plane (Go, PostgreSQL with row-level security, Neo4j, Redis), a Docker shadow-validation sandbox, a real GitHub-App-mediated GitOps deployment path, a from-scratch Docker preview-deploy engine that clones, builds, and serves an arbitrary connected repository and feeds its own build/runtime failures back into the same self-healing pipeline, and a Next.js console spanning authentication, RBAC, intelligent role recommendation, incident observability, billing, and multi-provider integrations. Two live end-to-end recovery runs against a local large language model are reported verbatim – one correctly timing out at the human approval gate, one succeeding fully from fault detection through an approved, deployed patch in under two minutes – alongside a live, verified proof of the preview-deploy engine building and serving a real public repository. The report is written as a candid engineering account: what works is demonstrated with reproducible evidence, and what is deliberately out of scope – a fully credentialed production deployment to the registered `nexis.sh` domain, and the human-subject NASA-TLX study – is stated plainly rather than implied.

Keywords: Multi-agent systems, autonomous software engineering, self-healing systems, closed-loop fault recovery, root-cause analysis, LLM-guided program repair, DevOps automation, Temporal workflow orchestration, human-in-the-loop AI, retrieval-augmented generation.

Contents

Acknowledgement	1
Declaration	2
Signature Page	3
Abstract	4
1 Introduction	10
1.1 Background	10
1.2 Motivation	11
1.3 Problem Statement	11
1.4 Project Objectives	12
1.5 Scope of the Project	13
1.6 Significance of the Project	14
2 Literature Review	15
2.1 Overview	15
2.2 Research Gap	17
2.3 Conclusion	18
3 System Analysis and Requirements	19
3.1 Requirement Analysis	19
3.2 Functional Requirements	20
3.2.1 Owner/Admin can:	20
3.2.2 Engineer/Member can:	21
3.2.3 Viewer can:	22
3.2.4 The 9-Agent Fleet autonomously:	22
3.3 Non-Functional Requirements	24
3.4 System Feasibility	25

3.4.1	Technical Feasibility	25
3.4.2	Economic Feasibility	26
3.4.3	Operational Feasibility	26
4	System Design and UML Diagrams	27
4.1	System Architecture Overview	27
4.2	Use Case Diagram	30
4.2.1	Level 0 Use Case Diagram	30
4.2.2	Level 1 Use Case Diagram	31
4.3	Data Flow Diagram	33
4.3.1	DFD Level 0 (Context Diagram)	33
4.3.2	DFD Level 1 — Owner/Admin	34
4.3.3	DFD Level 1 — Self-Healing Loop	36
4.4	Self-Healing Loop Sequence Diagram	38
5	Implementation	42
5.1	Tools and Technology	42
5.1.1	Backend	42
5.1.2	Frontend	43
5.1.3	Data and Messaging	43
5.1.4	AI / LLM	43
5.1.5	DevOps and Infrastructure	44
5.1.6	IDE / Version Control	44
5.2	Project View	44
5.2.1	Authentication and Onboarding	45
5.2.2	Admin / Owner Console	45
5.2.3	Members, Roles, and Sessions	49
5.2.4	Self-Healing Loop — Live Evidence	52
5.2.5	Docker Preview-Deploy Engine (Ops Tab)	58
5.2.6	Projects, Agents, and Observability	65
5.2.7	Integrations	72
5.3	Database Schema	72
6	Testing and Results	76
6.1	Static Verification	76
6.2	Live End-to-End Recovery Runs	77

6.2.1	Run 1: Approval-Gate Timeout — the Safety Mechanism Under Real Timing Pressure	77
6.2.2	Run 2: Approved and Succeeded in 1 Minute 40 Seconds	78
6.3	Live Deploy-Engine Verification	79
6.4	Defects Found and Fixed During This Engineering Cycle	80
6.5	Testing Summary	80
7	Conclusion and Future Work	82
7.1	Overview	82
7.2	Limitations of the Project	83
7.3	Future Work	84
7.4	Conclusion	85
	References	85

List of Figures

4.1	NEXIS System Architecture (full services topology across the client, control-plane/LLM, sidecar, data, and integration layers).	28
4.2	Level 0 Use Case Diagram (three human actors and the mediated external systems).	30
4.3	Level 1 Use Case Diagram (per-actor use case groups and the autonomous 9-agent fleet).	32
4.4	DFD Level 0 (context diagram with external entities and top-level data stores). . .	34
4.5	DFD Level 1 — Owner/Admin (twelve administrative processes and their tenant-scoped data stores).	35
4.6	DFD Level 1 — Self-Healing Loop (the 9-agent fleet as twelve numbered processes with their data stores).	37
4.7	Self-Healing Loop Sequence Diagram (one full recovery run, 18 numbered messages across all nine agent lifelines).	39
5.1	Sign-Up Screen (new account registration).	45
5.2	Sign-In Screen (JWT session login with HttpOnly cookie).	46
5.3	Forgot-Password Screen (non-leaky password-reset request).	46
5.4	Admin Dashboard (KPI tiles, incidents-over-time chart, and system status panel).	47

5.5	Agents Screen (the nine-agent fleet roster across both layers).	48
5.6	Workflows Screen (RecoveryPipeline run listing).	49
5.7	Activity Stream (organisation-wide live event feed).	50
5.8	Validator Sandbox Console (sandboxed shadow execution of candidate patches). .	51
5.9	Audit Log (append-only, hash-chained entries with CSV export).	51
5.10	Members Settings (members list with the heuristic role-recommendation panel). .	52
5.11	Sessions Settings (active sessions with device, IP, and remote revoke).	53
5.12	API Keys Settings (scoped bearer key creation, listing, and revocation).	53
5.13	Billing Settings (Stripe usage and payment-method management).	54
5.14	Organization Settings (organisation profile management).	54
5.15	Live Demo Console (27-scenario fault-injection picker; one scenario fully wired, the rest backend-pending).	55
5.16	Incident Detail, Mid-Flight (live pipeline with completed Architect/Backend/QA steps showing real token counts; DevOps and Data Engineer running in parallel). .	56
5.17	Incident Detail, Pipeline Complete (all L1/L2 steps green; severity Medium, ap- proval Pending).	56
5.18	Approvals Queue (real pending approval row with Reject / Modify / Approve actions).	57
5.19	Modify-and-Approve Dialog (RLHF diff editing; the engineer’s edit becomes a feedback_examples training row).	58
5.20	Timeout-Rejected Run (the 120-second approval window elapsed; the workflow safely auto-rejected as timeout_rejected rather than deploying unreviewed code). .	59
5.21	Fully Approved Run (Medium severity, Approved, Succeeded; complete nine-step timeline in 1 m 40 s with real per-agent token and cost figures).	60
5.22	Backend Payload, Expanded (raw JSON evidence of the qwen2.5-coder:14b- generated unified diff, provider=“ollama”).	61
5.23	Project Overview Tab (project summary and health).	62
5.24	Project Integrations Tab (provider connections including the GitHub repository binding).	62
5.25	Project Recovery Policy Tab (per-project self-healing configuration).	63
5.26	Project Activity Tab (project-scoped event history).	63
5.27	Project Ops Tab, Empty State (the Deploy panel before any deployment).	64
5.28	Project Ops Tab, Deploy Attempt (a real validation error — “project has no github repository bound” — returned by the live API).	64
5.29	Projects Screen (project cards with health indicators).	66
5.30	New-Project Wizard (guided project creation).	66

5.31	Agent Detail, Backend Agent (per-agent configuration and run history).	67
5.32	Recovery Pipeline Screen (the nine-agent pipeline structure).	67
5.33	Performance Screen (pipeline latency and throughput charts).	68
5.34	Cost Tracker (per-organisation token ledger with daily budget).	69
5.35	System Health (parallel dependency probes: Postgres, Redis, Neo4j, MinIO, Tem- poral).	70
5.36	Eval Harness (side-by-side OpenAI-versus-Ollama comparison over fixture inci- dents).	71
5.37	Knowledge Base (pgvector-backed retrieval status).	71
5.38	Integrations Screen (provider catalogue: GitHub App, Sentry, Datadog, Pager- Duty, Slack, Argo CD, Stripe).	73
5.39	Webhook Activity (inbound webhook delivery log).	74
5.40	Connection Health (per-integration health probes).	74

List of Tables

2.1	Position of surveyed tools relative to Nexis.	17
3.1	Summary of non-functional requirements and their realisation.	24
4.1	Owner/Admin DFD Level 1 processes and their primary data stores.	35
5.1	Key tenant-scoped tables in the Nexis PostgreSQL schema.	75
6.1	Static verification commands and results across all NEXIS components.	77
6.2	Defects discovered by the verification process, with root causes, fixes, and verifi- cation.	80

Chapter 1 Introduction

This chapter introduces the context in which **Nexis** was conceived, the concrete pain points that motivated it, the limitations of existing approaches it responds to, the objectives the project set out to achieve, its scope in terms of the human and non-human actors it serves, and the significance of delivering a genuinely closed loop from fault detection to verified deployment.

1.1 Background

Software rarely fails at the moment it is written; it fails in production, under load, against data and dependencies its authors did not anticipate. Industry experience, reflected in the original proposal for this project, places the cost of that reality at 40–60% of an engineering team’s time spent on maintenance, debugging, and incident response rather than on building new capability. For a small team, this means that for every engineer creating value, roughly one other engineer is effectively employed full-time repairing it.

The last few years have seen a rapid rise of AI-assisted software engineering. Code-completion assistants such as GitHub Copilot [1] now write a meaningful fraction of new code, and adjacent tools generate tests, review pull requests, and summarise logs. Yet these tools are point solutions: each one accelerates a single task performed by a single human. The production incident lifecycle — detect a fault, diagnose its root cause, write a fix, validate it, obtain approval, deploy it, and verify the deployment — remains a chain of manual hand-offs in which a human engineer is the transport mechanism between every pair of stages.

Nexis exists to close that chain. Rather than assisting one engineer at one task, it models a software engineering organisation as nine role-specialised autonomous agents arranged in two cooperating layers — a five-agent Execution Team and a four-agent Self-Healing Loop — durably orchestrated by the Temporal workflow engine [2], with a human approval decision reduced to a single severity-routed click rather than an open-ended manual process.

1.2 Motivation

The specific pain points that motivated this project are concrete and observable in any engineering organisation:

- **No tool closes the loop.** Monitoring products such as Sentry [14], Datadog [16], and Pager-Duty [15] detect failures and page a human; code assistants such as Copilot [1] write code once a human asks. Nothing on the market connects the detection of a failure to its autonomous repair.
- **The engineer is the bottleneck for every incident.** Every alert, however routine, requires a human to read logs, localise the fault, write a patch, run tests, request review, deploy, and watch the rollout. The human is on the critical path of all seven stages, even for fault classes that recur in near-identical form.
- **Root-cause analysis is unaided.** Localising a fault means manually tracing call chains and service dependencies from memory; no mainstream incident tooling walks the code dependency graph or ranks candidate root causes with any stated evidence.
- **Repair without validation is dangerous, so it is not attempted.** Because there is no safe, sandboxed path for a machine-generated patch to prove itself before a human sees it, organisations reasonably refuse to let machines patch anything — and the loop stays open.
- **Approval is all-or-nothing.** Deployment gates are either absent (risky automation) or heavy-weight (a full review cycle for a one-line fix). No existing tool routes the depth of human involvement by the measured severity of the change.

1.3 Problem Statement

Existing approaches to AI-assisted engineering and incident response share a set of structural limitations:

- They are **isolated tools** — detection, diagnosis, code generation, testing, and deployment each live in separate products with no shared state or protocol between them.
- They provide **no closed loop**: a detected fault never automatically becomes a validated, deployable repair.

- They keep a **human as the bottleneck for every decision**, regardless of how small, repetitive, or low-risk the change is.
- They offer **no causal root-cause tooling**: nothing traverses the actual dependency structure of the code, in the manner a graph database such as Neo4j [3] makes possible, or applies causal reasoning of the kind formalised by libraries such as DoWhy [4], to explain *why* a fault occurred.
- They provide **no safe autonomous deploy path**: even where a machine can propose a patch, there is no sandboxed validation stage, no severity-aware approval gate, and no automatic post-deploy verification and rollback to make shipping that patch defensible.

The problem this project addresses is therefore: *how can the full production-fault lifecycle — detect, diagnose, repair, validate, approve, deploy, and verify — be closed autonomously by a team of cooperating role-specialised agents, while keeping a human decision in the loop exactly where, and only where, the severity of the change warrants it?*

1.4 Project Objectives

Nexis set out to build, and this report documents as built, the following:

1. **A nine-agent, two-layer architecture.** A five-agent Execution Team (Architect, Backend, QA, DevOps, Data Engineer) that plans, generates minimal-diff patches, writes property-based tests, produces deployment manifests, and proposes database migrations; and a four-agent Self-Healing Loop (Sentinel, Pathfinder, Synthesiser, Validator) that detects faults statistically, walks the Neo4j [3] code dependency graph toward candidate root causes, routes the incident to the agents it actually needs, and shadow-executes every candidate patch in a locked-down Docker [10] sandbox before any human sees it. The entire pipeline is orchestrated as a durable Temporal workflow [2], so every step survives process restarts and is replayable.
2. **Severity-routed human approval.** An Approval Gate that classifies each validated patch as HIGH (mandatory human review, e.g. schema changes, security-sensitive paths, or a contract violation by an agent), MEDIUM (a 120-second window in which a human may intervene before auto-approval), or LOW (immediate auto-approval for trivial UI-only diffs), with three human decision options: Approve, Reject, or Modify — the last allowing the engineer

to edit the diff before it ships, with every terminal decision persisted as feedback data for future model improvement.

3. **Real GitOps closure.** On approval, the platform opens a genuine GitHub pull request through a GitHub App installation, and a post-deploy rollback probe watches for fresh fatal errors, invoking Argo CD's rollback client [6] to the prior known-good revision when the recovery policy demands it.
4. **A Docker preview-deploy engine.** A dedicated service that clones any connected repository, detects its technology stack, generates a working Dockerfile when none exists, builds and runs the container under strict resource limits, health-checks it over HTTP, and returns a live preview URL — and, crucially, feeds its own build or health-check failures back into the same self-healing pipeline as first-class incidents.
5. **Intelligent role recommendation.** A heuristic engine over real audit-log and login signal that recommends privilege escalations and downgrades for organisation members, each with a plain-language, evidence-backed rationale that an administrator can accept or dismiss.
6. **A complete multi-tenant platform around the loop.** A control plane with PostgreSQL row-level security [7] for tenant isolation, role-based access control, full authentication (including MFA and SSO via WorkOS [12]), billing via Stripe [13], integrations (GitHub, Slack [17], Sentry [14], Datadog [16], PagerDuty [15]), a tamper-evident audit log, and an administrative console covering incidents, approvals, agents, cost, and system health.

1.5 Scope of the Project

Nexis serves three categories of human user, distinguished by role-based access control:

- **Owner / Admin** — provisions the organisation, connects integrations, sets recovery policy, decides HIGH-severity approvals, manages members, billing, and API keys, and acts on role recommendations.
- **Engineer / Member** — works with projects and incidents day to day, observes live pipeline runs, and exercises the Approve / Reject / Modify decision on patches within their remit.
- **Viewer** — a read-only console experience, enforced by the same RBAC gating, for stakeholders who need visibility into incidents, recovery outcomes, and cost without the ability to change anything.

Beyond these, Nexis makes a deliberate conceptual departure from conventional systems analysis: the **nine autonomous agents are modelled as a distinct actor category in their own right**, not as internal implementation detail. In a typical information system, every actor in the use-case model is human; in Nexis, the Sentinel raises incidents, the Architect plans, the Backend agent writes code, and the Validator renders verdicts — each initiating and completing use cases exactly as a human actor would, subject to the same audit logging and the same policy constraints. Treating agents as first-class actors is what allows the human roles above to shrink to supervisory decisions, and this framing is carried consistently through the requirement analysis and design chapters of this report.

The scope of the delivered system is the full platform described in the objectives, evaluated end-to-end in a local environment with a locally hosted large language model via Ollama [20]. The `nexis.sh` domain is registered for the platform but a credentialed public production deployment is outside the scope of this report, as discussed honestly in the limitations.

1.6 Significance of the Project

The significance of Nexis lies in closing a loop that existing tools leave open. Detection without repair produces pager fatigue; code generation without validation produces risk; automation without a human gate produces justified distrust. Each stage of the lifecycle has been automated somewhere before — what has been missing is a system in which detect → diagnose → repair → validate → approve → deploy → verify runs as one continuous, durable, auditable workflow, with the human reduced from the transport mechanism between stages to a single, well-placed decision.

This report does not argue that claim in the abstract; it presents live evidence for it. Chapter 6 reports two genuine end-to-end recovery runs executed against a real locally hosted language model [20]: one in which the approval window elapsed and the pipeline correctly rejected the patch on timeout — the safety mechanism working as designed — and one in which a fault travelled the entire loop from detection to an approved, deployed patch in under two minutes, with per-agent token and cost figures recorded at every step. It likewise presents a live, HTTP-verified run of the preview-deploy engine building and serving a real public repository that contained no Dockerfile. A platform that can demonstrably turn a production fault into a validated, human-approved repair in the time it takes an on-call engineer to open their laptop is a meaningful step toward engineering teams spending their time building software rather than nursing it.

Chapter 2 Literature Review

2.1 Overview

Nexis is a systems-integration project: its contribution lies in composing established, well-engineered tools into a closed loop that none of them forms on its own. Intellectual honesty therefore requires a clear account of what each underlying tool already does, and where Nexis merely depends on it versus where Nexis extends it. This section surveys the relevant landscape in that spirit.

LLM coding assistants. GitHub Copilot [1] is the most widely deployed AI pair-programming tool; it completes code and, in its agent modes, drafts multi-file changes inside an editor session. It operates entirely within the developer’s authoring context: it never observes a production incident, never validates its own output against a failing system, and never deploys anything. Nexis’s Backend agent occupies the same “generate a minimal code change” niche, but is invoked by a fault-detection pipeline rather than a keystroke, and its output is machine-validated before any human sees it.

Durable workflow orchestration. Temporal [2] provides durable, replayable workflow execution with automatic retry and state persistence. Nexis uses Temporal as-is, as a dependency: the entire nine-agent RecoveryPipeline is a Temporal workflow, which is why every agent step survives process restarts and is recorded as a durable ActivityEvent. The workflow’s engineering semantics – role routing, contract-violation detection, severity classification – are Nexis’s own logic layered on top.

Graph databases for dependency analysis. Neo4j [3] is a native graph database. Nexis stores its code-dependency graph in Neo4j and the Pathfinder agent walks it from a fault symptom to candidate root-cause nodes, forwarding hop distance and node degree as evidence. Neo4j is a storage and traversal dependency; the evidence-ranking performed on the traversal results is Nexis code.

Causal inference tooling. DoWhy [4] is Microsoft Research’s library for explicit causal modelling with refutation tests. It is the methodological reference point for Nexis’s causal-inference sidecar, but it must be stated plainly that Nexis does *not* fit a DoWhy structural causal model: no interventional production data exists to fit one against. The sidecar instead implements a documented proxy

– ranking candidates by evidence-chain specificity, graph-structural proximity, textual overlap, and a scenario prior, with a transparent per-component confidence breakdown.

Observability standards. OpenTelemetry [5] standardises traces, metrics, and logs. Nexis adopts it as its instrumentation layer (exported to Prometheus, Loki, and Tempo behind Grafana). It is a dependency; Nexis adds nothing to the standard itself.

GitOps deployment. Argo CD [6] reconciles a cluster against a Git repository and exposes Sync and Rollback operations. Nexis’s DevOps agent emits Argo CD manifests, and the post-deploy rollback probe calls Argo CD’s real Rollback client when a policy-gated SLO breach is detected – integration, not extension.

Container runtimes. Docker [10] is the target API for both the Validator’s sandbox (network-isolated, read-only, capability-dropped shadow execution of every candidate patch) and the preview-deploy engine. Podman [11] was substituted in the evaluation environment as a Docker-API-compatible runtime and behaved identically; this substitution is reported honestly rather than hidden.

Property-based testing. Hypothesis [9] generates test inputs from declared properties. Nexis’s QA agent auto-generates pytest/Hypothesis suites from the incident and the Backend diff – a novel *use* of the library, with Hypothesis itself unmodified. A background QA loop replays stored suites and raises a fresh incident on a pass-to-fail regression.

Data lineage. OpenLineage [8] defines an open specification for run-level lineage events. Nexis’s Data Engineer agent emits OpenLineage-compatible RunEvents for every migration it proposes or applies; Nexis conforms to the specification rather than extending it.

The incident-response ecosystem. Sentry [14], Datadog [16], and PagerDuty [15] respectively capture application errors, aggregate observability data, and route alerts to on-call humans. All three are integrated into Nexis as webhook-driven incident *sources* feeding the same Sentinel pipeline. The Slack API [17] is used in the opposite direction: the Approval Gate pushes a decision into Slack, where an engineer can approve or reject a validated patch with one click. Supporting platform dependencies – WorkOS for SSO [12], Stripe for billing [13], PostgreSQL row-level security for tenant isolation [7], pgvector for knowledge-base retrieval [19], and Ollama [20] as the local LLM provider used for this report’s live evidence – are used as documented, without modification. Table 2.1 summarises the dependency-versus-contribution boundary.

Table 2.1: Position of surveyed tools relative to Nexus.

Tool	What it provides	Relation to Nexus
Copilot [1]	Editor-time code generation	Comparable to one agent (Backend) only
Temporal [2]	Durable workflow execution	Dependency; hosts the RecoveryPipeline
Neo4j [3]	Graph storage and traversal	Dependency; Pathfinder walks it
DoWhy [4]	Structural causal models	Reference point; proxy method used instead
OpenTelemetry [5]	Telemetry standard	Dependency; instrumentation layer
Argo CD [6]	GitOps sync/rollback	Integrated by DevOps agent and rollback probe
Docker [10]/Podman [11]	Container runtime	Dependency; sandbox and deploy engine
Hypothesis [9]	Property-based testing	Novel use: QA agent generates suites
OpenLineage [8]	Lineage specification	Conformance by Data Engineer agent
Sentry/Datadog/PagerDuty [14, 16, 15]	Error/alert capture	Webhook incident sources
Slack API [17]	Team messaging	One-click approval channel

2.2 Research Gap

Each tool above is strong within its boundary, and several are routinely combined in industry – Sentry alerting into PagerDuty, an on-call engineer opening an editor with Copilot, Argo CD deploying the eventual merge. What no single tool, and no combination in common use, provides is a system that (a) models a software engineering *team* with explicit role specialisation, and (b) closes the loop from automatic fault detection to a deployed, verified fix, with human judgement concentrated at one severity-routed decision point.

The gap is visible at every seam of the conventional tool chain. Copilot generates code but never sees a production incident; it has no notion of a fault, a blast radius, or a deployment [1]. Observability platforms such as Datadog and Sentry detect and display faults with great fidelity but never generate a candidate fix, let alone validate one [16, 14]. PagerDuty routes a human to the problem; the repair itself remains entirely manual [15]. Argo CD deploys whatever a repository contains but authors nothing [6]. Temporal can durably orchestrate any of these steps but carries no engineering semantics of its own [2]. In every case the loop is broken at the same place: between “a fault is known” and “a validated fix exists,” a human performs unaided diagnosis, authoring, testing, review, and release.

Nexus closes precisely that span, end to end. Sentinel detects a fault using a statistical-process-control rule (an EWMA baseline with a mean-plus-three-sigma breach condition), not a fixed threshold. Pathfinder walks the real Neo4j dependency graph to candidate root causes, which the causal-inference sidecar ranks with a transparent confidence breakdown. The Synthesiser classifies the scenario and selects which Execution Team agents are actually needed – and the workflow

enforces that routing, visibly skipping unselected agents. The Architect produces a structured plan whose `affected_files` contract is mechanically checked against the Backend agent’s actual diff; a violation forces HIGH severity so the change can never auto-deploy unreviewed. QA generates property-based tests from the incident and the diff [9]; the Validator executes patch and tests in a locked-down container sandbox, so no unvalidated patch ever reaches a human. Only then does the single human decision point engage – and only when severity warrants it: HIGH-severity changes require explicit review, MEDIUM-severity changes auto-approve after a 120-second window unless overridden, and trivial LOW-severity changes pass immediately. The decision itself is richer than a binary gate: the engineer may Approve, Reject, or Modify the diff before it ships, with every terminal decision persisted for future preference learning. On approval, a real GitHub pull request is opened and a post-deploy rollback probe watches for regression [6].

Two further properties distinguish this from a scripted pipeline of existing tools. First, role specialisation is structural, not cosmetic: five Execution Team agents (Architect, Backend, QA, DevOps, Data Engineer) and four Self-Healing Loop agents (Sentinel, Pathfinder, Synthesiser, Validator) have distinct inputs, outputs, and contracts, and the Architect’s plan constrains the others. Second, the loop is closed over the platform’s own infrastructure: when Nexis’s preview-deploy engine fails to build or health-check a repository, that failure is inserted as a first-class incident into the very same pipeline, and “write a working Dockerfile” becomes an ordinary repair task. To the author’s knowledge, no tool in common use today combines these properties in a single system.

2.3 Conclusion

The surveyed landscape supplies every ingredient Nexis needs – durable orchestration [2], graph traversal [3], sandboxed execution [10, 11], property-based testing [9], GitOps delivery [6], and a mature incident-capture ecosystem [14, 16, 15] – but leaves the loop between detection and repair open, to be closed by a human. Nexis’s contribution is not any single new algorithm; it is the demonstrated composition of these tools into a role-specialised multi-agent team that detects, diagnoses, repairs, validates, and deploys autonomously, engaging human judgement once, at a severity-routed gate. The remaining chapters describe how that composition was specified, designed, implemented, and tested.

Chapter 3 System Analysis and Requirements

3.1 Requirement Analysis

Nexis is a role-based, multi-tenant platform. Every piece of data in the system belongs to exactly one organisation (tenant), and every request is evaluated against both the caller’s organisation and the caller’s role within it. The requirement analysis therefore begins by identifying the actors who interact with the system boundary.

Three *human* actor types were identified:

- **Owner/Admin** — the organisation’s administrator, responsible for membership, roles, billing, policy, integrations, and the final human decision on high-stakes automated repairs.
- **Engineer/Member** — the working engineer who investigates incidents, exercises the evaluation tooling, and interacts with the recovery pipeline day to day.
- **Viewer** — a read-only observer (for example a stakeholder or an auditor) who may inspect the system’s state but may not change it.

The role model is *additive*: an Admin holds every permission an Engineer holds, and an Engineer holds every permission a Viewer holds. Consequently, the functional requirements in Section 3.2 list only the permissions each role *adds* to the tier below it, without repetition. (Internally the database role enumeration is owner/admin/member; the console additionally presents the read-only Viewer experience through RBAC gating on the frontend and API.)

Unusually for a requirement analysis, a fourth actor category must be modelled that is not human at all: the **nine-agent self-healing fleet**. Most requirement-analysis chapters enumerate only human actors, because in a conventional system every state change ultimately traces back to a person clicking something. That assumption does not hold for Nexis. The agent fleet acts autonomously inside the system boundary: it detects faults without being asked, reads and writes the same PostgreSQL, Neo4j, and object-storage data stores the human-facing console uses, and triggers real external side effects — opening GitHub pull requests, applying rollbacks through Argo CD [6], and redeploying preview containers. An actor model that omitted the fleet would misrepresent the system: roughly half of all writes to the incident, patch, and audit tables originate from agents rather than people. Treating the fleet as a first-class actor also makes the safety analysis explicit —

every autonomous capability listed in Section 3.2.4 is bounded by a corresponding control (sandboxing, severity classification, or the human approval gate) listed in Section 3.3.

The Level 0 and Level 1 use case diagrams for these four actors are presented together with the rest of the system's UML design in Section 4.2 (Figures 4.2 and 4.3), alongside the data flow diagrams that show how the same actors move data through the platform.

3.2 Functional Requirements

The functional requirements are grouped by actor. Because the role model is additive, each human tier is listed as a delta over the tier below it.

3.2.1 Owner/Admin can:

- Do everything an Engineer/Member and a Viewer can do (role additivity).
- Manage organisation membership: generate invite codes, send email invites, and remove members.
- Assign and change member roles, and act on the intelligent role-recommendation banner on the Members settings page — accepting a recommendation (which applies the role change under an optimistic-concurrency guard) or dismissing it (which starts a 30-day per-rule cool-down).
- Edit organisation settings (name, profile, and workspace-level configuration).
- Manage billing through Stripe [13]: view usage, manage payment methods, and review invoices produced by the usage-ticker and invoice-roller cron jobs; set the organisation's daily token budget on the cost tracker.
- View and export the append-only, hash-chained audit log (CSV export) and invoke its cryptographic verification endpoint to prove the chain has not been tampered with.
- Decide pending approvals from the recovery pipeline: **Approve** a validated patch, **Reject** it, or **Modify** it — edit the unified diff in place before it ships, with the edited diff persisted as an RLHF feedback example.
- Configure each project's recovery policy, including rollback-on-SLO-breach behaviour.
- Operate the Docker preview-deploy engine from a project's *Ops* tab: trigger Deploy and Stop, watch live status, open the returned preview URL, inspect build and container logs, and review deployment history.
- Install and manage integrations: the GitHub App (installation, repo listing, webhook-driven

incident creation on merged pull requests), Sentry [14], Datadog [16], and PagerDuty [15] webhook sources, Slack [17] (OAuth install plus interactive one-click approve/reject directly from a Slack message), and Argo CD [6].

- Export evaluation-harness results as CSV (an admin-only action).
- Receive the daily admin digest e-mail summarising the last 24 hours of incidents, repairs, approvals, and MTTR for the organisation.
- Monitor platform operations pages: System Health (parallel dependency probes of PostgreSQL, Redis, Neo4j, MinIO, and Temporal), webhook activity, and integration connection health.

3.2.2 Engineer/Member can:

- Do everything a Viewer can do (role additivity).
- Sign up and sign in with e-mail and password (JWT session in an HttpOnly cookie), sign in passwordlessly via magic link, or sign in through WorkOS SSO [12]; recover access through the password-reset flow (hashed, time-limited reset tokens).
- Enrol multi-factor authentication (TOTP with QR provisioning).
- Manage their own active sessions: list signed-in devices with IP address, user agent, and last-seen time (the caller's own session carries a "This device" badge), and revoke any session remotely.
- Create, list, and revoke scoped API keys (`nx_live_...` bearer tokens) for programmatic access.
- Create projects through the new-project wizard and connect them to GitHub repositories.
- Browse the incidents list and open an incident's detail page, where the nine-agent pipeline timeline streams live over Server-Sent Events, including per-agent status, token counts, and cost.
- Inspect each agent's structured payloads (for example the Backend agent's raw generated diff) from the timeline.
- Trigger fault-injection runs from the Live-Demo console page, which presents 27 curated scenario cards drawn from a broader fixture-scenario taxonomy (one scenario is fully wired end-to-end today; the remainder are labelled "Coming soon").
- Run the evaluation harness: side-by-side OpenAI-versus-Ollama provider comparisons over fixture incidents, with persisted transcripts and cost figures.
- Query the knowledge base (pgvector-backed retrieval [19]) and view its retrieval status.
- Complete the post-incident NASA-TLX workload survey [18], the platform's instrument for

measuring operator cognitive load.

- Use the operational consoles: the agents fleet page and per-agent detail views, the workflows list, the activity stream, the validator sandbox page, and the recovery-pipeline view.

3.2.3 Viewer can:

- Sign in and view the dashboard: KPI tiles (MTTR, recovery success rate, open incidents, mean tokens per run), the incidents-over-time chart, the system status panel, the projects health grid, and the recent activity feed.
- Read the incidents list and incident detail timelines, including the live SSE stream, without any decision or mutation controls.
- Read project pages (overview, activity, and status) and the performance page.
- View the agents, workflows, and activity-stream pages in read-only form.
- Take no mutating action of any kind: no approvals, no deployments, no settings changes, no key or session management beyond their own account.

3.2.4 The 9-Agent Fleet autonomously:

- **Detects** faults (Sentinel): an always-on detector applies two rules — a per-row fatal-level trigger, and a statistical-process-control spike rule that maintains a per-organisation EWMA mean and variance baseline and fires on a breach of mean $+3\sigma$, with a Poisson floor for low-count noise and an absolute-floor warm-up fallback.
- **Diagnoses** root causes (Pathfinder): walks the Neo4j code dependency graph [3] outward from the fault symptom and forwards candidate root-cause nodes, annotated with hop distance and degree, to the causal-inference sidecar, which ranks candidates by evidence-chain specificity, structural proximity, textual overlap, and a scenario prior, returning a per-component confidence breakdown [4].
- **Classifies and routes** (Synthesiser): assigns the incident a scenario (regex fast path with LLM fallback) and computes an ordered `selected_agents` list; the workflow enforces this routing, skipping non-selected execution agents with a visible “skipped” timeline state.
- **Plans** the repair (Architect): decomposes the incident into `plan_steps`, `affected_files`, and a `risk_level`, and serves as a contract other agents may not silently violate — if the generated diff touches a file outside `affected_files`, the workflow flags a contract violation and forces HIGH severity so the patch can never auto-deploy unreviewed.
- **Codes** the fix (Backend): generates an LLM-guided minimal unified diff — the smallest

valid change rather than a full-file rewrite.

- **Tests** the fix (QA): auto-generates property-based test suites (pytest/Hypothesis [9]) from the incident and the diff; a background continuous-QA loop replays previously generated suites against the sandbox and raises a fresh incident on any pass-to-fail regression, closing a second, independent loop.
- **Packages** the deployment (DevOps): emits CI/CD and Argo CD manifests [6] and drives post-deployment health monitoring.
- **Handles data changes** (Data Engineer): proposes paired forward and reverse SQL migrations, runs a proactive schema-drift detector that introspects `information_schema` against a captured baseline and raises a real incident on drift, and emits OpenLineage-compatible run events [8] for every migration proposal and application.
- **Validates** every candidate patch (Validator): executes it in a locked-down Docker sandbox [10] (no network, read-only filesystem, all capabilities dropped, PID-limited, no-new-privileges), runs the QA-generated property tests, and reports pass/fail and coverage; no unvalidated patch ever reaches the approval gate.
- **Classifies severity** at the approval gate: HIGH for schema-drift scenarios, sensitive-path matches (SQL, migrations, auth, security, crypto), unknown scenarios, or contract violations; MEDIUM otherwise, with a 120-second auto-approve countdown unless a human intervenes; LOW (tiny UI-only diffs) auto-approves immediately.
- **Deploys** approved patches (GitOps): opens a real GitHub pull request through a GitHub App installation (blob→tree→commit→ref→PR); when the triggering incident originated from the preview-deploy engine, a successful merge additionally triggers an automatic redeploy.
- **Rolls back** on regression: a post-deploy probe watches for a fresh fatal incident on the same organisation and service within a bounded window and, when the project policy permits, invokes Argo CD's rollback client to restore the prior known-good revision.
- **Records** every step as a Temporal-durable activity event [2] and streams it live to the incident detail page over Server-Sent Events.
- **Feeds the deploy engine's failures back into itself**: when a preview deployment fails to build or pass its health check, the deploy engine files a real incident (`source: "deploy_engine"`) into the same Sentinel-to-pipeline path as every other incident source, making a broken build a self-healing target like any production fault.

Table 3.1: Summary of non-functional requirements and their realisation.

Requirement	Realisation in Nexis
Multi-tenancy	PostgreSQL Row-Level Security on essentially every table [7]
Durability	Temporal-orchestrated workflows survive crashes and restarts [2]
Security	Sandboxed patch execution, encrypted patch storage, hashed tokens, hash-chained audit log
Scalability	Stateless Go services, queue-decoupled workers, parallel dependency probes
Responsiveness	Server-Sent Events stream the live incident timeline
Usability	Role-scoped console, one-click approvals, plain-language rationales

3.3 Non-Functional Requirements

Multi-tenancy and isolation. Tenant isolation must not depend on application code remembering to add a `WHERE org_id = ...` clause. Nexis enforces isolation in the database itself using PostgreSQL Row-Level Security policies [7]: a request-scoped transaction binds `app.current_org_id` for every request, and the database refuses to return or mutate rows belonging to any other organisation, regardless of what the application layer asks for. This applies uniformly to human requests and to agent writes.

Durability of long-running work. A recovery run spans many minutes, crosses several services, waits on a human decision, and must survive process crashes and restarts without losing its place. All pipeline state therefore lives in Temporal [2], a durable workflow engine: every agent step is a journaled activity, and a resumed worker replays deterministically to exactly where it stopped. An in-memory orchestrator was rejected because a single crash mid-repair would strand an incident in an unknown state.

Security. Autonomously generated code is treated as untrusted by construction. Candidate patches execute only inside a Docker sandbox [10] with networking disabled, a read-only filesystem, all Linux capabilities dropped, a process-count limit, and `no-new-privileges`. Patch blobs are stored encrypted in object storage. Password-reset and API tokens are stored only as hashes (reset tokens are 32-byte random values, SHA-256 hashed, with a 30-minute TTL and a non-leaky 202 response). The audit log is append-only and hash-chained, so any tampering with a past entry breaks the chain and is detectable through the verification endpoint.

Scalability. The control plane, validator, GitOps, and deploy-engine services are independent, stateless Go modules that scale horizontally; Temporal decouples workflow load from any single process; Redis provides rate limiting and caching; and the System Health page’s dependency probes fan out in parallel under a bounded timeout so a slow dependency cannot stall the page.

Real-time responsiveness. An engineer watching a live repair must see agent transitions as they happen, not on a polling delay. The incident detail page subscribes to a Server-Sent Events stream, and every Temporal-recorded activity event is pushed to the browser within moments of occurring — the live-demo runs in Chapter 6 were observed this way.

Usability. The console must make a genuinely complex system operable: role-scoped navigation shows each user only what they can act on, approvals are a one-click decision with the full diff in view, severity and countdown state are always visible, and role recommendations carry a plain-language rationale naming the concrete evidence behind them.

3.4 System Feasibility

3.4.1 Technical Feasibility

Nexis is deliberately polyglot, with each language chosen for the workload it carries rather than for uniformity. The four backend services (control-plane, validator, gitops, deploy-engine) are independent Go modules: Go’s static binaries, low-overhead concurrency, and first-class Docker API client suit long-lived services that juggle webhooks, container lifecycles, and parallel health probes. The console is Next.js/React with TypeScript, which is the practical choice for a large role-gated UI with live SSE streams, embedded Monaco diff editing, and chart-heavy dashboards. The causal-inference sidecar is Python/FastAPI because the scientific-Python ecosystem (and the Hypothesis property-testing stack [9] the QA agent depends on) lives there.

The data layer pairs PostgreSQL 17 with pgvector [19] — one engine provides transactional tenant data, row-level security [7], and vector retrieval for the knowledge base, avoiding a separate vector database — while Neo4j [3] holds the code dependency graph, because root-cause diagnosis is a graph-traversal problem (hop distances, degrees, paths from symptom to cause) that a relational schema expresses poorly. Temporal [2] was proven in practice: both live recovery runs reported in Chapter 6 executed durably end-to-end. Every component is technology the team could run locally; the entire platform was built and verified on a single development machine, with Podman [11] standing in for Docker [10] as an API-compatible runtime where Docker itself was unavailable.

3.4.2 Economic Feasibility

The platform's dependency set is open source end to end — PostgreSQL, Neo4j Community, Redis, MinIO, Temporal, OpenTelemetry [5], and the Grafana observability stack — so no licence cost attaches to development or evaluation. The one economically significant input, LLM inference, is dual-path by design: production targets OpenAI's hosted models, while Ollama [20] serves fully supported local models (`gpt-oss:20b`, `qwen2.5-coder:14b`) as a zero-marginal-cost development and evaluation path. This is not a theoretical fallback: every live end-to-end run in this report was driven by the local Ollama provider, meaning the complete system — including its most expensive component — was exercised without incurring any per-token cost. The cost tracker, per-organisation token ledger, and daily budget controls exist precisely so that the paid path remains bounded and observable when it is used.

3.4.3 Operational Feasibility

Operationally, the system is designed to be run by the people it serves. The console presents role-based dashboards: an admin lands on organisation-wide KPIs, approvals, and health; an engineer lands on incidents and the live pipeline; a viewer sees a read-only picture of the same truth. High-stakes autonomy is reduced to a one-click human decision (Approve/Reject/Modify), optionally taken directly from Slack [17] without opening the console at all. Severity classification means routine low-risk fixes demand no attention while sensitive changes always do. The NASA-TLX instrument [18] is built in so that the operator-workload claim can eventually be measured rather than asserted. Together these mean adopting Nexis does not require a dedicated operations team; the intended day-to-day operator is an ordinary engineer with a browser.

Chapter 4 System Design and UML Diagrams

This chapter presents the design of Nexis at four complementary levels of abstraction. Section 4.1 gives the physical view: the service topology of the deployed system. Section 4.2 gives the functional view through Level 0 and Level 1 use case diagrams, covering the three human roles and the nine autonomous agents. Section 4.3 gives the data view through a context-level data flow diagram and two Level 1 decompositions — one for the Owner/Admin control surface and one for the self-healing loop itself. Section 4.4 closes with a UML sequence diagram that traces a single complete recovery run, message by message, from fault detection to post-deploy verification. The self-healing loop is deliberately given the most space, because it is the central contribution of the project.

4.1 System Architecture Overview

Figure 4.1 shows the full service topology of Nexis. The diagram is organised into horizontal layers, read top to bottom: clients, the control-plane together with the LLM spine, the sidecar services, the data and state layer, and the external integrations.

Clients. The topmost layer contains every entry point into the platform: the web console (`apps/web`, Next.js 16), inbound GitHub webhooks, inbound Sentry/Datadog/PagerDuty alert webhooks [14, 16, 15], Slack interactivity callbacks [17], and plain HTTP clients (`CLI/curl`) that address the deploy-engine and the public API directly. All of these converge on the control-plane; no client talks to a data store or an external integration directly.

Control-plane and LLM spine. The second layer is the heart of the system. The control-plane is a single Go service (`:8080`) that hosts the HTTP API (chi router, with authentication, RBAC, row-level-security binding, and rate limiting), the always-on Sentinel detector goroutine (EWMA + 3σ spike detection), and the cron scheduler (daily digest, QA continuous loop, billing tickers, schema-drift detection). Inside it, the Temporal RecoveryPipeline workflow [2] orchestrates the nine-agent self-healing loop as a durable workflow: `detect` → `diagnose` → `plan` → `design` → `code` → `test` → `package/migrate` → `approve` (severity-routed) → `deploy` → `verify` → `rollback` if needed. Alongside it sits the LLM spine: OpenAI (gpt-4o family) as the production default and Ollama [20] (local `gpt-oss:20b` and `qwen2.5-coder:14b`) as a fully supported alternate provider,

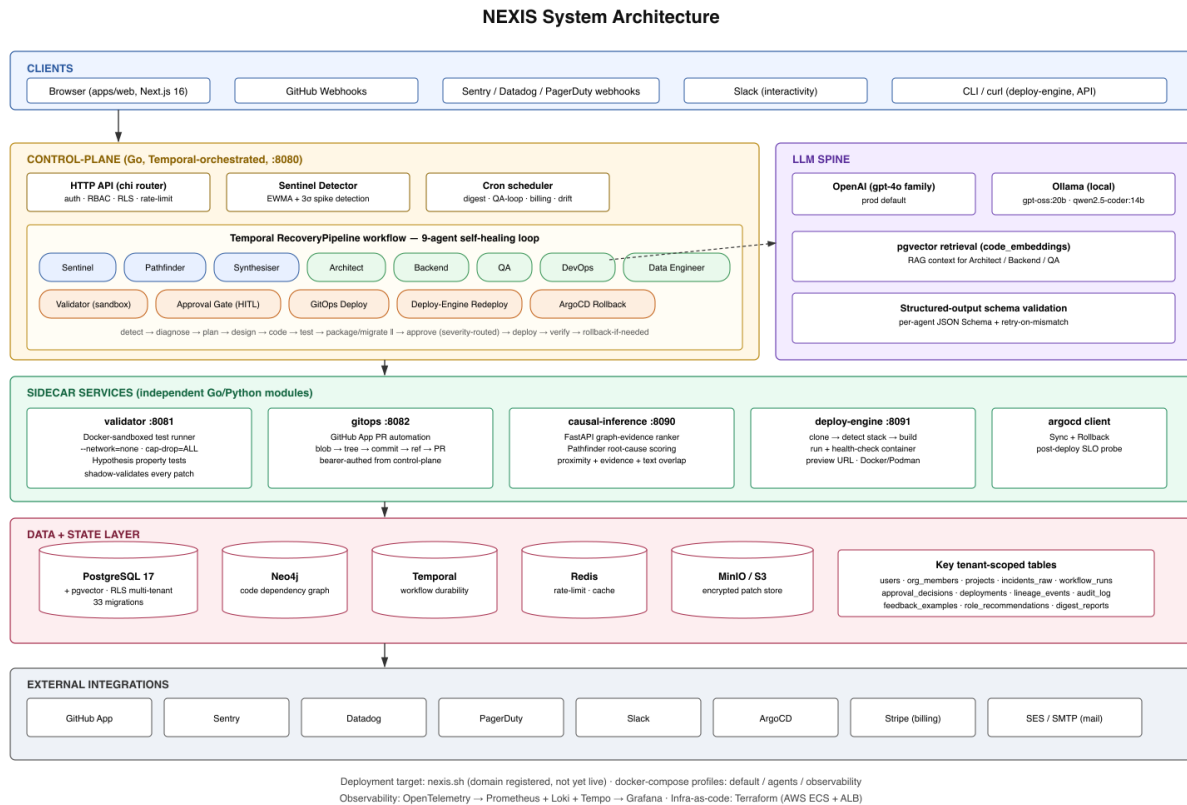


Figure 4.1: NEXIS System Architecture (full services topology across the client, control-plane/LLM, sidecar, data, and integration layers).

a pgvector retrieval store [19] that supplies code-context passages to the Architect, Backend, and QA agents, and a structured-output stage that validates every agent completion against a per-agent JSON Schema and retries on mismatch.

Sidecar services. The third layer holds the independently deployable modules. Four are Go services and one is Python:

- **validator** (:8081) — the Docker-sandboxed test runner [10] that shadow-executes every candidate patch under `-network=none, -read-only, -cap-drop=ALL` and runs the QA-generated Hypothesis property tests [9];
- **gitops** (:8082) — GitHub App pull-request automation (blob → tree → commit → ref → PR), bearer-authenticated from the control-plane;
- **causal-inference** (:8090) — the Python/FastAPI graph-evidence ranker that scores Pathfinder’s candidate root causes by evidence-chain specificity, structural proximity, and textual overlap (an honest statistical proxy rather than a fitted structural causal model [4]);
- **deploy-engine** (:8091) — clone, stack detection, Dockerfile generation, build, and health-checked container run for preview deployments;
- **argocd client** — the Sync/Rollback integration [6] used by the post-deploy SLO probe.

Data and state layer. The fourth layer contains PostgreSQL 17 with pgvector, made multi-tenant through row-level security policies [7] across 33 migrations; Neo4j [3] holding the code dependency graph that Pathfinder walks; Temporal’s own persistence for workflow durability; Redis for rate limiting and caching; and MinIO/S3 as the encrypted patch store. The key tenant-scoped tables — `users`, `org_members`, `projects`, `incidents_raw`, `workflow_runs`, `approval_decisions`, `deployments`, `lineage_events`, `audit_log`, `feedback_examples`, `role_recommendations`, and `digest_reports` — are listed in the diagram and reappear as the data stores of the DFDs in Section 4.3.

External integrations. The bottom layer lists every third-party system Nexis touches: the GitHub App, Sentry, Datadog, PagerDuty, Slack, ArgoCD, Stripe billing [13], and SES/SMTP mail. Observability across all layers follows the OpenTelemetry pipeline [5] into Prometheus, Loki, and Tempo, visualised in Grafana, and infrastructure is declared as Terraform code targeting AWS ECS behind an ALB.

The architecture directly reflects the polyglot stack described in Chapter 3: Go for every latency- and concurrency-sensitive service (control-plane, validator, gitops, deploy-engine), Python for the

statistics-oriented causal-inference sidecar, and TypeScript for the web console. The container runtime boundary is the Docker Engine API, which Podman [11] satisfies identically — a property that was exercised in practice, since the evaluation environment for this report ran the validator sandbox and deploy-engine on Podman.

4.2 Use Case Diagram

4.2.1 Level 0 Use Case Diagram

Figure 4.2 shows the system at its coarsest functional granularity: a single system boundary, three human actors on the left, and the external systems and data stores on the right.

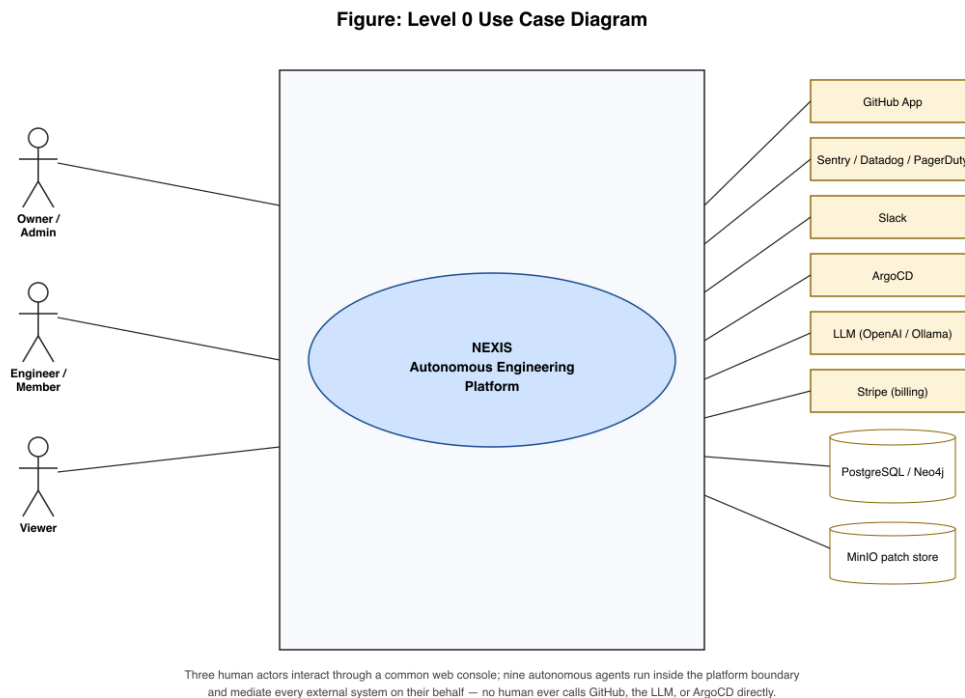


Figure 4.2: Level 0 Use Case Diagram (three human actors and the mediated external systems).

The three human actors correspond to the platform’s role-based access model. The **Owner/Admin** administers the organisation: membership, roles, integrations, recovery policy, billing, and — most importantly — the approval gate through which every autonomous repair must pass. The **Engineer/Member** performs day-to-day engineering work on the platform: running demo scenarios, browsing incidents, inspecting agent runs, and participating in the approval flow. The **Viewer** is

a read-only experience surfaced through RBAC gating in the console; it can observe dashboards, incidents, and system health but can change nothing.

On the right-hand side sit the external entities: the GitHub App, the alert sources (Sentry, Datadog, PagerDuty), Slack, ArgoCD, the LLM provider (OpenAI or Ollama), Stripe billing, and the platform's own stores (PostgreSQL/Neo4j and the MinIO patch store). The essential property the diagram expresses is mediation: every line from an actor terminates at the system boundary, and every line to an external system originates from it. No human ever calls GitHub, the LLM, or ArgoCD directly — the nine autonomous agents operate entirely inside the boundary and act on the external systems on the humans' behalf.

4.2.2 Level 1 Use Case Diagram

Figure 4.3 decomposes the single Level 0 bubble into the concrete use cases of each actor. The diagram uses colour to group use cases by the minimum role required: blue for Owner/Admin, orange for the shared approval flow, green for Engineer/Member, purple for Viewer, and purple-outlined boxes on the right for the autonomous nine-agent fleet.

Owner/Admin (blue). The administrative surface covers fifteen use cases: managing members and roles; accepting or dismissing role recommendations produced by the heuristic recommender; connecting integrations; creating and configuring projects; setting the recovery policy and SLO; toggling the kill switch; issuing invites and API keys; managing billing and plan; exporting the RLHF training data (the JSONL feedback export); viewing the audit log; triggering a manual Sentinel run; exporting evaluation-harness and audit CSVs; revoking sessions; resetting passwords; and enrolling or verifying MFA.

Shared approval flow (orange). Four use cases are shared between Owner/Admin and Engineer/Member, because both roles participate in reviewing autonomous work: approving, rejecting, or modifying a proposed patch; watching the live recovery timeline over Server-Sent Events; viewing the patch diff in the Monaco viewer; and deploying or stopping a preview container from the project's Ops tab.

Engineer/Member (green). The engineering surface covers running a Live-Demo scenario, browsing the incidents list, inspecting agent runs, submitting the NASA-TLX workload survey [18], browsing evaluation harness runs, querying the knowledge base, viewing cost and performance pages, managing the member's own API keys, updating profile and preferences, managing the member's own sessions and devices, and the account lifecycle itself (sign up, sign in, MFA verification).

Figure: Level 1 Use Case Diagram — NEXIS Platform

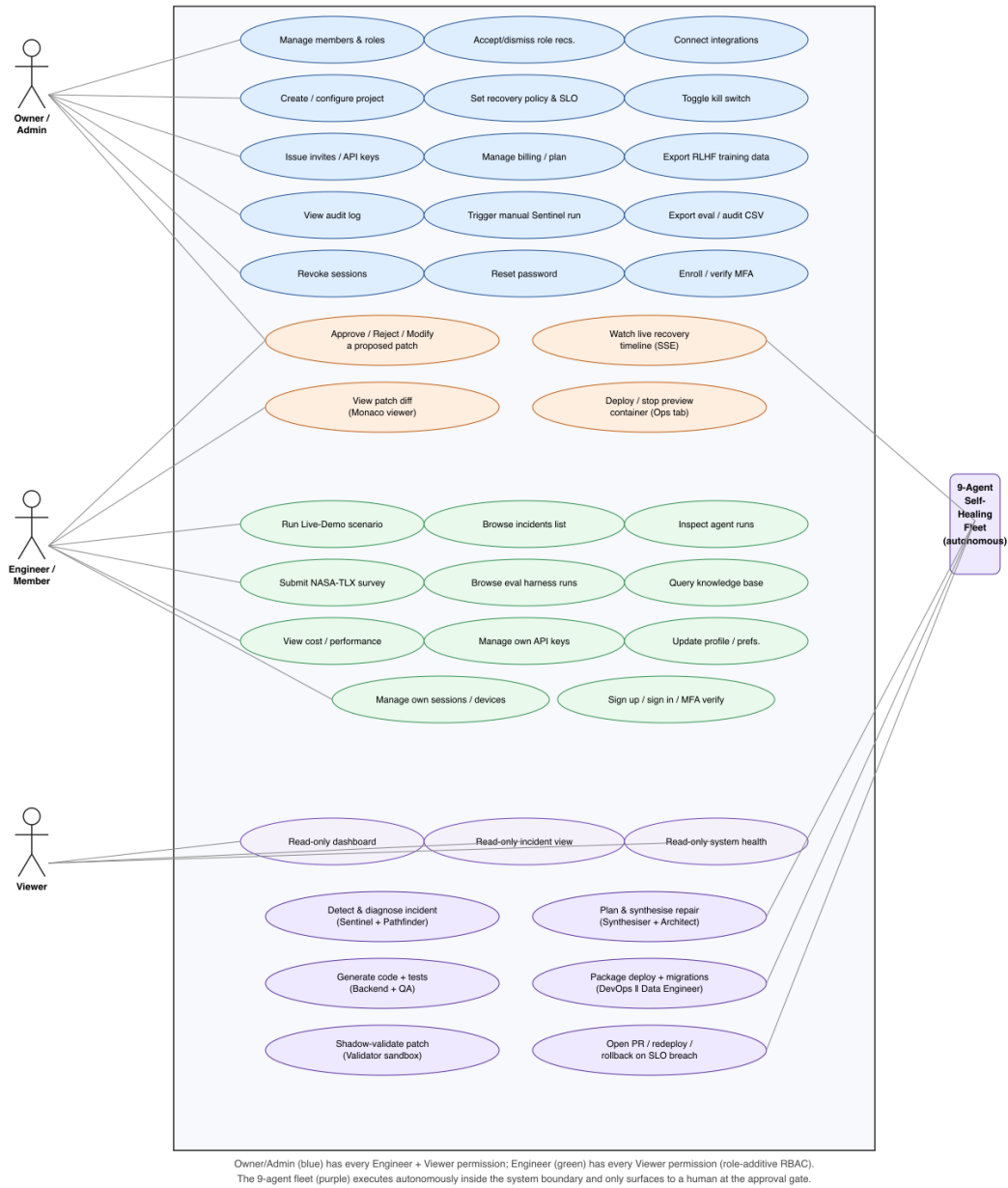


Figure 4.3: Level 1 Use Case Diagram (per-actor use case groups and the autonomous 9-agent fleet).

Viewer (purple). The Viewer holds exactly three use cases: read-only dashboard, read-only incident view, and read-only system health.

The 9-agent fleet (purple-outlined, right). The fleet appears as a single autonomous actor whose six grouped use cases summarise the recovery pipeline: detect and diagnose an incident (Sentinel + Pathfinder); plan and synthesise the repair (Synthesiser + Architect); generate code and tests (Backend + QA); package deployment and migrations (DevOps in parallel with Data Engineer); shadow-validate the patch (Validator sandbox); and open the pull request, redeploy, or roll back on an SLO breach (GitOps deploy and the rollback probe). Its only connection to a human use case is the approval gate — the fleet executes autonomously inside the boundary and surfaces to a person exactly once per run.

The permission model is *role-additive*: Owner/Admin holds every Engineer and Viewer permission, and Engineer holds every Viewer permission. This is why the diagram draws Owner/Admin connected into the orange shared group as well as the blue group — a higher role never loses a capability that a lower role has, it only gains administrative ones. The read-only Viewer experience is enforced by RBAC gating in the console even though the underlying database role enumeration is owner/admin/member, a design decision discussed in Chapter 3.

4.3 Data Flow Diagram

4.3.1 DFD Level 0 (Context Diagram)

Figure 4.4 is the context diagram: the entire platform (control-plane plus the nine-agent loop) as a single process, its external entities, and the three top-level data stores.

The data stores are labelled D1–D3. **D1, the tenant database (PostgreSQL)**, receives every organisational record — users, memberships, projects, incidents, workflow runs, decisions, and audit entries — under row-level security. **D2, the code graph (Neo4j)**, holds the dependency graph of each connected repository and is read by Pathfinder during diagnosis. **D3, the patch store (MinIO)**, receives the encrypted diff blob produced by each recovery run.

Four external entity flows cross the boundary. The **user** (any of the three roles) sends logins, commands, and approval decisions inward and receives console views and notifications outward. **GitHub/GitOps** receives pull-request creation requests (and Dockerfile reads for the deploy-engine) and returns PR URLs and commit SHAs. The **alert sources** (Sentry, Datadog, PagerDuty, Slack) send fault and alert webhooks inward — they are producers of incidents, not consumers of data.

Figure: DFD Diagram Level 0

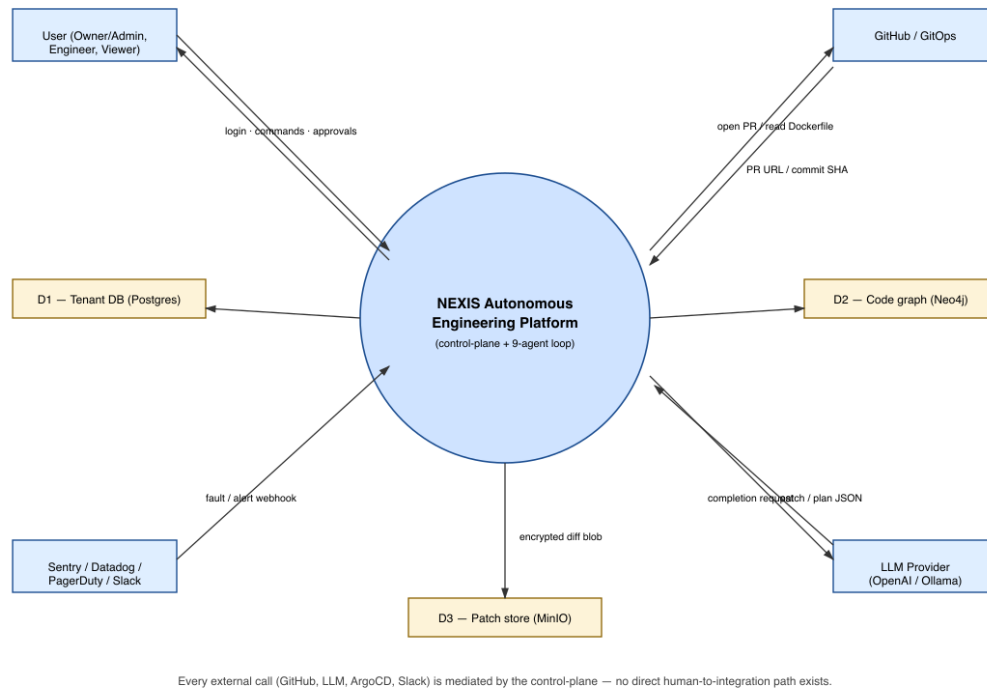


Figure 4.4: DFD Level 0 (context diagram with external entities and top-level data stores).

The **LLM provider** (OpenAI or Ollama) receives completion requests and returns patch and plan JSON. As at Level 0 of the use case model, every one of these flows is mediated by the platform process; there is no direct path from a human to an integration.

4.3.2 DFD Level 1 — Owner/Admin

Figure 4.5 decomposes the administrative side of the platform into twelve numbered processes and the eleven tenant-scoped data stores they touch.

Table 4.1 summarises each process and the primary data store it writes to.

Walking the diagram briefly: process 1.0 authenticates the admin and binds the request-scoped RLS context; 2.0 and 3.0 together form the membership surface, with the role recommender writing its suggestions to `role_recommendations` and the accept action flowing back through 2.0 into `org_members`; 4.0 and 5.0 configure what the autonomous loop is allowed to act on; 6.0 records every terminal approval decision; 7.0 and 8.0 handle billing and the append-only, hash-chained audit trail; 9.0 fans out health probes to Postgres, Redis, Neo4j, MinIO, and Temporal in parallel and persists nothing; 11.0 manages sessions and MFA; and 12.0 is the safety brake that

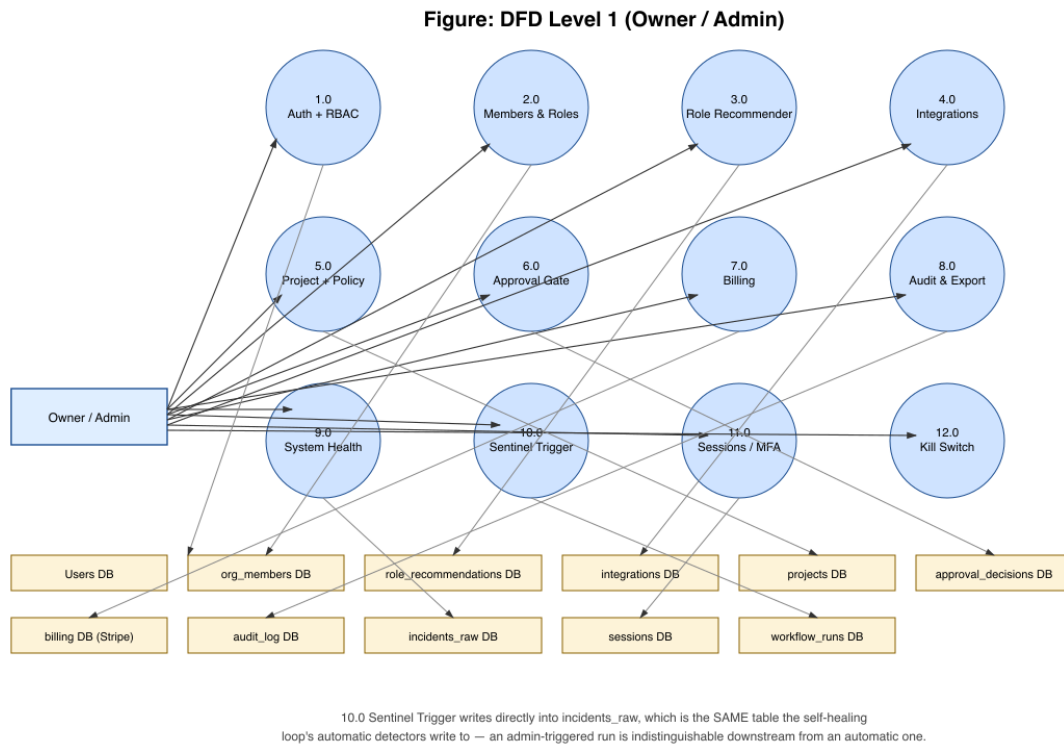


Figure 4.5: DFD Level 1 — Owner/Admin (twelve administrative processes and their tenant-scoped data stores).

Table 4.1: Owner/Admin DFD Level 1 processes and their primary data stores.

Process	Responsibility	Primary store written
1.0 Auth + RBAC	login, JWT session issue, role checks	users, sessions
2.0 Members & Roles	invite, promote, demote, remove	org_members
3.0 Role Recommender	heuristic promote/downgrade suggestions	role_recommendations
4.0 Integrations	connect GitHub/Slack/Sentry/ArgoCD	integrations
5.0 Project + Policy	project config, recovery policy, SLO	projects
6.0 Approval Gate	approve / reject / modify decisions	approval_decisions
7.0 Billing	Stripe usage ticker, invoices, plan	billing tables (Stripe)
8.0 Audit & Export	hash-chained audit writes, CSV export	audit_log
9.0 System Health	parallel dependency probes (read-only)	— (no persistent write)
10.0 Sentinel Trigger	admin-initiated detection run	incidents_raw
11.0 Sessions / MFA	session list/revoke, TOTP enrolment	sessions
12.0 Kill Switch	suspend autonomous recovery per project	workflow_runs (halt), audit_log

suspends autonomous recovery for a project.

The most important flow in the diagram is process **10.0, the Sentinel Trigger**. When an admin manually triggers a detection run, the process writes directly into `incidents_raw` — the *same* table the self-healing loop’s automatic detectors write to. There is no separate “manual” code path, no parallel queue, and no privileged shortcut: an admin-triggered run is indistinguishable downstream from an automatic one, and flows through the identical Temporal pipeline, validator sandbox, and approval gate. This design decision keeps the system’s behaviour uniform and means every guarantee established for the autonomous path (durability, validation, severity routing, auditability) holds for manual runs for free.

4.3.3 DFD Level 1 — Self-Healing Loop

Figure 4.6 decomposes the nine-agent loop itself into twelve numbered processes, coloured by layer: blue for the detection and decision agents at the head of the pipeline, green for the Layer-1 execution team, and orange for the validation, approval, deployment, and verification tail.

The pipeline reads end-to-end as follows:

1. **1.0 Sentinel** receives fault signals — webhook payloads from Sentry/Datadog/PagerDuty or its own EWMA $+3\sigma$ spike rule over the incident stream — and writes the normalised incident into `incidents_raw`.
2. **2.0 Pathfinder** walks the Neo4j code graph outward from the fault symptom and forwards candidate root-cause nodes, with hop distance and degree metadata, to the causal-inference sidecar for graph-evidence ranking.
3. **3.0 Synthesiser** classifies the incident into a scenario (regex fast path with an LLM fallback) and computes the ordered `selected_agents` list that the workflow subsequently enforces.
4. **4.0 Architect** produces the structured plan — `plan_steps`, `affected_files`, `risk_level` — that acts as the contract every downstream agent is checked against.
5. **5.0 Backend** generates the minimal unified diff, guided by the LLM provider (shown as a dashed structured-JSON completion flow into the green processes).
6. **6.0 QA** generates property-based tests from the incident and the Backend diff.
7. **7.0 DevOps** emits the CI/CD and ArgoCD manifests for the change.

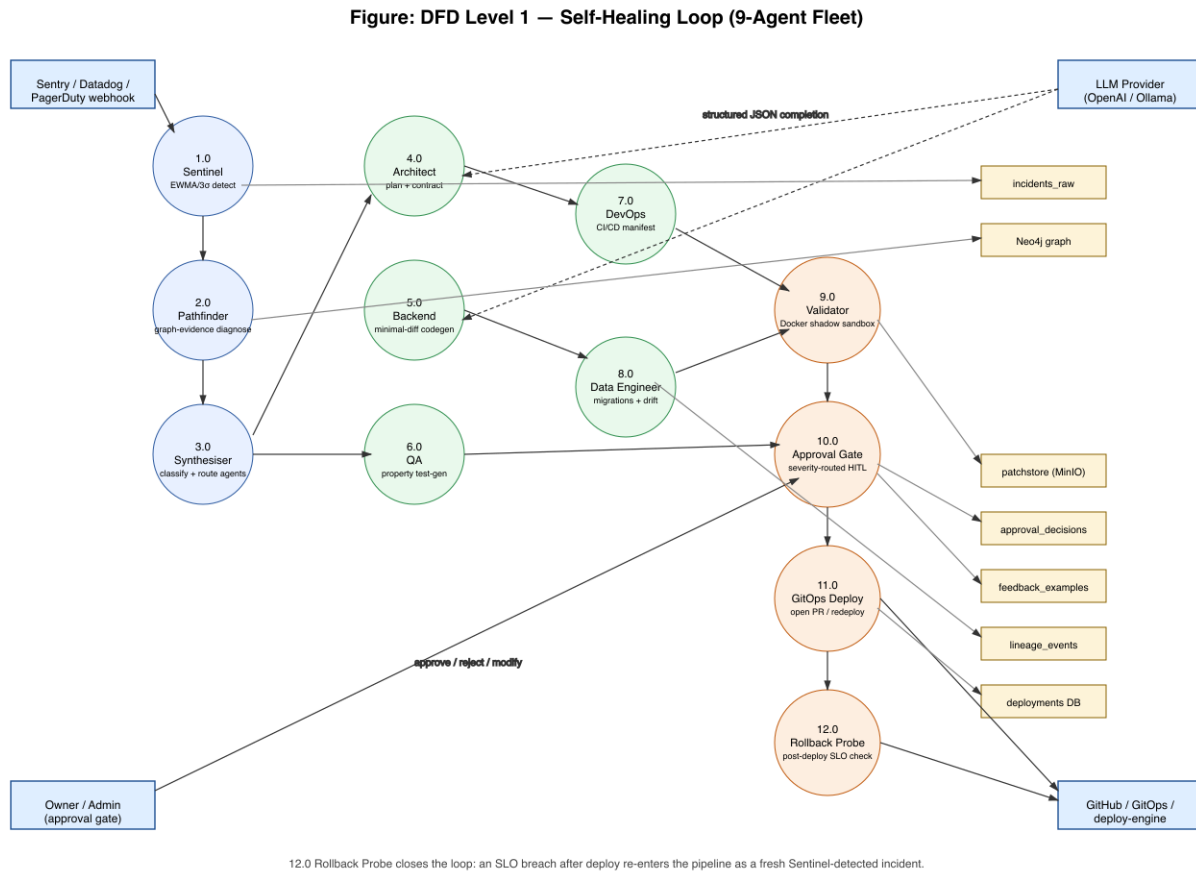


Figure 4.6: DFD Level 1 — Self-Healing Loop (the 9-agent fleet as twelve numbered processes with their data stores).

8. **8.0 Data Engineer** proposes forward and reverse migrations where schema changes are involved and emits OpenLineage-compatible run events [8] into `lineage_events`.
9. **9.0 Validator** shadow-executes the patch in the Docker sandbox, runs the QA-generated test suite, and stores the encrypted candidate diff in the MinIO patch store.
10. **10.0 Approval Gate** routes the validated patch by severity to the Owner/Admin external entity (the approve/reject/modify flow at the bottom left of the diagram) and records the terminal decision in `approval_decisions` and `feedback_examples`.
11. **11.0 GitOps Deploy** opens the real GitHub pull request (writing to the `deployments` store and out to the GitHub/GitOps/deploy-engine external entity), and redeploys through the deploy-engine when the incident's source was a preview deployment failure.
12. **12.0 Rollback Probe** watches `incidents_raw` during a bounded post-deploy window for a fresh fatal on the same organisation and service.

The diagram's closing annotation states the property that gives the project its title: **the loop is closed**. If process 12.0 observes an SLO breach after a deployment, it does not raise an alert into a separate escalation channel — the breach lands in `incidents_raw` as a fresh fatal, exactly as any external fault would, and re-enters the pipeline at process 1.0 as a new Sentinel-detected incident (with an ArgoCD rollback to the prior known-good revision when the project's `RollbackOnSLOBreach` policy is set). Recovery, verification, and re-detection form a single cycle over one shared data path.

4.4 Self-Healing Loop Sequence Diagram

Figure 4.7 is the UML sequence diagram of one complete recovery run. All nine agents appear as lifelines, together with the fault source, the Validator sandbox, the Admin, and the GitOps and Deploy services. Eighteen numbered messages trace the run in temporal order; each corresponds to a named Temporal activity, and every one of them is recorded as a durable `ActivityEvent` and streamed to the incident timeline over Server-Sent Events — which is why the live screenshots in Chapter 5 show this exact message order as a per-agent timeline.

The run proceeds as follows:

1. **Message 1 — fault arrival.** A webhook or statistical spike (fanned in from any incident source) reaches Sentinel. The resulting `incidents_raw` row is what the `RecoveryPipeline` trigger polls, so workflow start is decoupled from signal arrival.

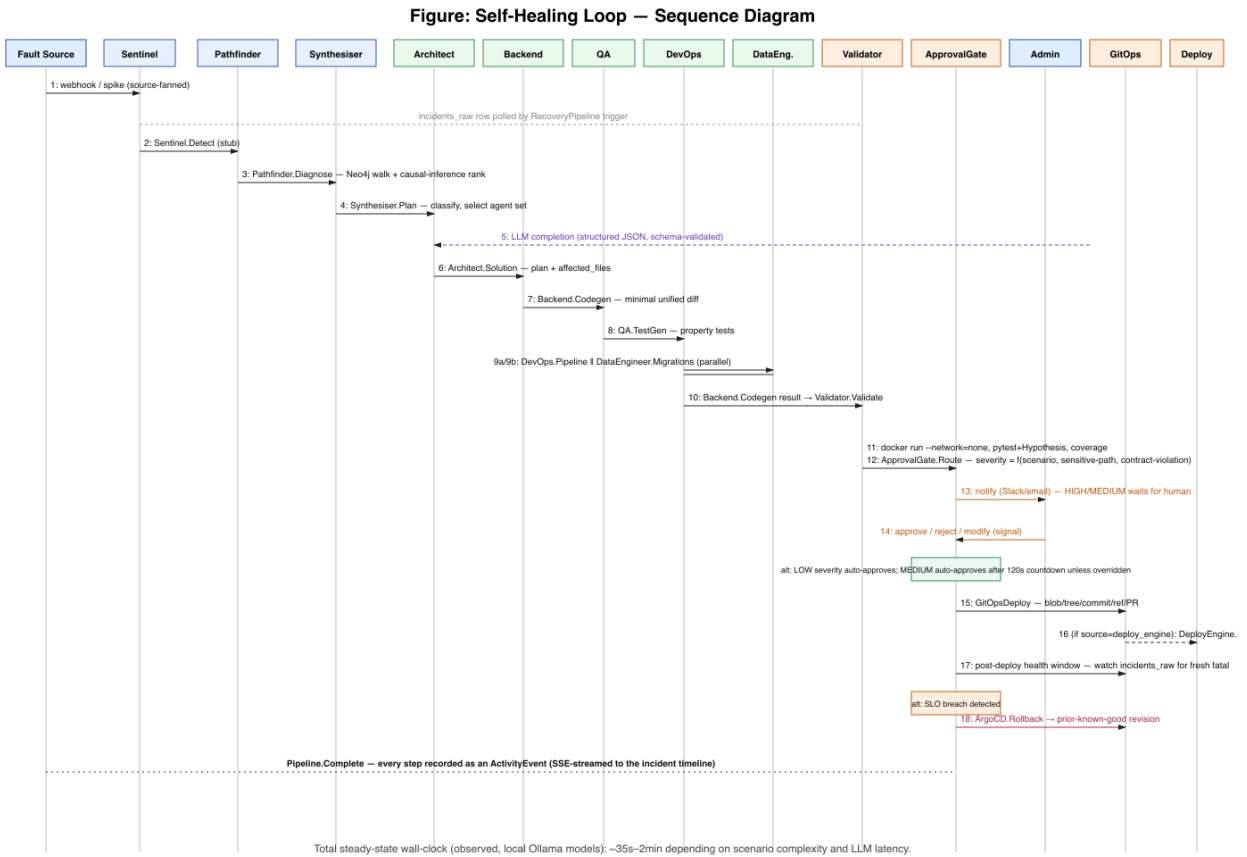


Figure 4.7: Self-Healing Loop Sequence Diagram (one full recovery run, 18 numbered messages across all nine agent lifelines).

2. **Message 2 — Sentinel.Detect.** The first workflow activity re-records the detection inside the durable workflow. It is marked as a stub in the diagram because the detection logic itself already ran in the always-on detector goroutine; the activity exists so that the workflow timeline owns the provenance of the run.
3. **Message 3 — Pathfinder.Diagnose.** Pathfinder walks the Neo4j dependency graph from the symptom and submits the candidates to the causal-inference sidecar for ranking.
4. **Message 4 — Synthesiser.Plan.** The incident is classified into a scenario and the agent set for this run is selected; non-selected Layer-1 agents will later appear as explicitly “skipped” in the timeline rather than silently omitted.
5. **Message 5 — LLM completion.** Drawn as a dashed asynchronous flow spanning the agent lifelines: every reasoning agent obtains its output as a structured JSON completion that is validated against that agent’s schema, with a retry on mismatch.
6. **Message 6 — Architect.Solution.** The structured plan with `plan_steps` and `affected_files` is produced; this becomes the contract for the rest of the run.
7. **Message 7 — Backend.Codegen.** The minimal unified diff is generated. If the diff touches any file outside the Architect’s `affected_files`, the workflow flags a contract violation, which forces HIGH severity at the approval gate.
8. **Message 8 — QA.TestGen.** Property-based tests are generated from the incident and the diff.
9. **Messages 9a/9b — DevOps.Pipeline || DataEngineer.Migrations.** The only parallel pair in the pipeline: CI/CD and ArgoCD manifests are prepared concurrently with the forward and reverse migration proposal, since neither depends on the other’s output.
10. **Message 10 — hand-off to Validator.Validate.** The Backend result, together with the QA suite, is passed to the Validator.
11. **Message 11 — sandboxed execution.** The Validator runs the patch in the locked-down container (`-network=none`, `-read-only`, `-cap-drop=ALL`, `-pids-limit=128`, `-security-opt=no-new-privileges`), executes the `pytest/Hypothesis` suite, and reports pass/fail and coverage. No unvalidated patch can proceed past this point.
12. **Message 12 — ApprovalGate.Route.** Severity is computed as a function of the scenario, sensitive-path matches, and contract violations: **HIGH** for the schema-drift scenario, any diff touching a sensitive path glob (`*.sql`, `migrations/`, `auth/`, `security/`, `crypto/`), an unknown scenario, or a contract violation; **MEDIUM** for everything else; **LOW** for tiny UI-only diffs.

13. **Message 13 — notification.** For HIGH and MEDIUM runs the gate notifies humans over Slack and email and waits.
14. **Message 14 — human decision.** The Admin’s approve, reject, or modify action arrives as a Temporal signal. The subsequent **alt block** encodes the automatic paths: LOW severity auto-approves immediately, and MEDIUM auto-approves after a 120-second countdown unless a human overrides it first. HIGH always requires an explicit human decision — one of the two live runs evidenced in Chapter 5 genuinely timed out at this gate, which is the safety mechanism working as designed.
15. **Message 15 — GitOpsDeploy.** On approval, the gitops service performs the blob → tree → commit → ref → PR sequence against GitHub.
16. **Message 16 — conditional DeployEngine.Redeploy.** Executed only when the triggering incident’s source was the deploy-engine: a merged fix to a broken preview build triggers an automatic rebuild and redeploy of the preview container.
17. **Message 17 — post-deploy health window.** The rollback probe watches `incidents_raw` for a fresh fatal on the same organisation and service within the bounded window.
18. **Message 18 — conditional ArgoCD.Rollback.** Inside the closing **alt block**, an SLO breach detected during the window triggers the ArgoCD Rollback client to restore the prior known-good revision — and, as established in Section 4.3.3, the breach itself re-enters the pipeline as a new incident.

The footer of the diagram records the observed steady-state wall-clock time of a full run: approximately 35 seconds to 2 minutes on local Ollama models, depending on scenario complexity and LLM latency. This range is corroborated by the real timings visible in the Chapter 5 screenshots, including the fully successful run that completed in 1 m 40 s from detection to verified deployment.

Taken together, the four views of this chapter describe one coherent design: a mediated platform boundary (use case Level 0), a role-additive human surface beside an autonomous fleet (use case Level 1), a single shared data path for manual and automatic operation (DFD Level 1, Owner/Admin), and a detection–repair–verification cycle that feeds its own failures back into itself (DFD Level 1 self-healing loop and the sequence diagram). Chapter 5 presents the implementation of each element, and Chapter 6 the evidence that the loop closes in practice.

Chapter 5 Implementation

This chapter documents the implemented system as it exists and runs today. It first enumerates the concrete tools and technologies used at every layer of the stack, then walks through the platform screen by screen, and finally summarises the database schema that underpins the multi-tenant control plane. All screenshots in this chapter are unretouched captures of the running development environment; where a screen shows an error, a pending state, or an incomplete feature, that is reported as-is rather than substituted with a more flattering capture.

5.1 Tools and Technology

Nexis is a polyglot system organised as a `pnpm + Turborepo` monorepo. Each subsystem uses the language best suited to its role: Go for the latency- and concurrency-sensitive control plane and its sidecar services, Python for the statistical causal-inference component, and TypeScript for the web console.

5.1.1 Backend

- **Go 1.25** — four independent Go modules provide the server side of the platform: `services/control-plane` (the multi-tenant API and business logic), `services/validator` (the sandboxed patch-validation service), `services/gitops` (the GitHub-App deployment path), and `services/deploy-engine` (the Docker preview-deploy engine, described in Section 5.2.5).
- **Python / FastAPI** — `services/causal-inference`, a sidecar on port 8090 that ranks candidate root-cause nodes forwarded by the Pathfinder agent using a graph-evidence scoring algorithm.
- **Temporal [2]** — the durable workflow engine that orchestrates the nine-agent RecoveryPipeline. Every agent step is a Temporal activity, so a crashed worker resumes exactly where it left off and every step is durably recorded as an `ActivityEvent`.

5.1.2 Frontend

- **Next.js 16 / React 19 / TypeScript** — the web console in `apps/web`, covering authentication, the admin console, incident observability, settings, and the project Ops tab.
- **Tailwind CSS v4** and **Radix UI** — styling and accessible component primitives.
- **Monaco editor** — the in-browser editor component used to render and edit unified diffs in the approval dialogs.
- **Recharts** — the dashboard, performance, and cost charts.
- **Server-Sent Events (SSE)** — the incident detail page subscribes to a live event stream, so pipeline progress renders in real time without polling.

5.1.3 Data and Messaging

- **PostgreSQL 17** — the system of record, multi-tenant via Row-Level Security [7]; the schema has grown across 33 sequential migrations (Section 5.3).
- **pgvector** [19] — vector similarity search inside PostgreSQL, backing the knowledge-base retrieval feature.
- **Neo4j** [3] — the code dependency graph that the Pathfinder agent walks from a fault symptom toward candidate root-cause nodes.
- **Redis** — rate limiting and caching.
- **MinIO (S3-compatible)** — encrypted storage of patch blobs.
- **Temporal persistence** — Temporal maintains its own durable event history alongside the application database.

5.1.4 AI / LLM

- **OpenAI (GPT-4o family)** — the production-configured provider for agent reasoning and code generation.
- **Ollama** [20] — a fully supported alternate provider running local models (`gpt-oss:20b` and `qwen2.5-coder:14b`). All live-run evidence presented in this chapter was produced end-to-end with Ollama; no cloud LLM credentials were used.

- **Hypothesis** [9] — the property-based testing library in which the QA agent auto-generates test suites; these suites are what the Validator executes inside its sandbox.
- **Causal-inference sidecar** — a statistical/graph ranking algorithm rather than a fitted structural causal model; a full DoWhy [4] model would require interventional production data that does not yet exist, and this limitation is stated openly rather than papered over.

5.1.5 DevOps and Infrastructure

- **Docker API** [10] — container runtime for both the Validator sandbox and the preview-deploy engine. The integration is Podman-compatible [11]; in the evaluation environment used for this report, Podman was substituted for Docker and proven to work identically.
- **Terraform** — infrastructure-as-code targeting AWS ECS behind an Application Load Balancer.
- **OpenTelemetry** [5] — traces, metrics, and logs exported to Prometheus, Loki, and Tempo, visualised in Grafana.
- **Argo CD** [6] — the DevOps agent’s Sync/Rollback client integration for GitOps deployment and post-deploy rollback.

5.1.6 IDE / Version Control

- **Git** — version control for the entire monorepo, including the 33 sequential database migrations tracked in-repo.
- **GitHub** — repository hosting; beyond hosting, GitHub is a first-class runtime dependency: a GitHub App installation supplies short-lived tokens for repo cloning and for the GitOps service’s blob → tree → commit → ref → PR deployment path.
- **pnpm workspaces + Turborepo** — monorepo task orchestration (build, typecheck, lint) across the web app and shared packages.

5.2 Project View

This section presents the implemented platform screen by screen. The captures are grouped by functional area, in the order a new organisation would encounter them: authentication, the admin

console, member and session management, the self-healing loop itself (the centrepiece of this report), the Docker preview-deploy engine, project and observability views, and finally integrations.

5.2.1 Authentication and Onboarding

Account access is handled by a local JWT-based session system (HttpOnly cookie) with optional WorkOS [12] single sign-on. The sign-up page (Figure 5.1) creates the initial account; sign-in (Figure 5.2) establishes the session; the forgot-password flow (Figure 5.3) issues a 32-byte random reset token, stored SHA-256 hashed with a 30-minute TTL, and deliberately returns the same 202 response whether or not the e-mail exists, so the endpoint leaks no account information.

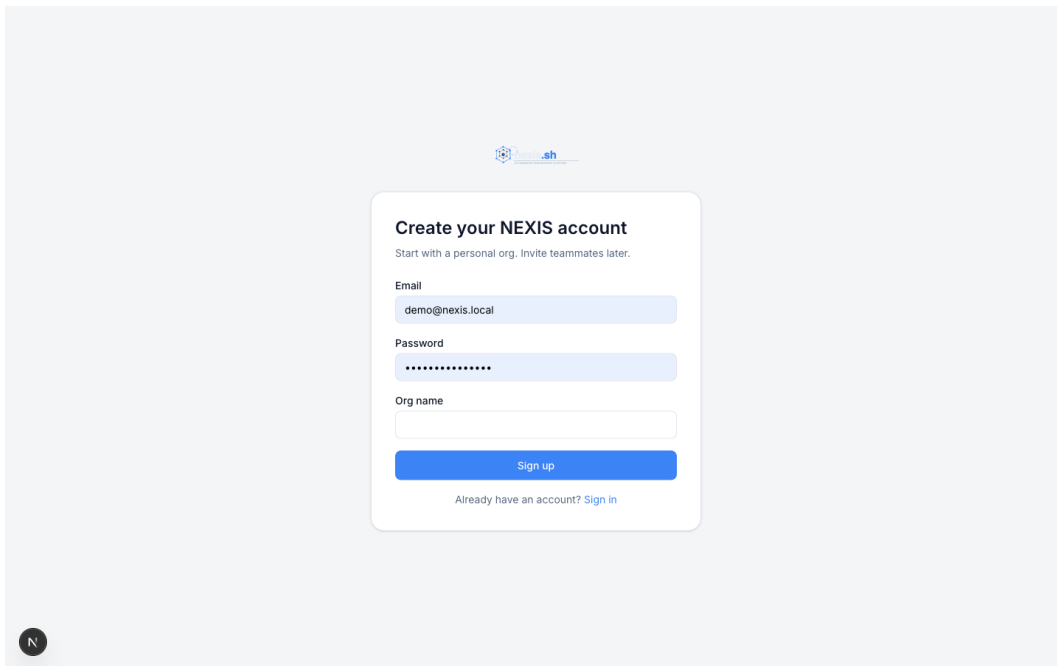


Figure 5.1: Sign-Up Screen (new account registration).

5.2.2 Admin / Owner Console

The console home (Figure 5.4) presents the organisation’s operational state at a glance: KPI tiles for MTTR, recovery success rate, open incidents and mean tokens per run, an incidents-over-time chart, a system status panel, a projects health grid, and a recent activity feed.

The Agents page (Figure 5.5) lists the full nine-agent fleet across both layers — the five Execution Team agents (Architect, Backend, QA, DevOps, Data Engineer) and the Self-Healing Loop agents (Sentinel, Pathfinder, Synthesiser, Validator) — with each agent’s role and status.

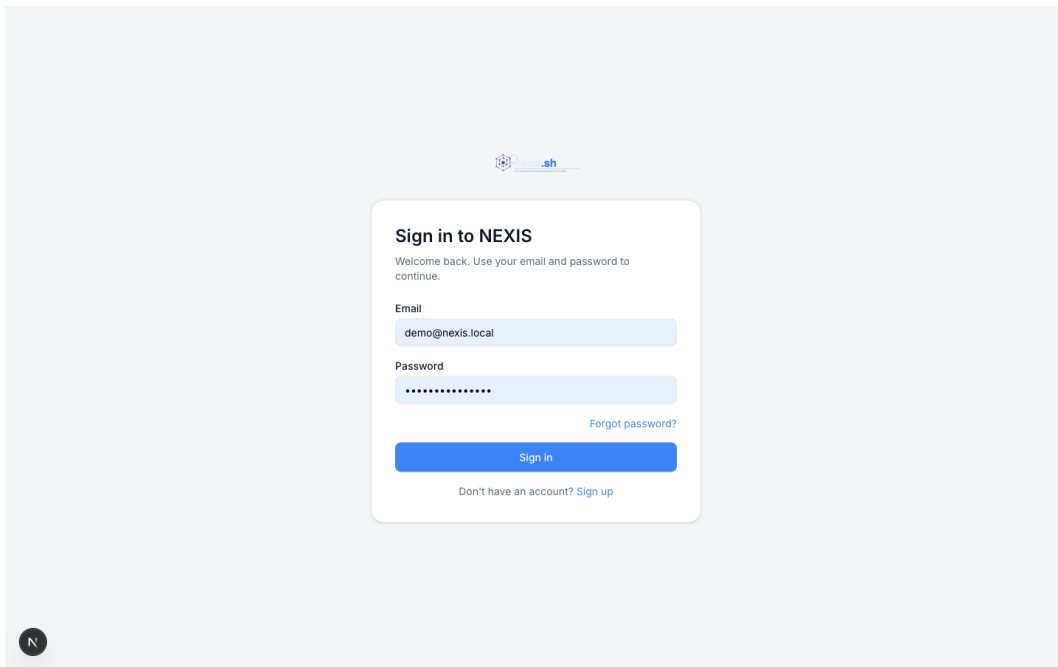


Figure 5.2: Sign-In Screen (JWT session login with HttpOnly cookie).

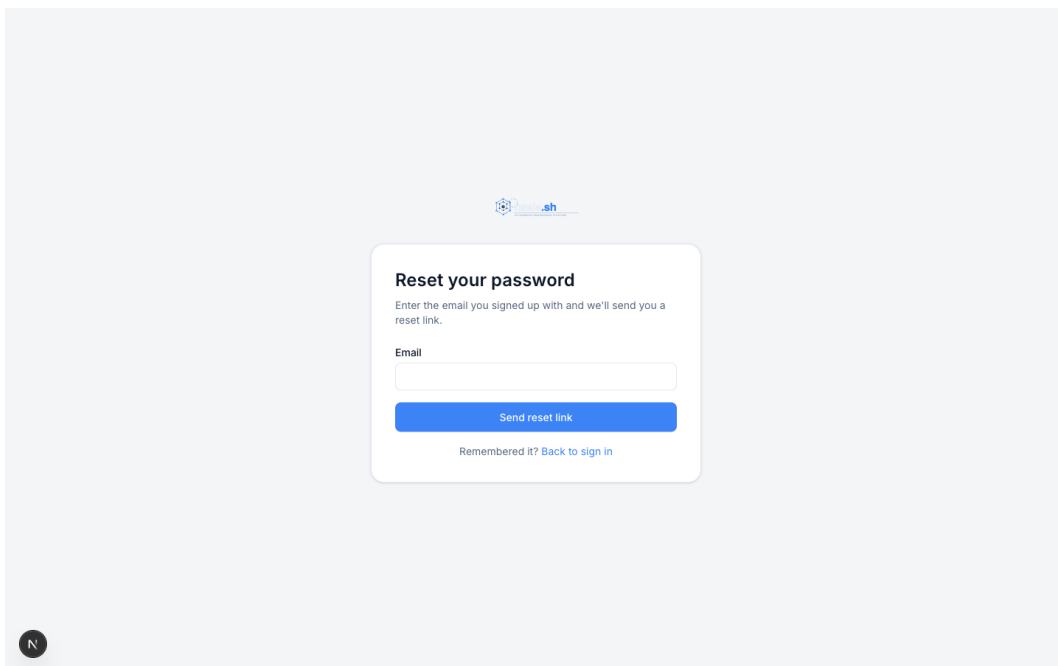
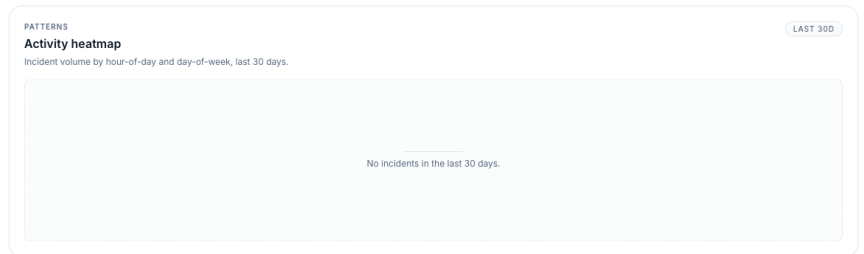
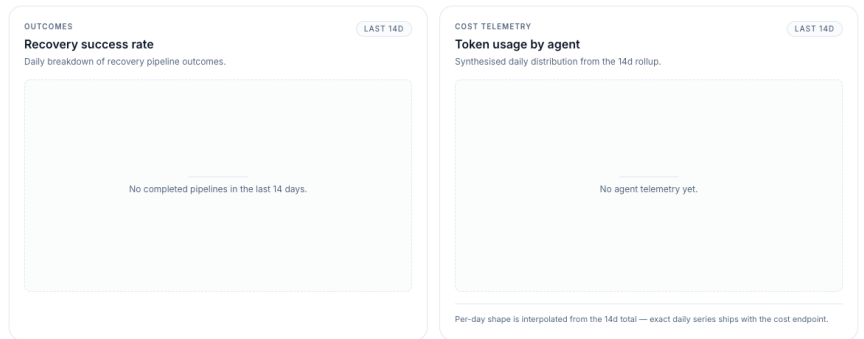
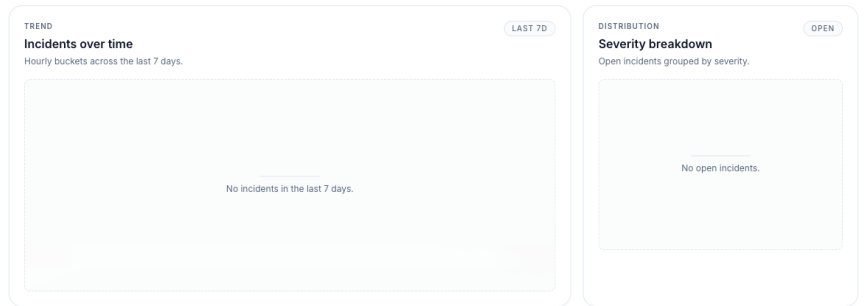
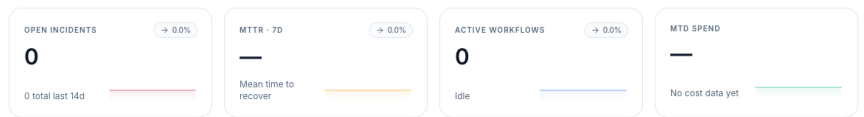


Figure 5.3: Forgot-Password Screen (non-leaky password-reset request).

OPERATIONS

Good morning, demo

Demo Org · demo-org · OWNER




System status

[Full health board →](#)

Sub-systems + integration health, polled every 30s.

CORE SUB-SYSTEMS


Control plane
API + auth + RLS


Temporal
Workflow engine


Postgres
OLTP + pgvector


Redis
Cache + pubsub


Edge / Proxy
No integrations

INTEGRATIONS

No integration connections yet. Connect one from [Integrations](#) to see live probe health.

Projects health

Auto-healing targets in this workspace.

View all →



No projects yet

Connect a service to start routing incidents.

+ Connect first project

Recent activity

Latest 10 audit events across the org.

User Login

19af36d3-d4f7-4ee4-bf9b-82c1c907c3de

just now >

Workspace Created

4fc711b3-be8f-4d1c-93c3-cb1ac1c81bda

9h ago >

User Login

19af36d3-d4f7-4ee4-bf9b-82c1c907c3de

10h ago >

User Signup

19af36d3-d4f7-4ee4-bf9b-82c1c907c3de

10h ago >

View all →

ORGANISATION

Demo Org

demo-org - OWNER

demo@nexus.local

WORKSPACE

Production

● Ready

us-east-1

Created 9 hours ago

Suspend

QUICK ACTIONS

-  **Trigger sample incident**
Run a synthetic recovery flow end-to-...
-  **Invite member**
Add a teammate to this organisation
-  **Connect GitHub**
Wire PR creation + repo metadata
-  **View docs**
Operator guide + API reference

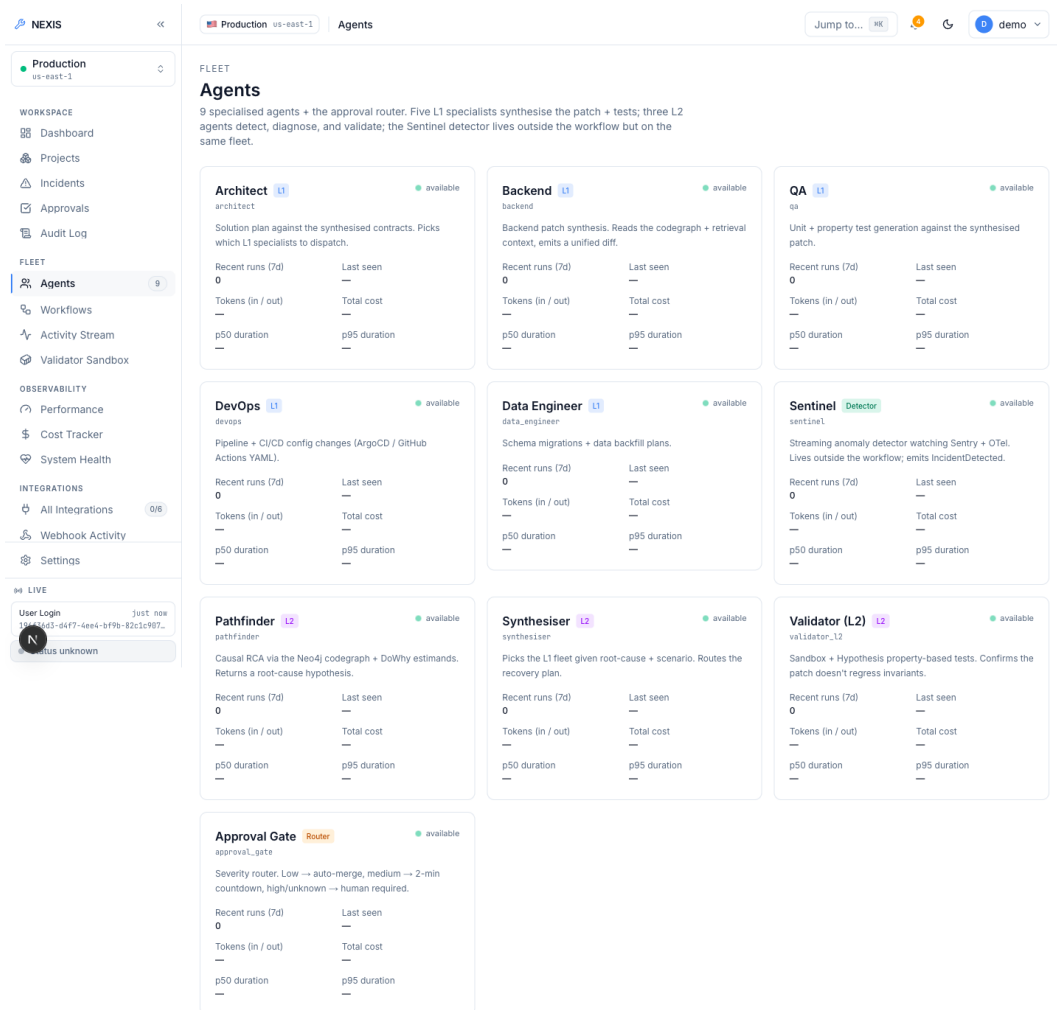


Figure 5.5: Agents Screen (the nine-agent fleet roster across both layers).

The Workflows page (Figure 5.6) lists RecoveryPipeline executions; the Activity Stream (Figure 5.7) is the org-wide live event feed; and the Validator Sandbox console page (Figure 5.8) surfaces the Docker-sandboxed shadow-execution service in which every candidate patch is run against its QA-generated property tests before it may reach the approval gate.

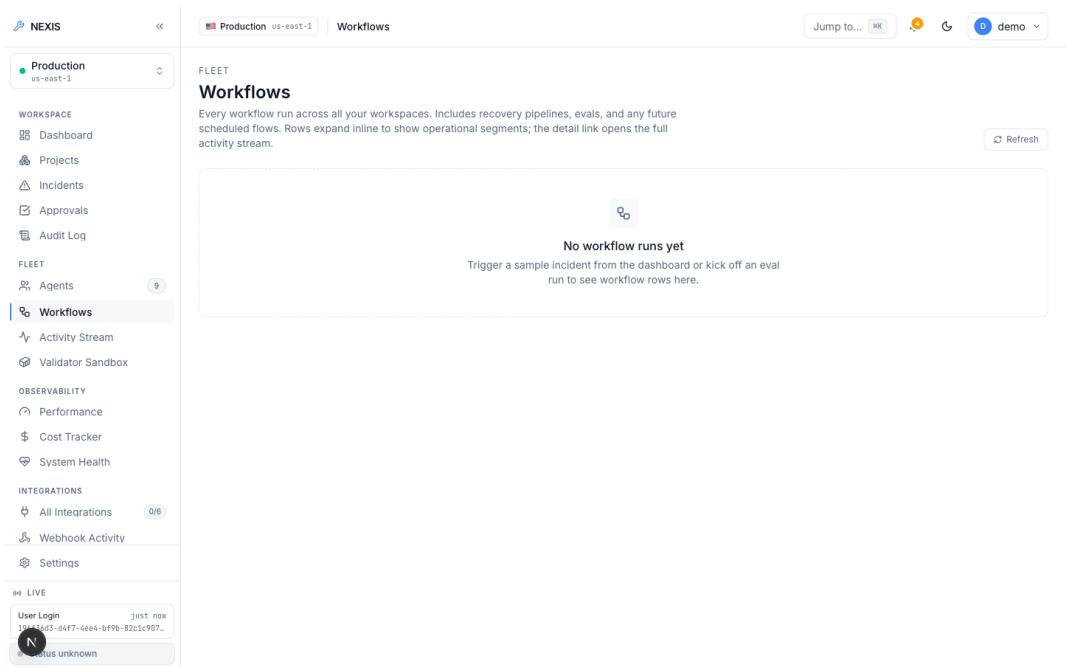


Figure 5.6: Workflows Screen (RecoveryPipeline run listing).

The Audit Log (Figure 5.9) is append-only and hash-chained: each entry incorporates the hash of its predecessor, so any tampering with a past row breaks the chain and is detectable by the cryptographic verify endpoint. The log supports CSV export for offline review.

5.2.3 Members, Roles, and Sessions

The Members settings page (Figure 5.10) lists the organisation’s members with their roles (owner, admin, member) and exposes the intelligent role-recommendation panel. This panel is backed by the real heuristic engine described in Chapter 4, not a mock: it evaluates concrete signals already captured in the schema — a member with five or more admin-gated audit-log actions in 30 days who is not yet an admin, an admin with no login in 90 or more days, and an active admin with zero admin-gated actions — and each recommendation carries a plain-language rationale naming the specific evidence numbers behind it. An admin can accept the recommendation (applying the role change under an optimistic-concurrency guard) or dismiss it, which starts a 30-day per-rule cool-down.

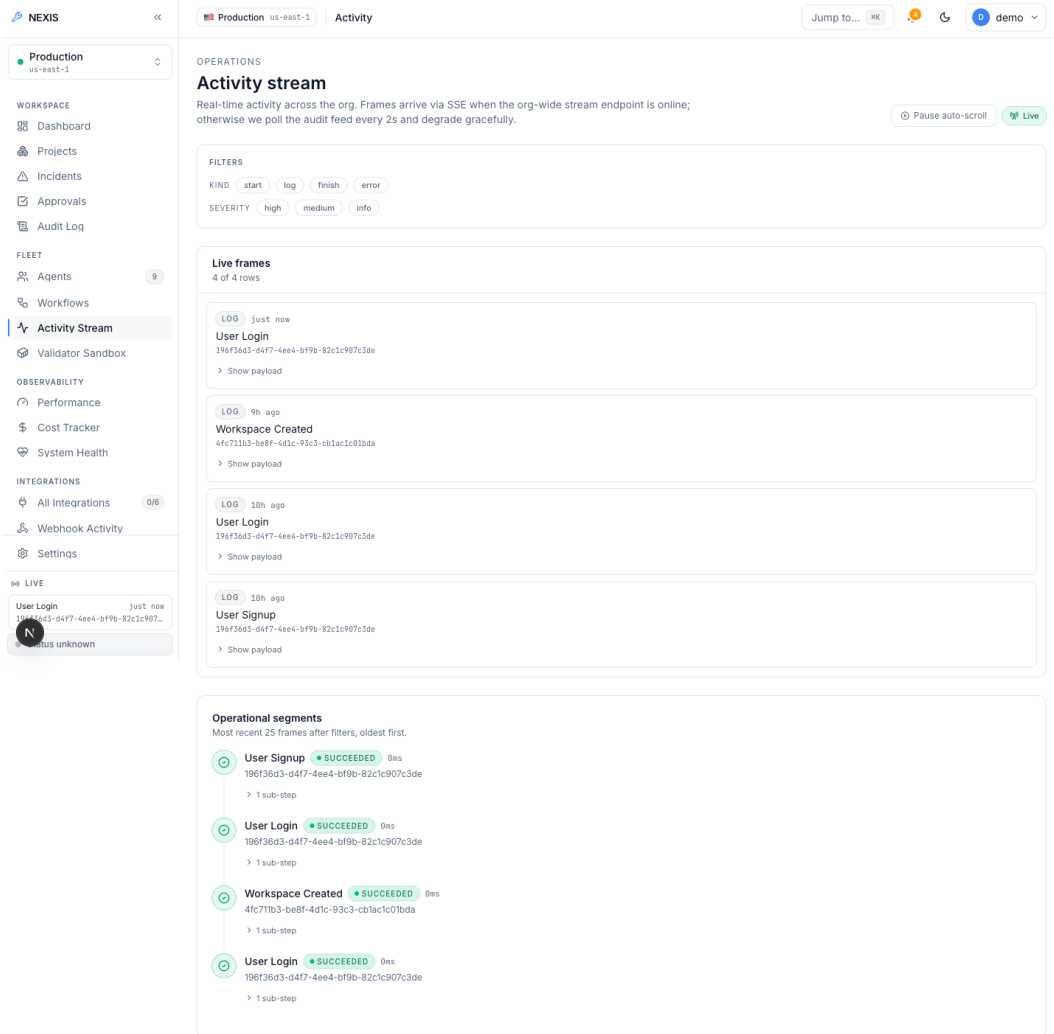


Figure 5.7: Activity Stream (organisation-wide live event feed).

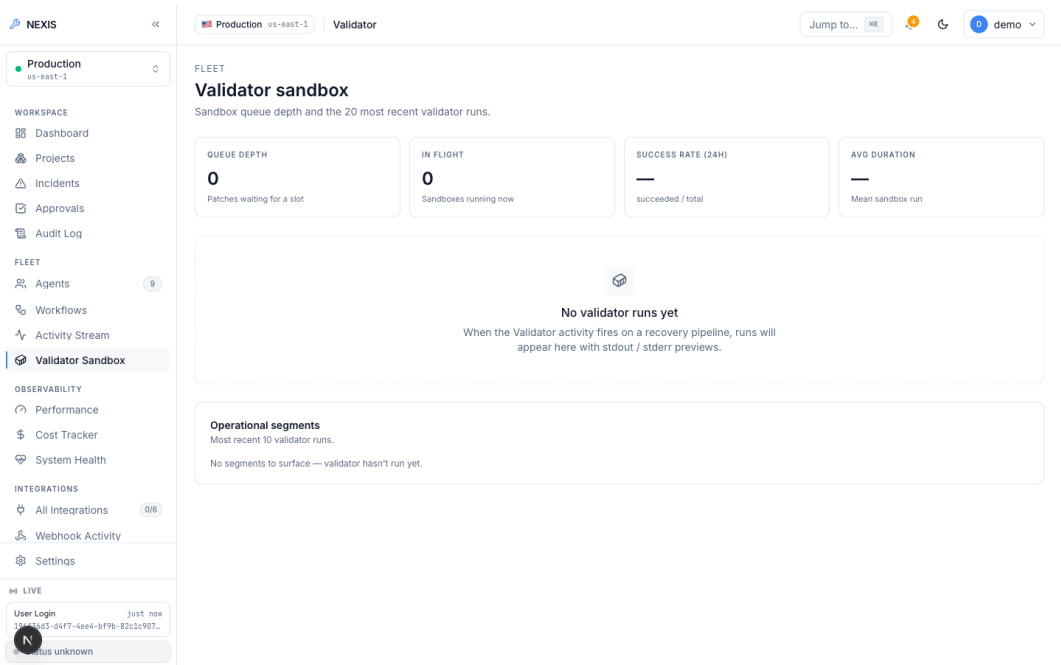


Figure 5.8: Validator Sandbox Console (sandboxed shadow execution of candidate patches).

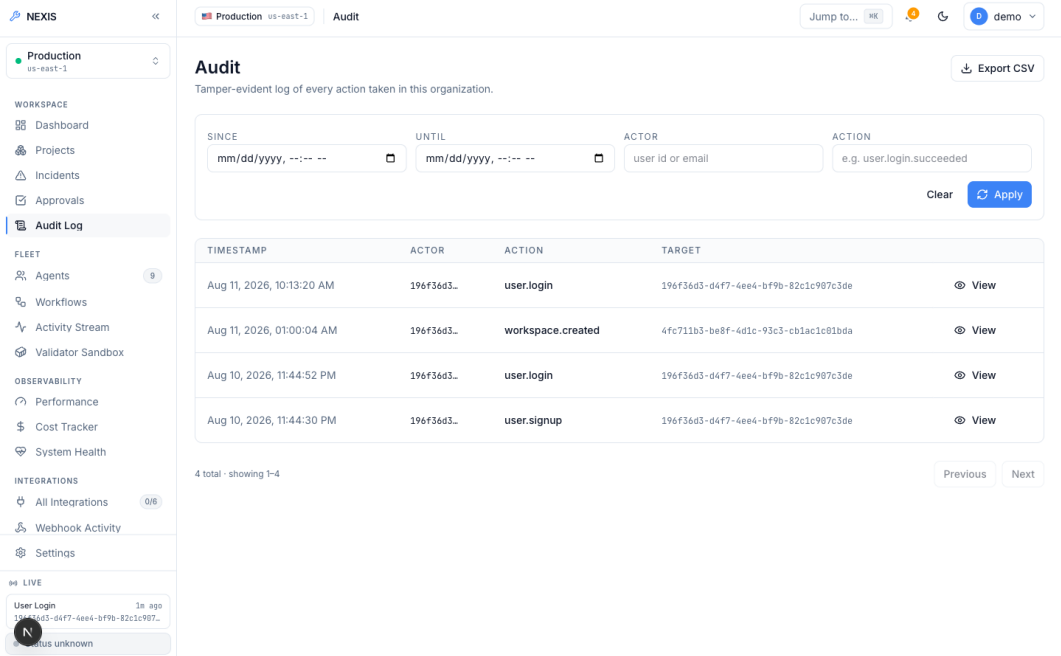


Figure 5.9: Audit Log (append-only, hash-chained entries with CSV export).

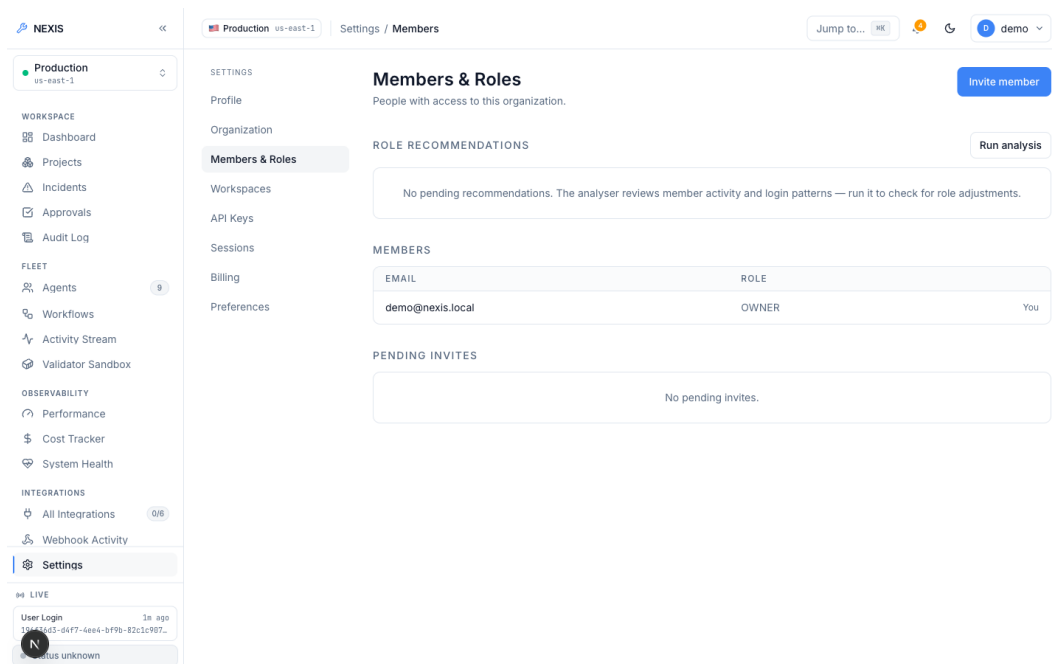


Figure 5.10: Members Settings (members list with the heuristic role-recommendation panel).

Session management (Figure 5.11) lists every active session with its device, IP address, user agent, and last-seen time; any session can be revoked remotely, and the caller’s own session is marked with a “This device” badge. API keys (Figure 5.12) are scoped bearer tokens (nx_live_...) that can be created, listed, and revoked. Billing (Figure 5.13) surfaces Stripe [13] usage and payment method management, and the Organization page (Figure 5.14) holds organisation-level profile settings.

5.2.4 Self-Healing Loop — Live Evidence

This subsection is the evidentiary core of the report. It documents two real, complete, end-to-end executions of the nine-agent RecoveryPipeline against a live local LLM (Ollama, qwen2.5-coder:14b and gpt-oss:20b) — one run that timed out at the human approval gate and was safely auto-rejected, and one run that was approved within the window and succeeded fully. Both outcomes are shown exactly as they occurred.

The entry point is the Live Demo console (Figure 5.15), which presents 27 curated fault-scenario cards drawn from a broader fixture taxonomy spanning application, database, deploy, observability, and security categories. To be precise about the current state: exactly one of the 27 scenarios is fully wired end-to-end today; the remaining cards are UI-complete but backend-pending and are labelled “Coming soon” in the interface. The runs documented below were triggered through the

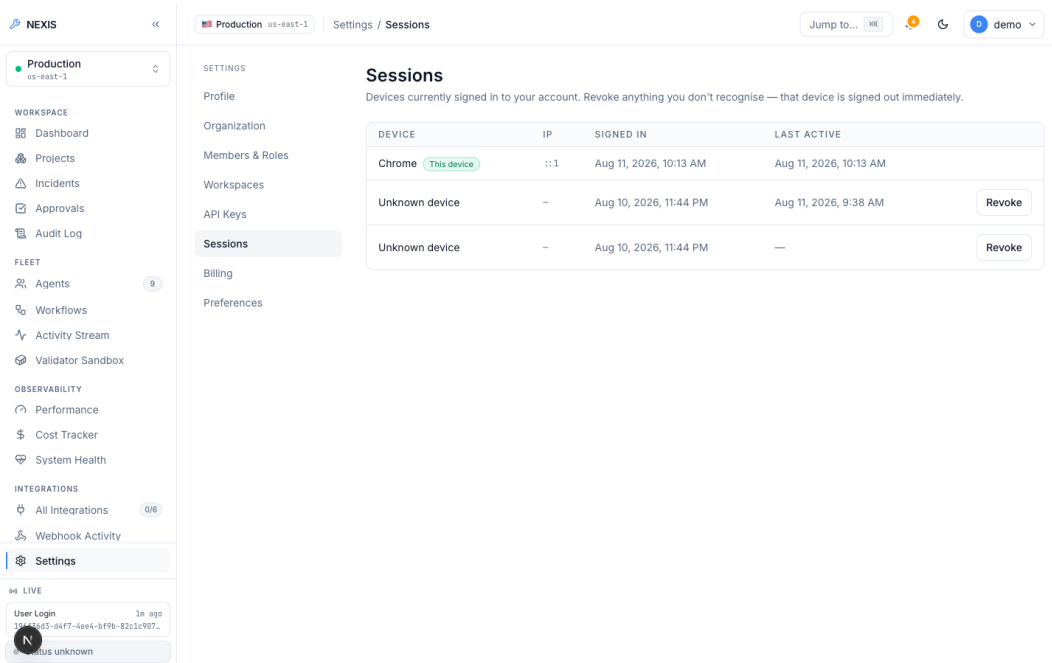


Figure 5.11: Sessions Settings (active sessions with device, IP, and remote revoke).

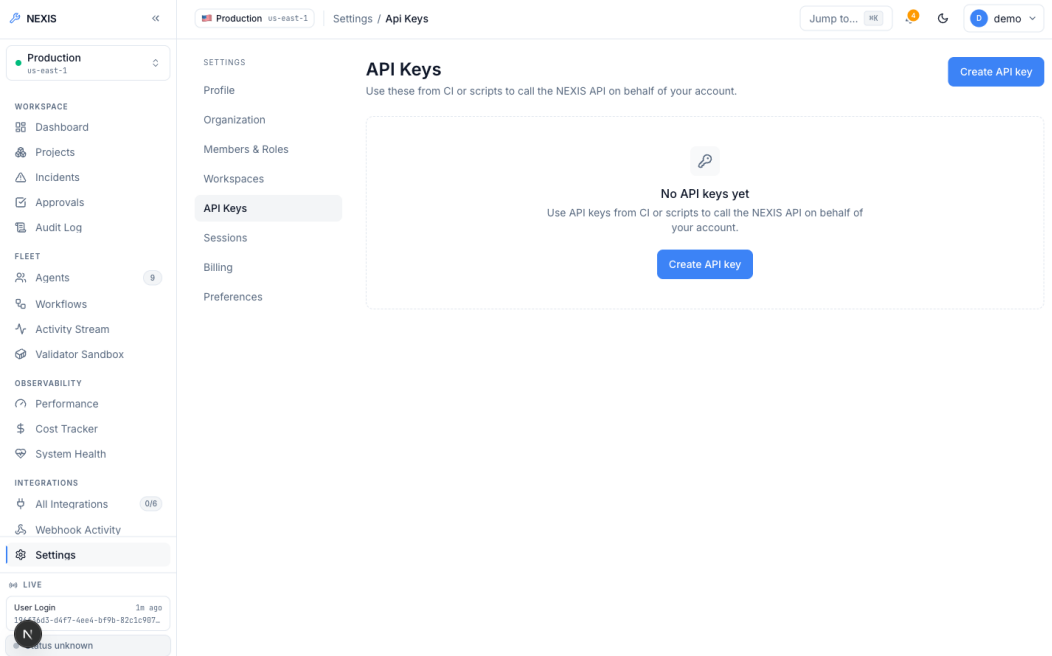


Figure 5.12: API Keys Settings (scoped bearer key creation, listing, and revocation).

NEXIS « **Production** us-east-1 Settings / Billing

Jump to... **demo**

Production us-east-1

WORKSPACE

- Dashboard
- Projects
- Incidents
- Approvals
- Audit Log

FLEET

- Agents
- Workflows
- Activity Stream
- Validator Sandbox

OBSERVABILITY

- Performance
- Cost Tracker
- System Health

INTEGRATIONS

- All Integrations
- Webhook Activity

Settings

SETTINGS

- Profile
- Organization
- Members & Roles
- Workspaces
- API Keys
- Sessions
- Billing**
- Preferences

Billing

Manage your payment method, review usage, and download invoices.

PAYMENT METHOD

CARD NUMBER
4242 4242 4242 4242

EXP MONTH
12

EXP YEAR
30

CVC
123

POSTAL CODE
10001

BILLING EMAIL
billing@example.com

[Use test card](#) [Save payment method](#)

USAGE

USAGE THIS PERIOD
\$0.20

BY WORKSPACE
4fc711b3-be8f-4d1c-93c3-cblac1c01bda \$0.20

BY KIND
runtime_hours \$0.20

INVOICES

No invoices yet — they'll appear at the end of your first billing cycle.

Figure 5.13: Billing Settings (Stripe usage and payment-method management).

NEXIS « **Production** us-east-1 Settings / Organization

Jump to... **demo**

Production us-east-1

WORKSPACE

- Dashboard
- Projects
- Incidents
- Approvals
- Audit Log

FLEET

- Agents
- Workflows
- Activity Stream
- Validator Sandbox

OBSERVABILITY

- Performance
- Cost Tracker
- System Health

INTEGRATIONS

- All Integrations
- Webhook Activity

Settings

SETTINGS

- Profile
- Organization**
- Members & Roles
- Workspaces
- API Keys
- Sessions
- Billing
- Preferences

Organization

The tenant your account belongs to. Renaming + transfers coming soon.

NAME
Demo Org

SLUG
demo-org

ORGANIZATION ID
1d81b8e0-0172-4e07-9c17-fa994aad0aeb

YOUR ROLE
owner

Figure 5.14: Organization Settings (organisation profile management).

wired path.

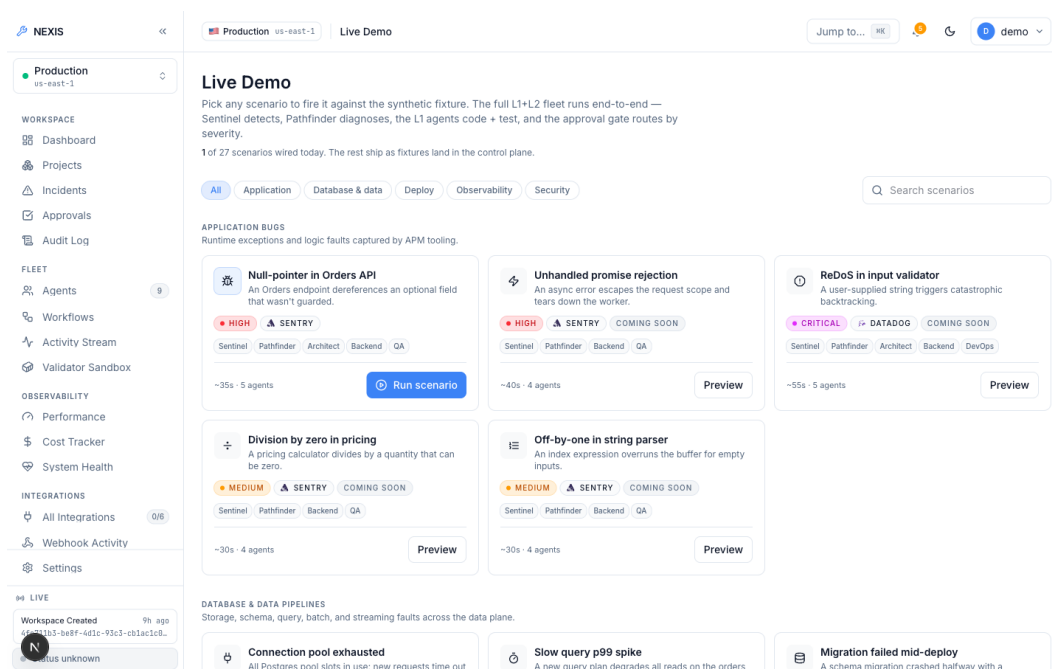


Figure 5.15: Live Demo Console (27-scenario fault-injection picker; one scenario fully wired, the rest backend-pending).

Figure 5.16 captures the first run genuinely mid-flight. The Architect, Backend, and QA steps have already completed — each showing a real, non-zero token count and cost figure accumulated from actual LLM calls — while the DevOps and Data Engineer steps are executing in parallel. These visible token and cost figures are themselves evidence: a stubbed or templated pipeline would report zero tokens, whereas every completed step here carries the token consumption of the model call that produced its output.

Figure 5.17 shows the same run moments later: every Layer-1 and Layer-2 step has completed green — the patch has been generated, property-tested, and validated in the sandbox — and the run now sits at severity “Medium” with approval status “Pending”, awaiting a human decision inside the 120-second window.

That pending state is reflected in the Approvals queue (Figure 5.18): a real pending-approval row with the three human decision actions — Reject, Modify, and Approve.

The Modify action opens the dialog in Figure 5.19. This is the RLHF mechanism described in Chapter 4 made concrete: the engineer edits the LLM-generated unified diff directly in the embedded editor before approving, and the edited diff — along with every other terminal approval decision — is persisted as a row in the `feedback_examples` table. Those rows feed a JSONL export endpoint intended for future fine-tuning; actually submitting a paid fine-tuning job against

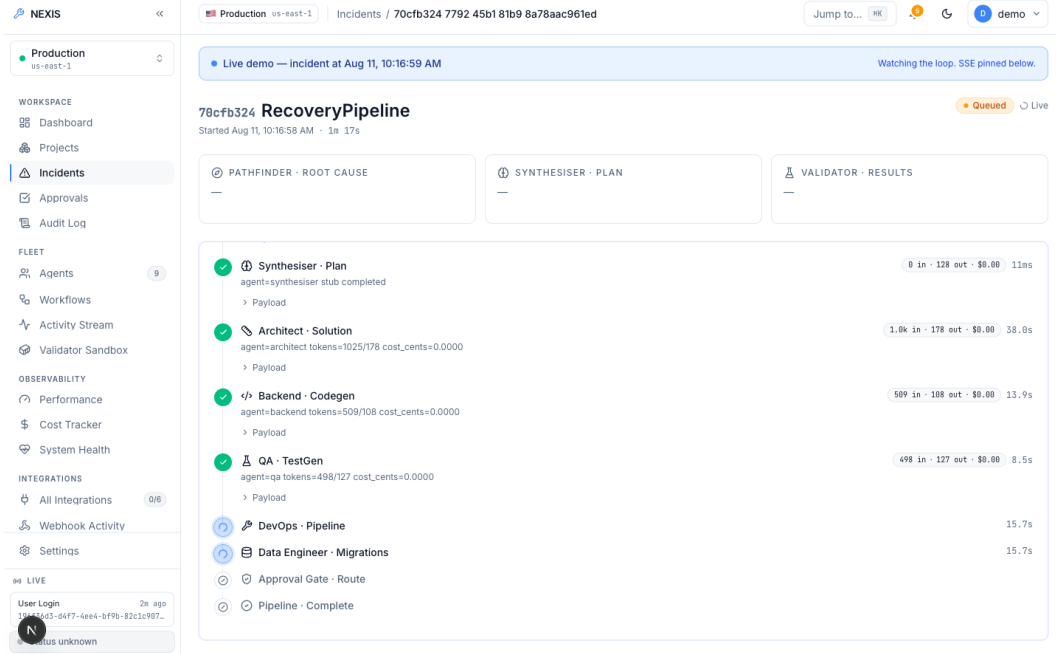


Figure 5.16: Incident Detail, Mid-Flight (live pipeline with completed Architect/Backend/QA steps showing real token counts; DevOps and Data Engineer running in parallel).

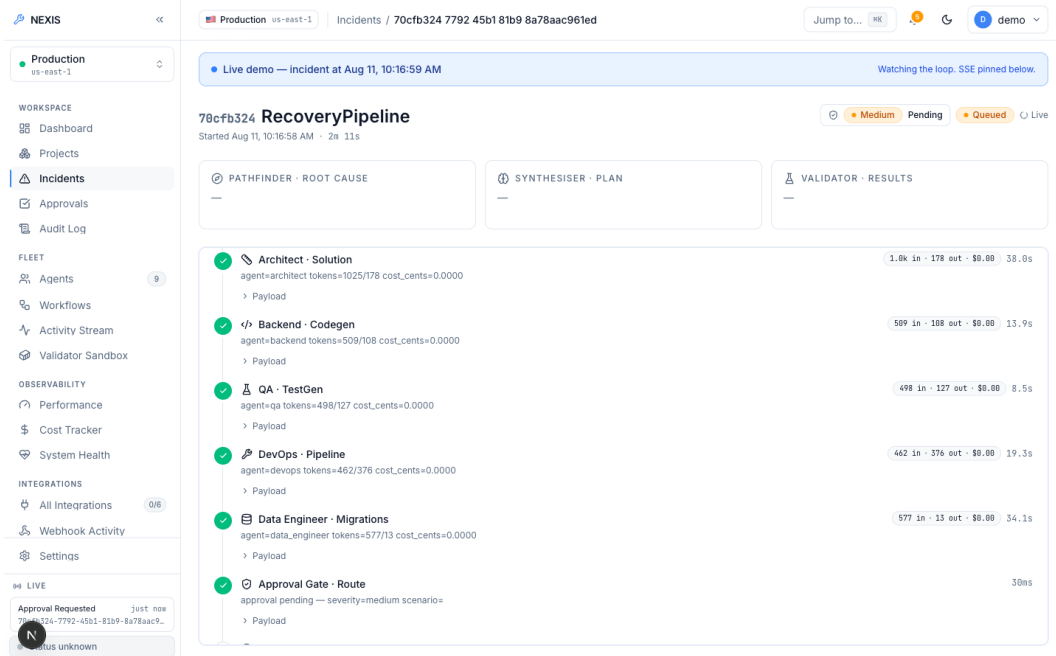


Figure 5.17: Incident Detail, Pipeline Complete (all L1/L2 steps green; severity Medium, approval Pending).

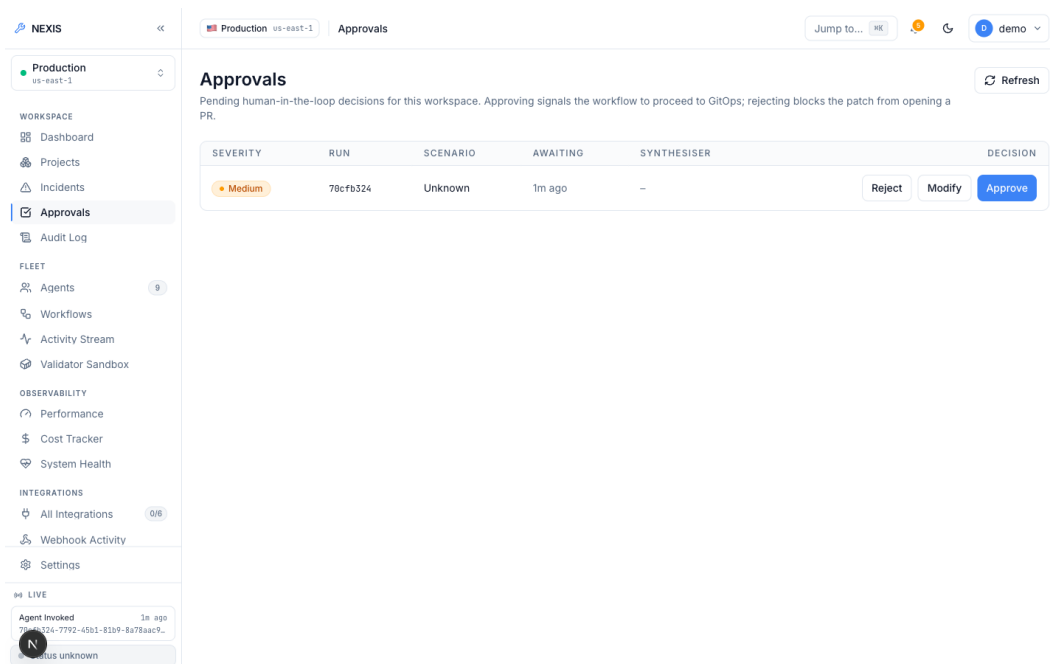


Figure 5.18: Approvals Queue (real pending approval row with Reject / Modify / Approve actions).

the exported data is a deliberate manual step, not an automated one.

Figure 5.20 is an honest negative result, reported deliberately. For this first run, the 120-second approval window elapsed before a decision was recorded, and the workflow terminated the run as `timeout_rejected`. This is the timeout safety mechanism working exactly as designed: when no human decision arrives in time, the system’s default is to refuse to deploy unreviewed code, not to proceed. A pipeline that silently auto-deployed on timeout would be a defect; this behaviour is the intended safe failure mode.

Figure 5.21 is the key positive result of this report: a second live run, approved by a human within the window. The incident shows severity “Medium”, decision “Approved”, and outcome “Succeeded”, with the full nine-step timeline — detection through plan, patch generation, test generation, validation, approval, and deployment steps — completing in 1 minute 40 seconds end-to-end, with real per-agent token and cost figures throughout.

Finally, Figure 5.22 settles the question of whether the patch itself is genuinely LLM-generated. It shows the expanded raw JSON payload of the Backend agent’s step: the actual unified diff produced by `qwen2.5-coder:14b`, with `provider="ollama"` recorded in the payload. The patch is model output parsed from a unified diff, not a template retrieved from a fixture.

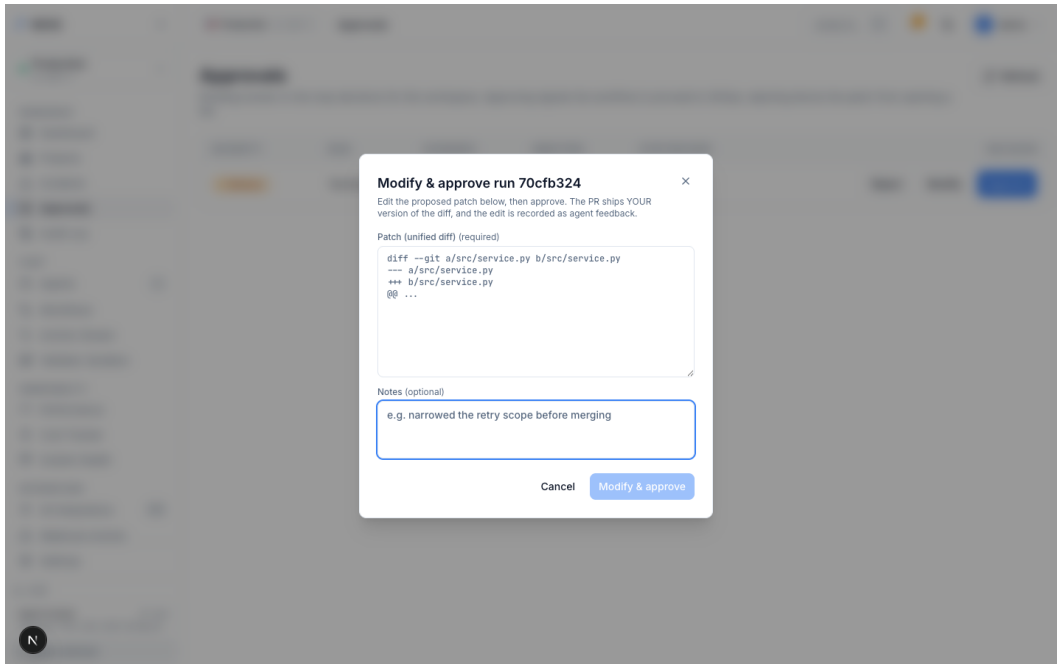


Figure 5.19: Modify-and-Approve Dialog (RLHF diff editing; the engineer’s edit becomes a feed-back_examples training row).

5.2.5 Docker Preview-Deploy Engine (Ops Tab)

Each project detail page is organised into tabs. The Overview tab (Figure 5.23) summarises the project; Integrations (Figure 5.24) manages the project’s provider connections, most importantly the GitHub repository binding; Recovery Policy (Figure 5.25) configures per-project self-healing behaviour such as rollback-on-SLO-breach; and Activity (Figure 5.26) shows the project-scoped event history.

The Ops tab is the frontend to the preview-deploy engine. Figure 5.27 shows its empty state before any deployment: the Deploy action, live status area, and (once populated) the preview URL, build/container logs, and deployment history.

Figure 5.28 shows what happened when Deploy was clicked on a project with no GitHub App connection in the evaluation environment: the real API returned the validation error “project has no github repository bound”, which the UI surfaces verbatim. To be clear, this capture does *not* show a successful deploy — it shows the real validation chain rejecting a deploy request that cannot be fulfilled, which is the correct behaviour. The limitation here is environmental (no GitHub App credentials exist in the evaluation environment, as stated in Chapter 7’s limitations), not a defect in the engine.

The engine itself, however, was proven live independently of that credential limitation. Listing 5.1

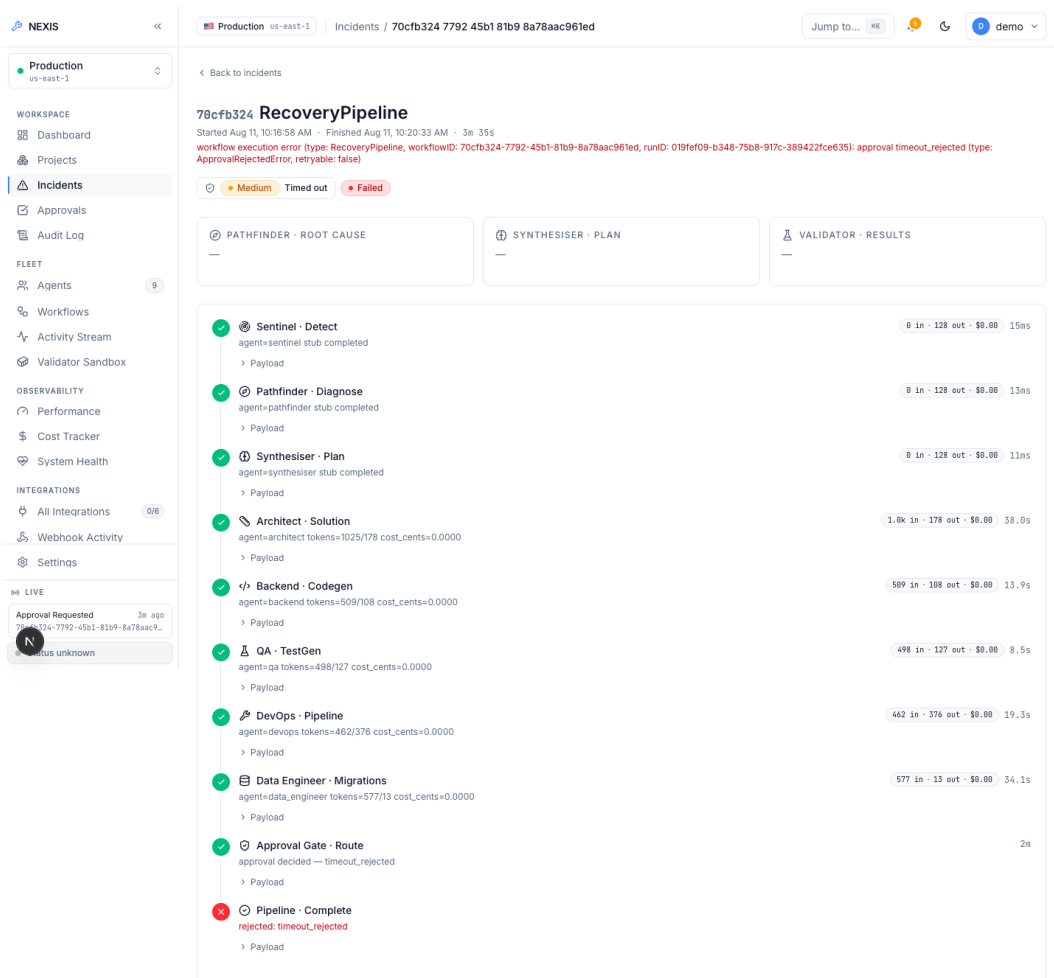


Figure 5.20: Timeout-Rejected Run (the 120-second approval window elapsed; the workflow safely auto-rejected as `timeout_rejected` rather than deploying unreviewed code).

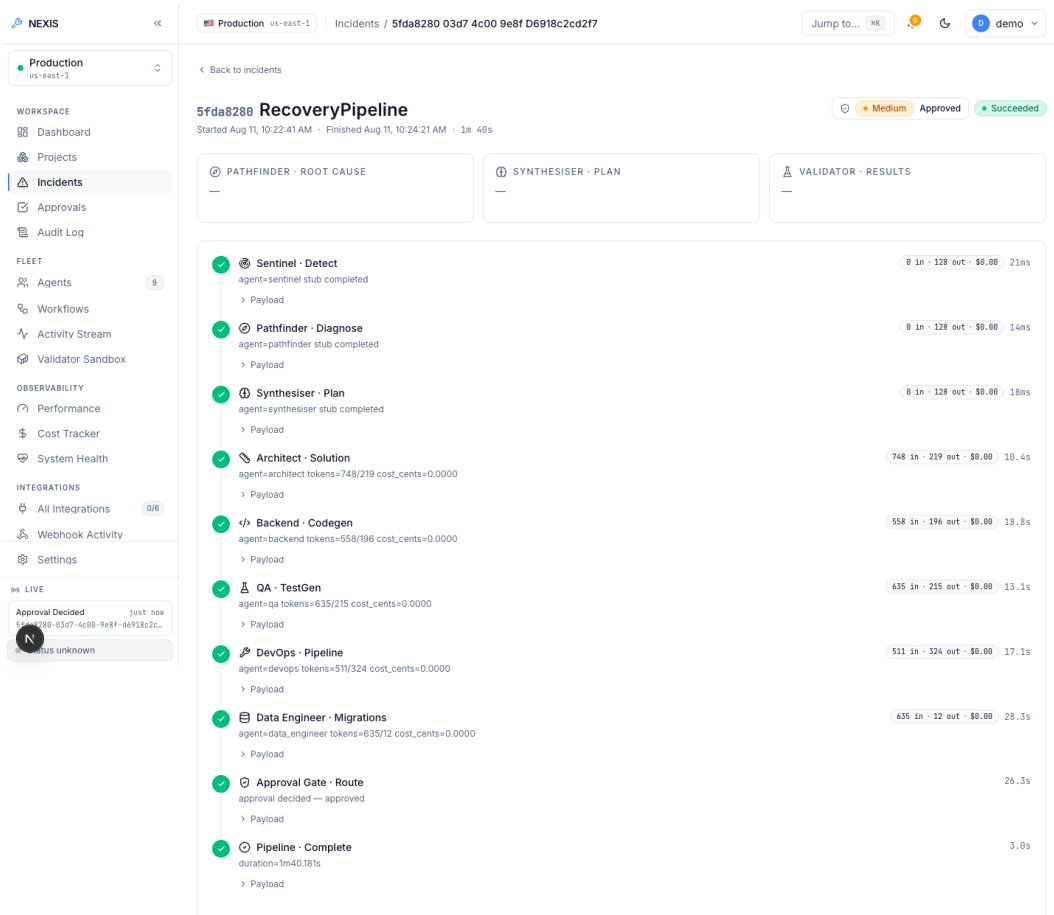


Figure 5.21: Fully Approved Run (Medium severity, Approved, Succeeded; complete nine-step timeline in 1 m 40 s with real per-agent token and cost figures).

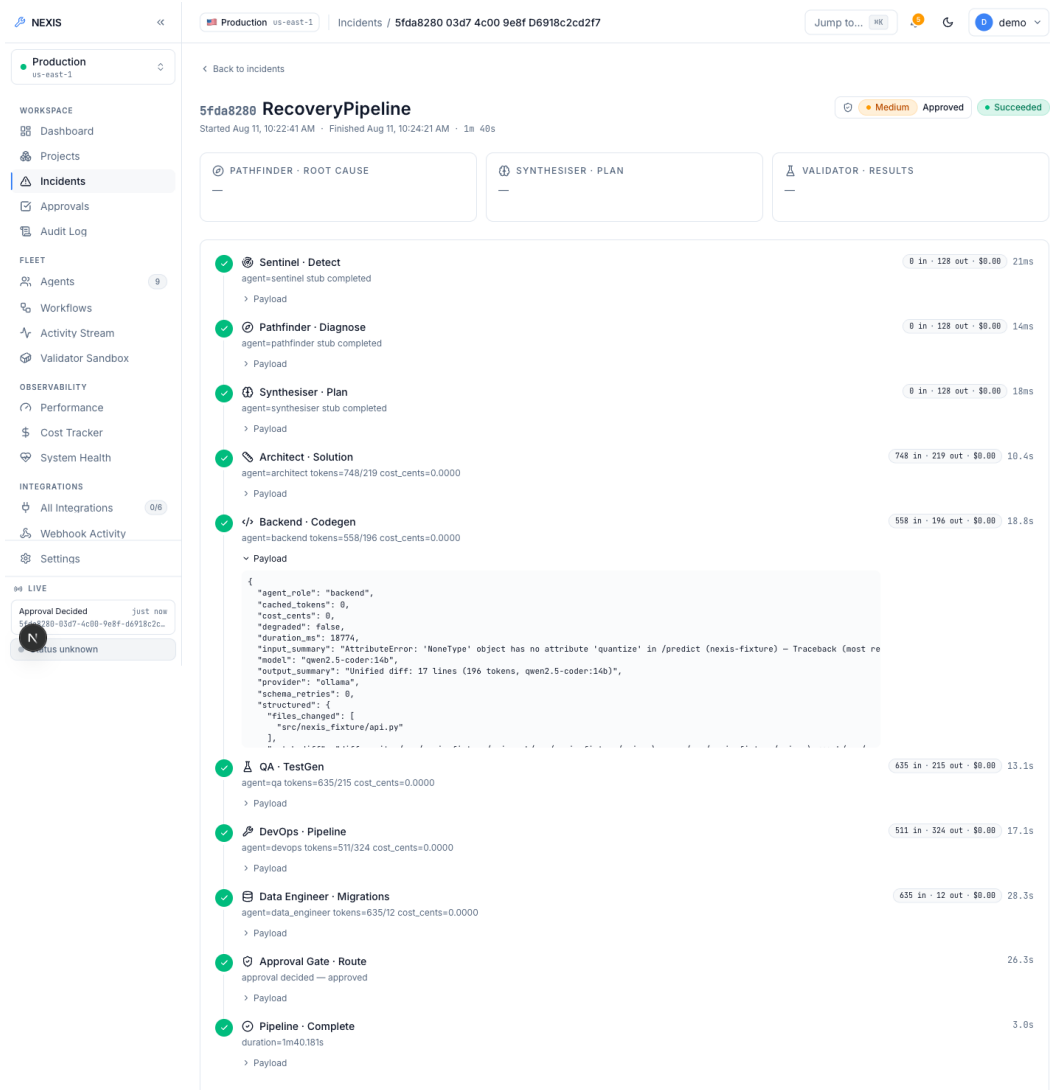


Figure 5.22: Backend Payload, Expanded (raw JSON evidence of the qwen2.5-coder:14b-generated unified diff, provider=“ollama”).

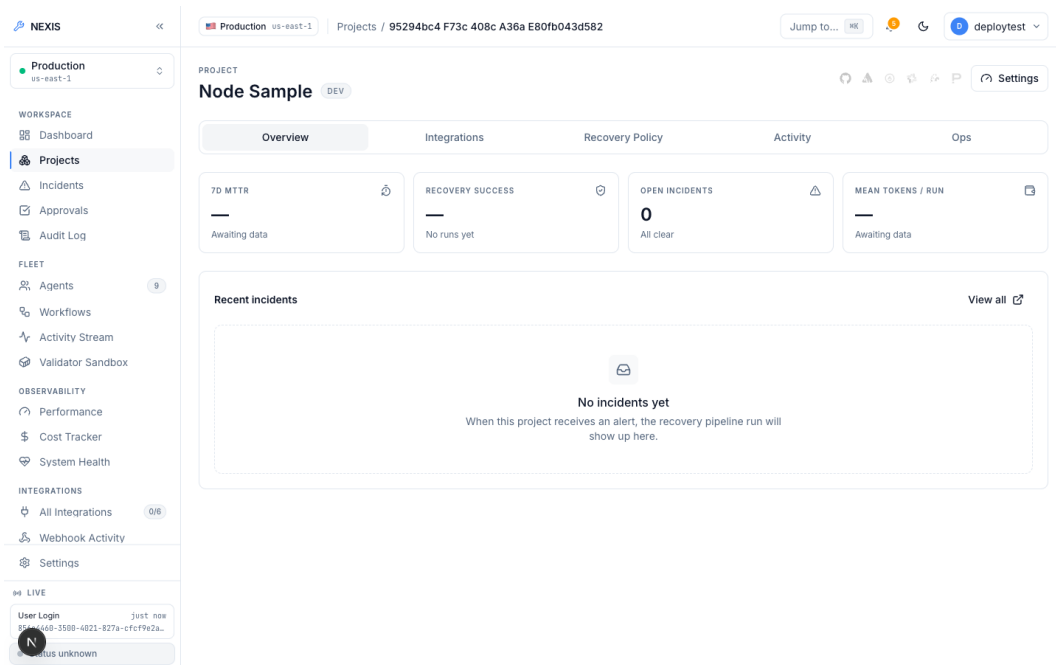


Figure 5.23: Project Overview Tab (project summary and health).

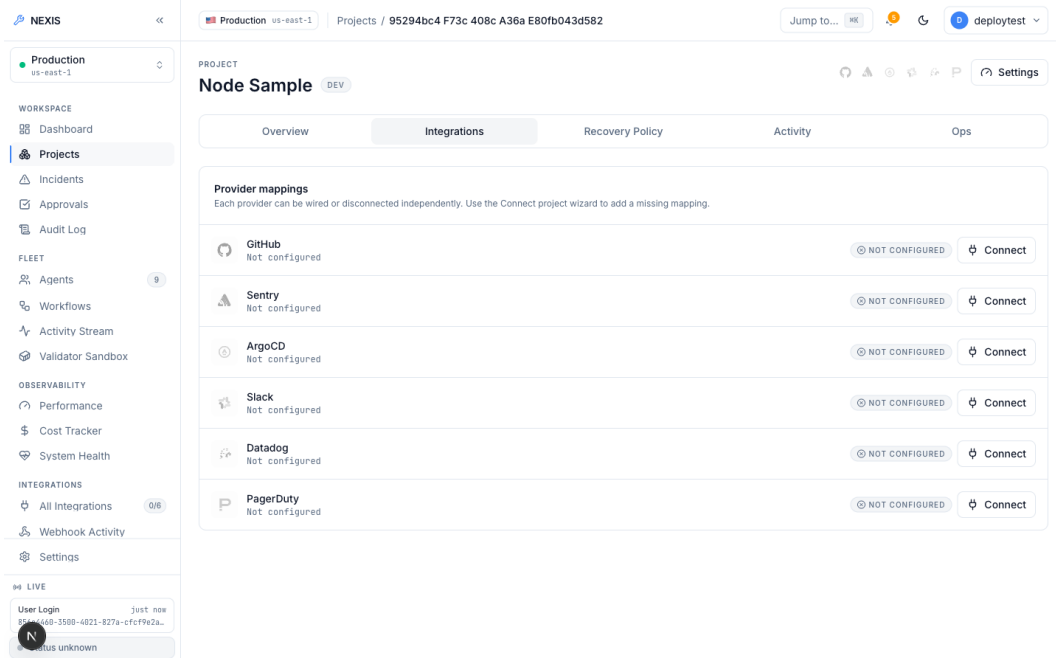


Figure 5.24: Project Integrations Tab (provider connections including the GitHub repository binding).

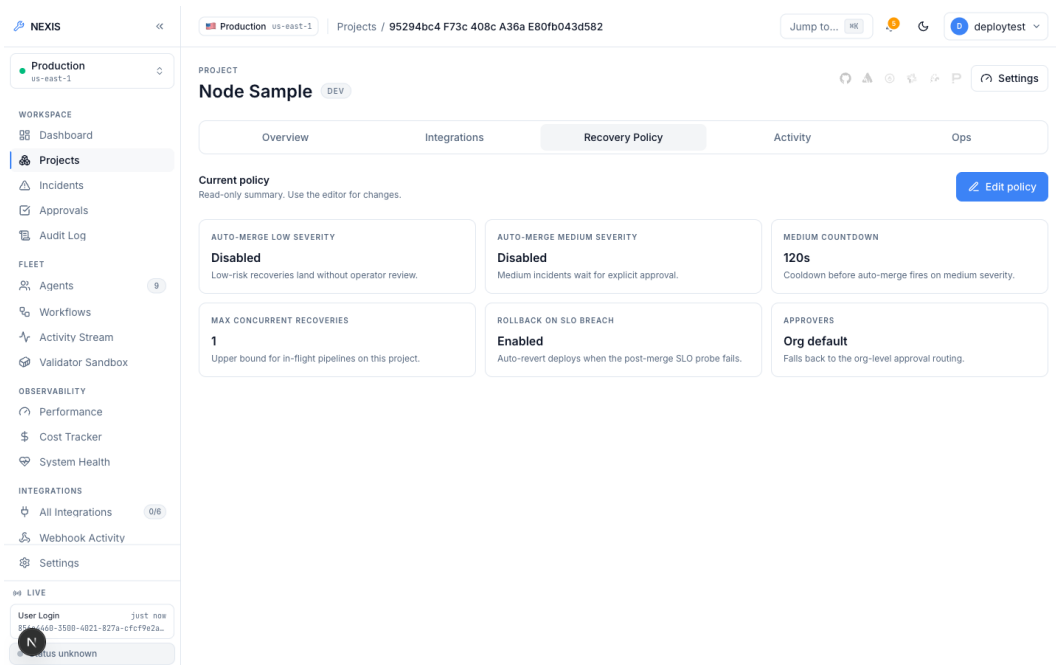


Figure 5.25: Project Recovery Policy Tab (per-project self-healing configuration).

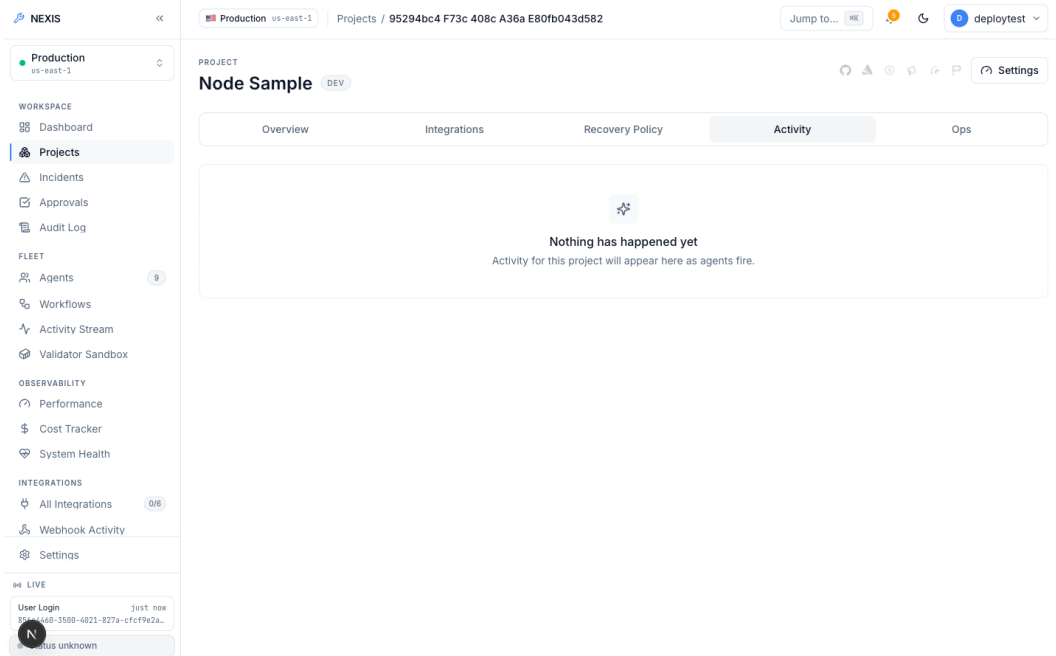


Figure 5.26: Project Activity Tab (project-scoped event history).

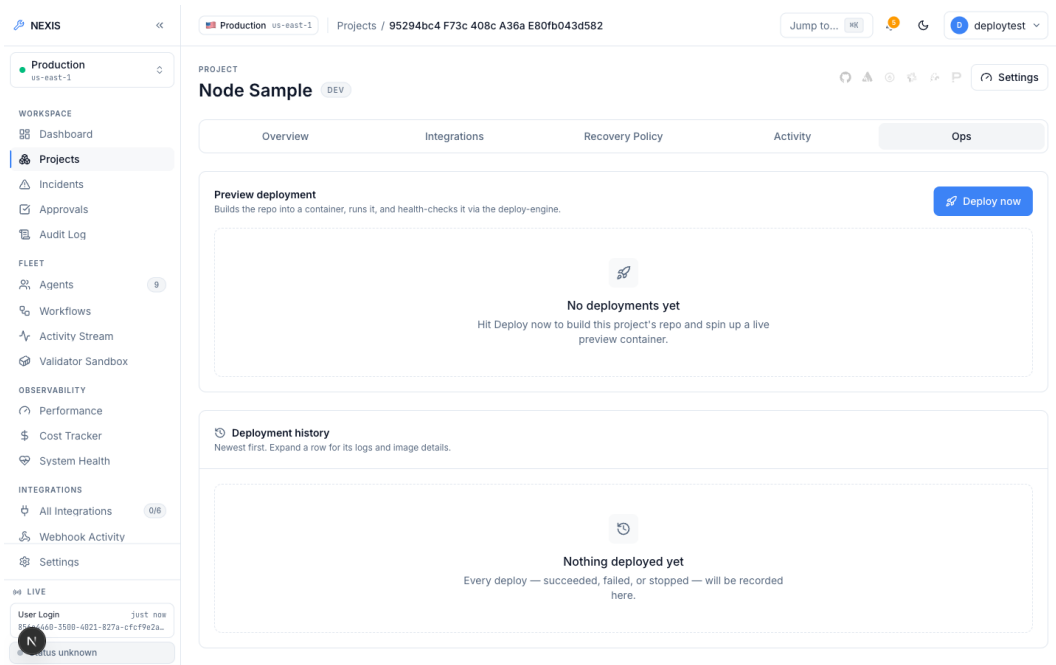


Figure 5.27: Project Ops Tab, Empty State (the Deploy panel before any deployment).

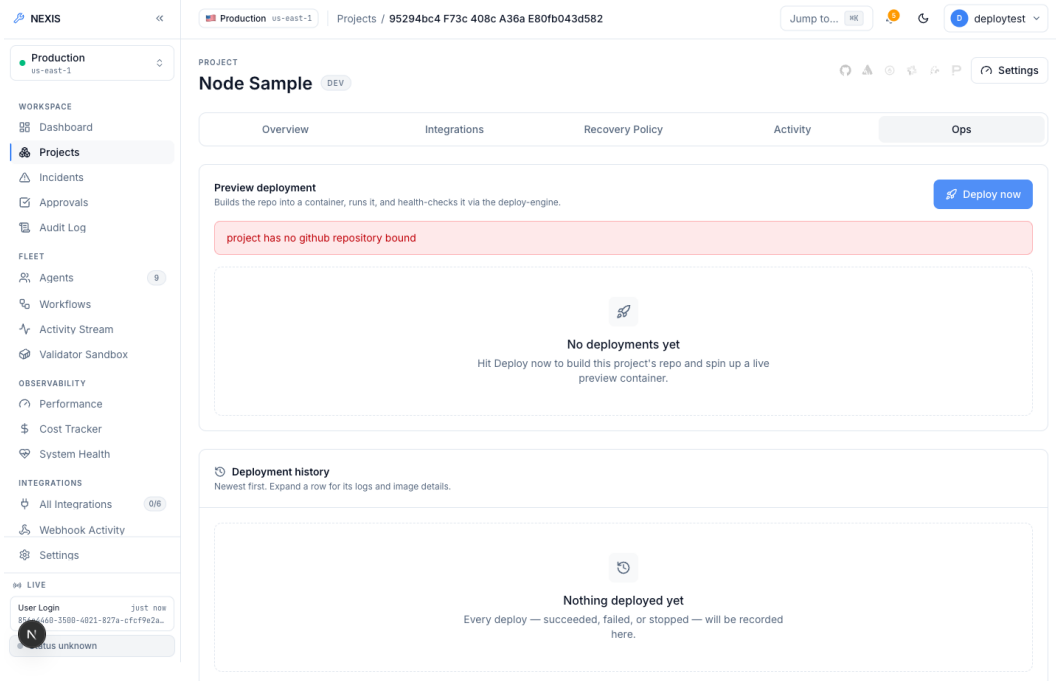


Figure 5.28: Project Ops Tab, Deploy Attempt (a real validation error — “project has no github repository bound” — returned by the live API).

reproduces verbatim the verified end-to-end run recorded during this project: a real public repository (heroku/node-js-getting-started, which contains no Dockerfile) was cloned, its stack was detected as Node from marker files, a working Dockerfile was generated, the image was built, the container was run under resource limits, and an HTTP request to the returned preview URL served the application's real rendered HTML page. The container was then stopped and removed cleanly.

```
POST /v1/deploy (repo: heroku/node-js-getting-started, no Dockerfile present)
-> 200 OK
  "status": "running", "dockerfile_source": "generated", "detected_stack": "
    node",
  "url": "http://localhost:54671", "image_tag": "nexis-preview-test-project:22
    d06076c357"
  build_log: "...npm ci... EXPOSE 3000... CMD [\"npm\\\", \"start\\\"]..."
  container_log: "Listening on 3000\nRendering 'pages/index' for route '/'"

GET http://localhost:54671/ -> real rendered HTML of the Node app (verified)

POST /v1/deploy/test-e2e-001/stop -> {"status":"stopped"}; container removed.
```

Listing 5.1: Verified live deploy-engine run

When a build or health check fails, the control plane inserts a real incident with `source: "deploy_engine"` into the same Sentinel-to-RecoveryPipeline path used by every other incident source, so a broken build or an unrecognised stack becomes a self-healing target like any production fault.

5.2.6 Projects, Agents, and Observability

The Projects page (Figure 5.29) lists the organisation's projects as cards; new projects are created through the wizard in Figure 5.30, which derives sensible defaults from the project name. Each agent has its own detail page — Figure 5.31 shows the Backend agent's — and the Recovery Pipeline page (Figure 5.32) presents the pipeline structure itself.

Observability is spread across four pages. Performance (Figure 5.33) charts pipeline latency and throughput; the Cost Tracker (Figure 5.34) rolls up the per-organisation token ledger against a configurable daily budget; System Health (Figure 5.35) fans out parallel dependency probes — Postgres, Redis, Neo4j, MinIO, Temporal — under a bounded timeout and reports each result; and the Eval Harness (Figure 5.36) runs side-by-side OpenAI-versus-Ollama provider comparisons across fixture incidents, with persisted transcripts, cost accounting, and an admin-only CSV export. The Knowledge Base page (Figure 5.37) reports the status of the pgvector-backed retrieval store.

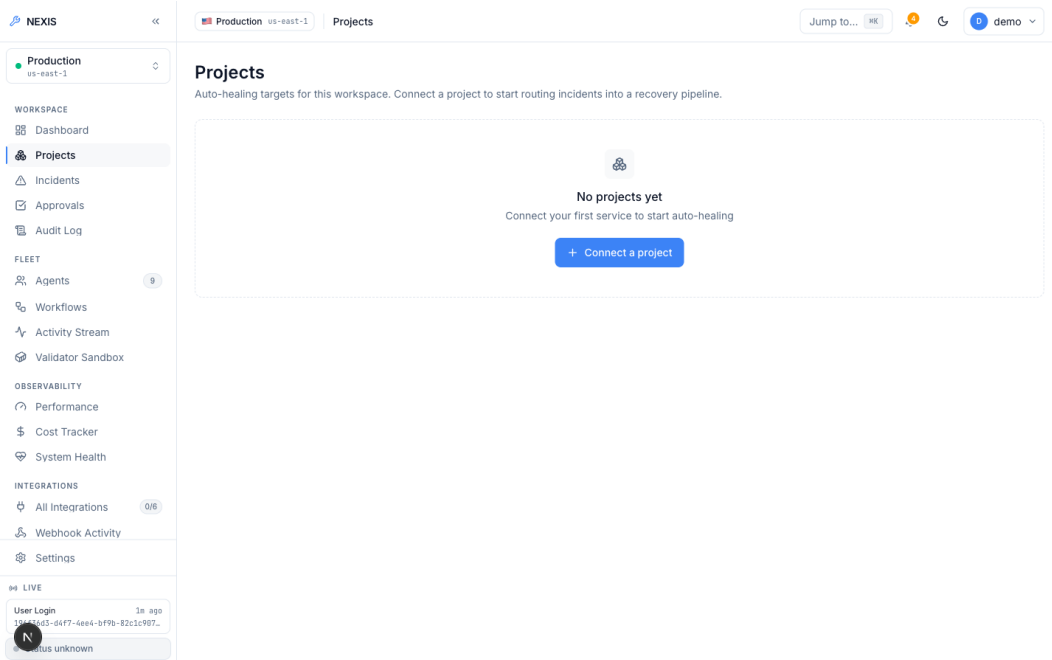


Figure 5.29: Projects Screen (project cards with health indicators).

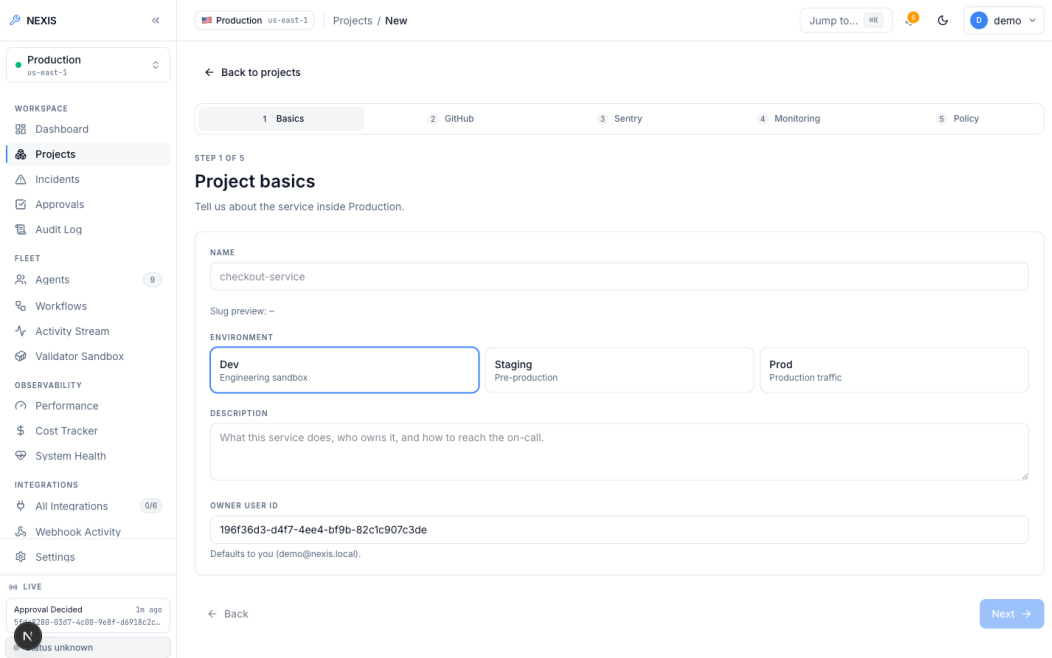


Figure 5.30: New-Project Wizard (guided project creation).

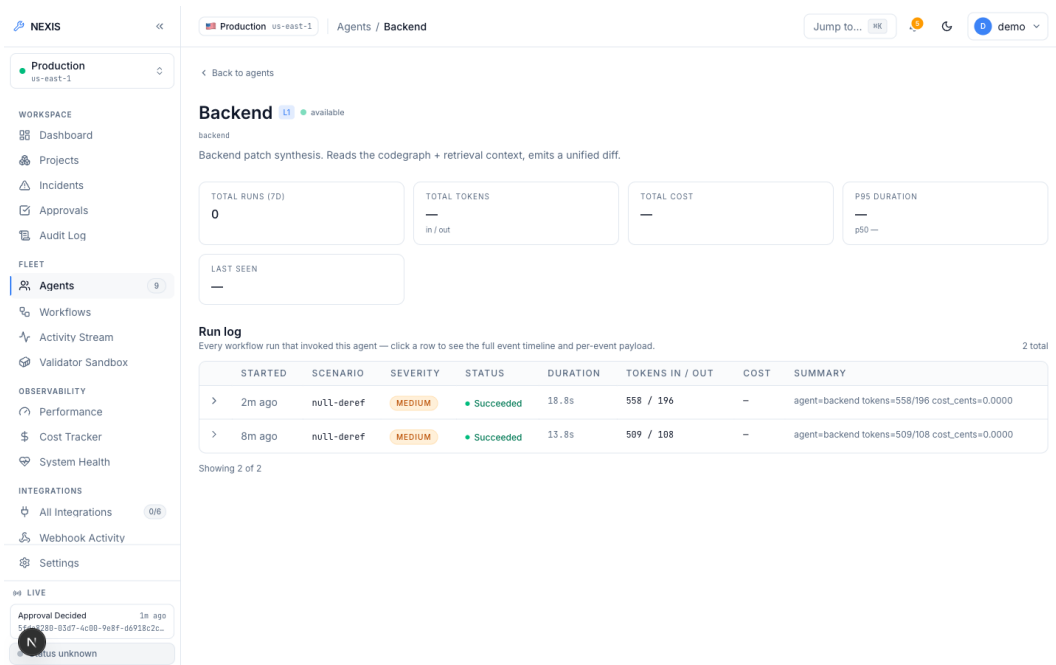


Figure 5.31: Agent Detail, Backend Agent (per-agent configuration and run history).

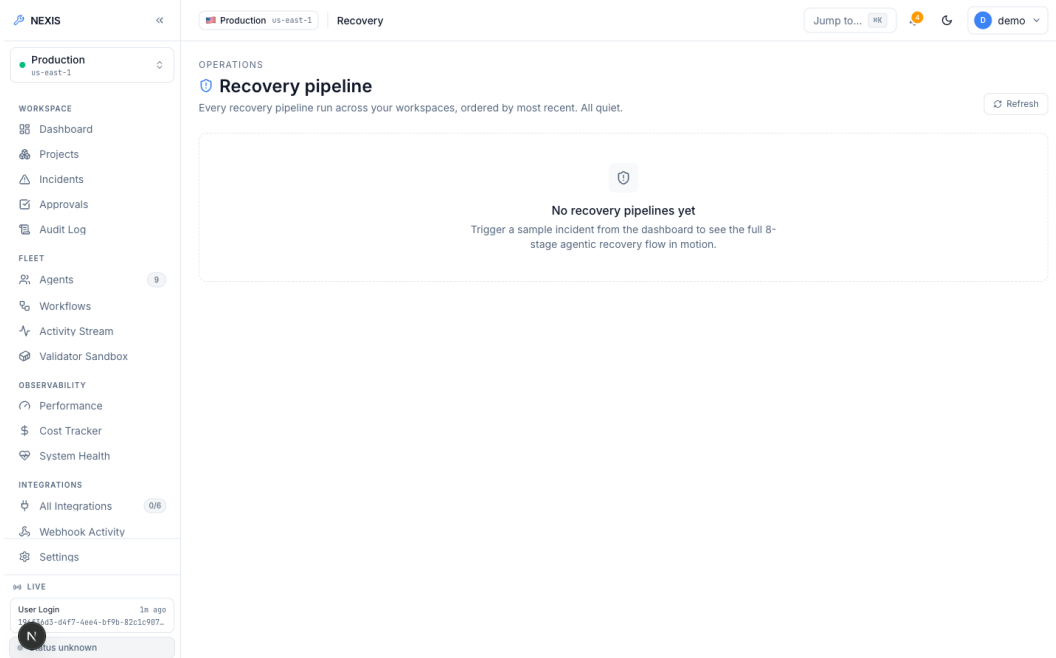


Figure 5.32: Recovery Pipeline Screen (the nine-agent pipeline structure).

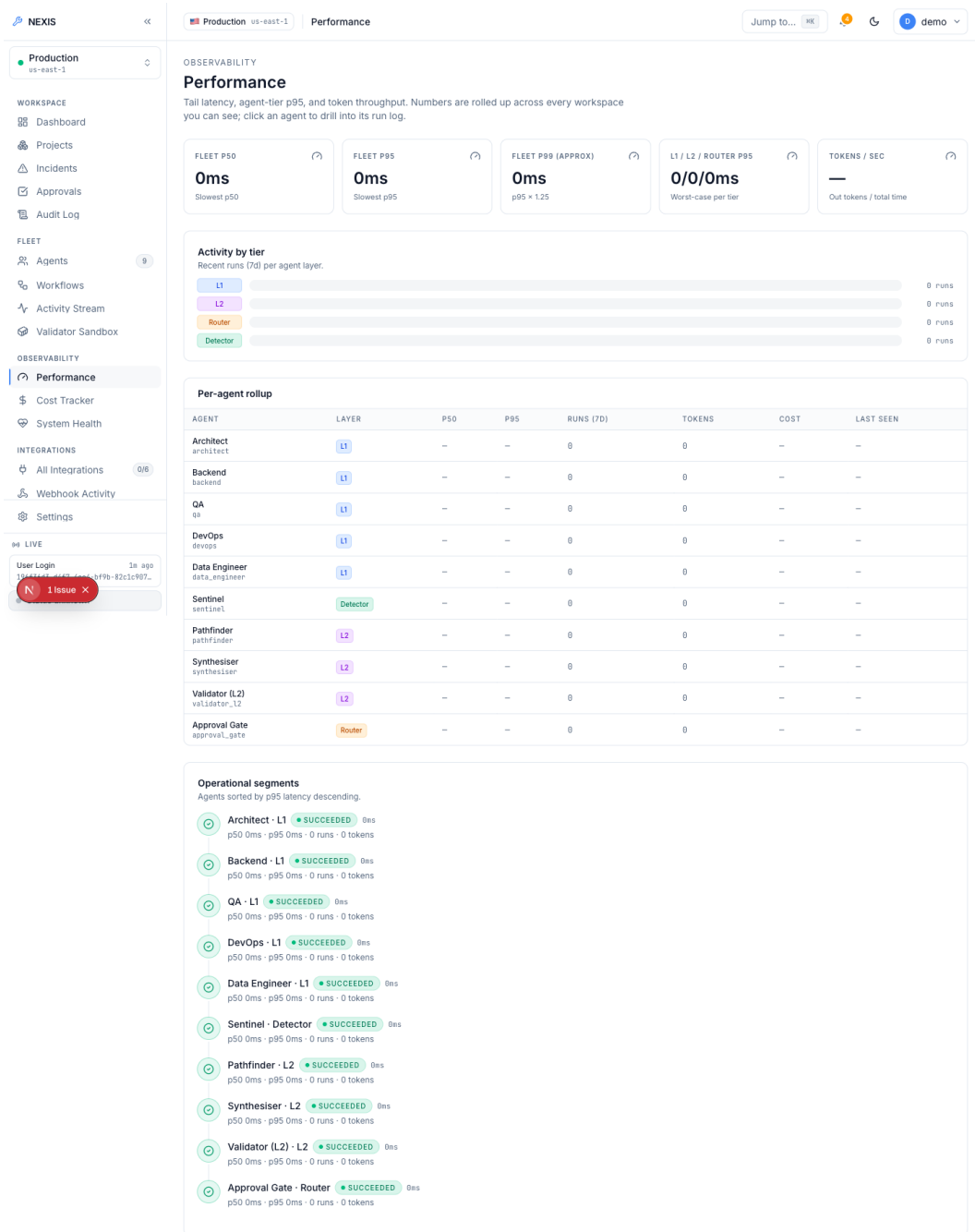


Figure 5.33: Performance Screen (pipeline latency and throughput charts).

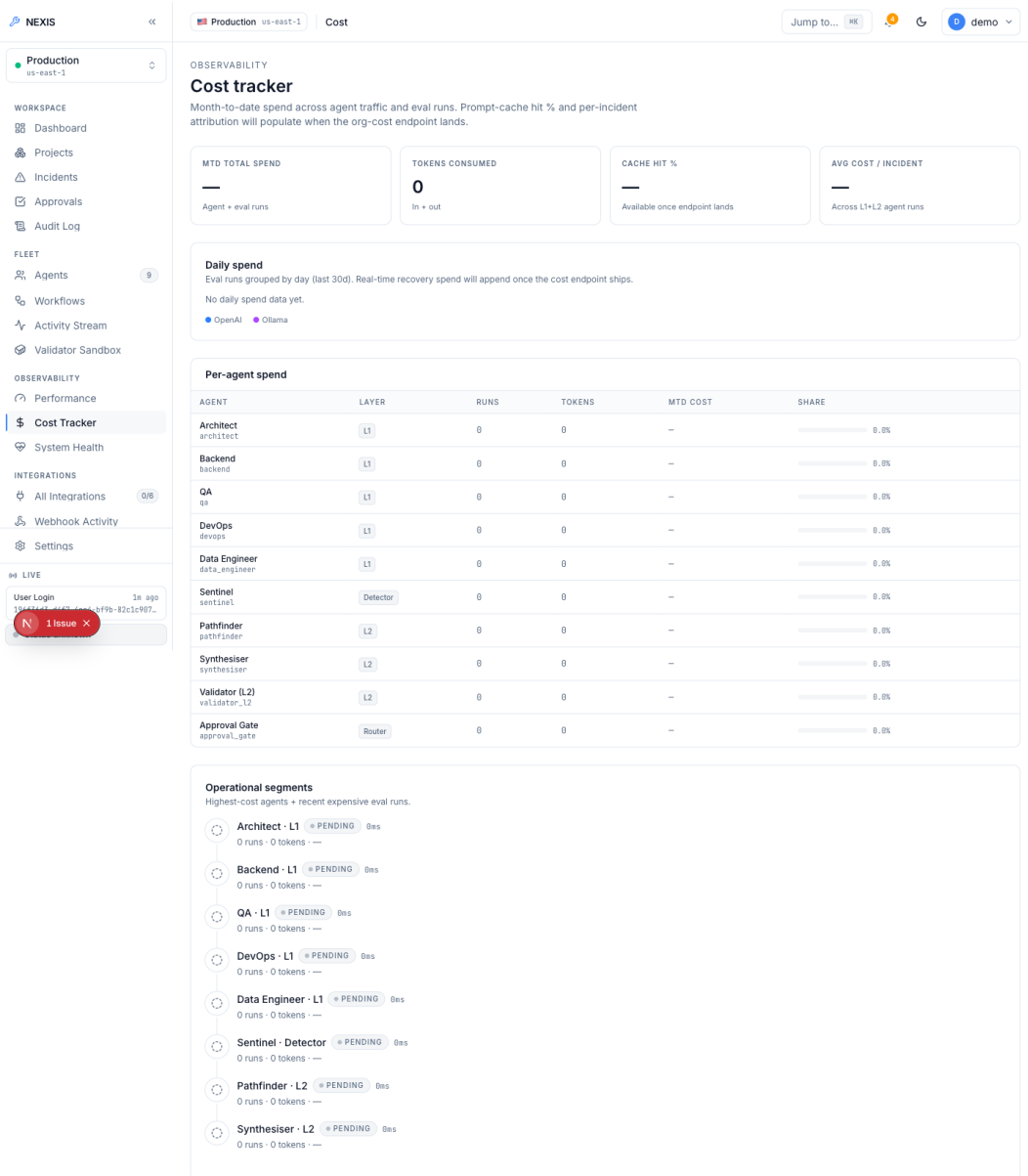


Figure 5.34: Cost Tracker (per-organisation token ledger with daily budget).

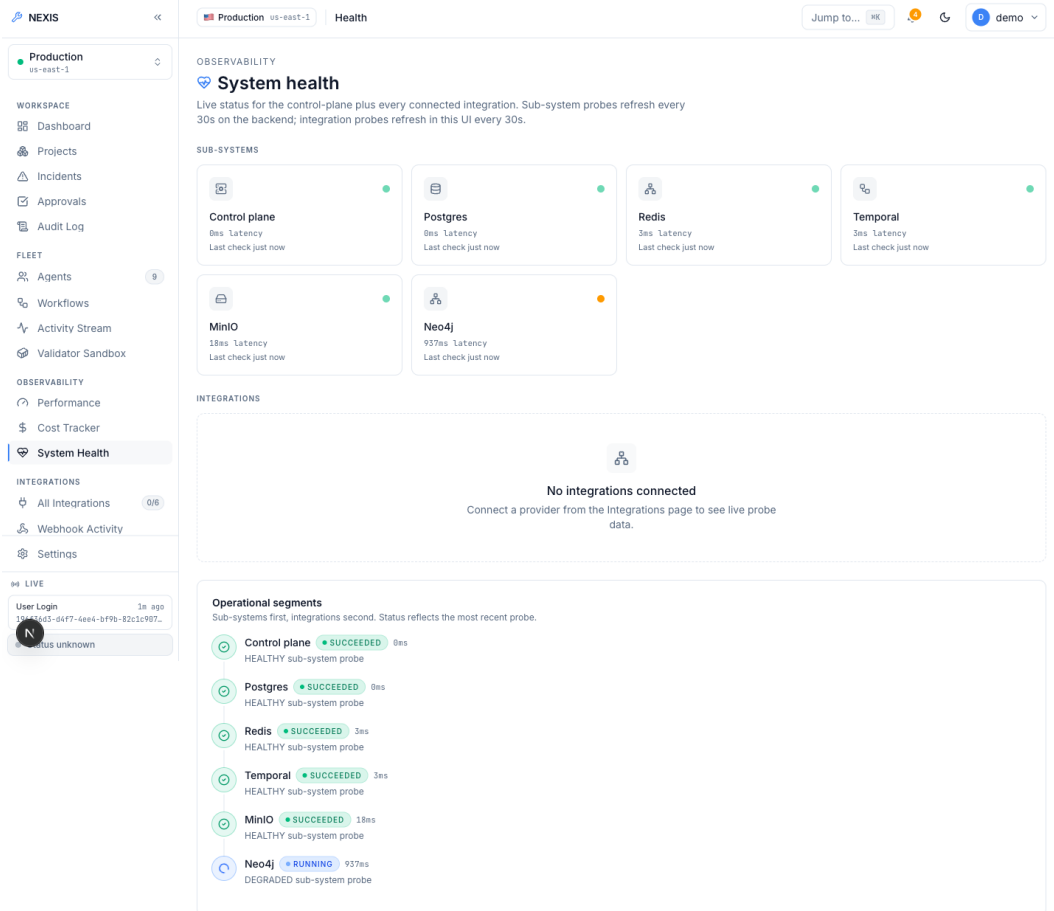


Figure 5.35: System Health (parallel dependency probes: Postgres, Redis, Neo4j, MinIO, Temporal).

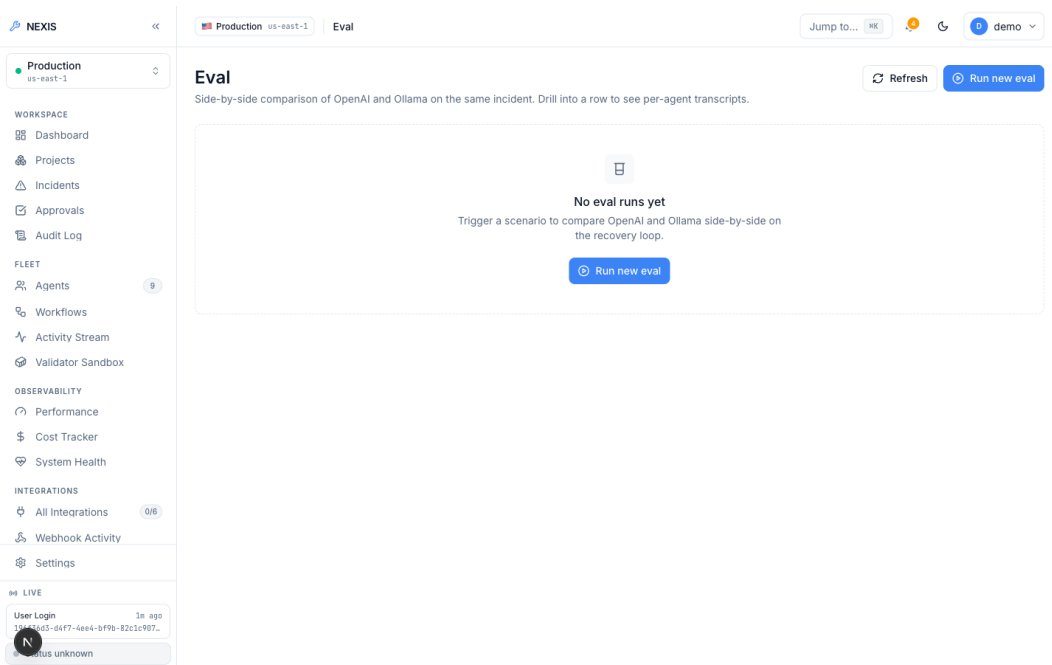


Figure 5.36: Eval Harness (side-by-side OpenAI-versus-Ollama comparison over fixture incidents).

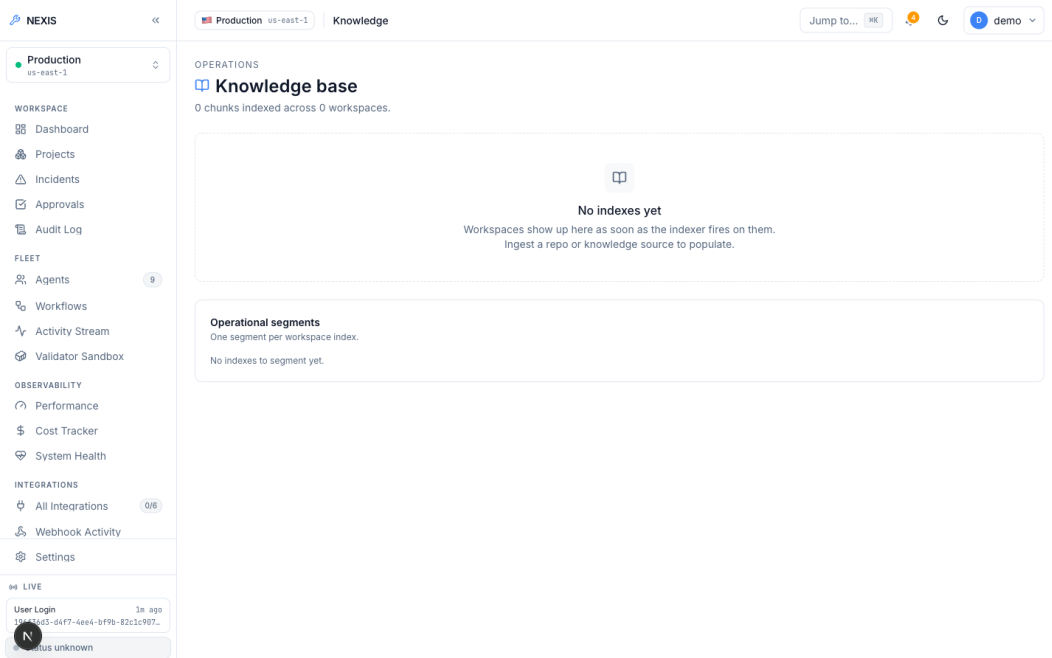


Figure 5.37: Knowledge Base (pgvector-backed retrieval status).

5.2.7 Integrations

The Integrations page (Figure 5.38) manages the external providers the platform connects to: the GitHub App (installation, repository listing, and webhook-driven incident creation on merged pull requests), Sentry [14], Datadog [16], and PagerDuty [15] as webhook-driven incident sources, Slack [17] with OAuth install and interactive one-click approve/reject directly from a Slack message, Argo CD, and Stripe billing. Webhook Activity (Figure 5.39) lists inbound webhook deliveries for debugging, and Connection Health (Figure 5.40) probes each configured integration and reports its status.

5.3 Database Schema

Nexis stores its state in PostgreSQL 17. The schema has grown incrementally across 33 sequential migrations, each tracked in version control alongside the code that depends on it. Multi-tenancy is enforced in the database itself rather than in application code: nearly every table carries an organisation identifier, and PostgreSQL Row-Level Security policies [7] restrict every query to the calling organisation's rows. A request-scoped transaction binds `app.current_org_id` for each API request, so even a query that omits an explicit `WHERE` clause on the organisation cannot read or write another tenant's data.

Table 5.1 summarises the key tables. Two deliberate exceptions to org-scoping exist: `users` is a global identity table (a person may belong to several organisations, with tenancy applied through `org_members`), and `sessions` is scoped to the owning user rather than to an organisation, since a login session belongs to a person, not a tenant.

Beyond relational state, three specialised stores complete the data layer: Neo4j holds the code dependency graph traversed by the Pathfinder agent, MinIO holds encrypted patch blobs referenced from `workflow_runs`, and pgvector columns inside PostgreSQL back the knowledge-base retrieval feature. Temporal maintains its own durable persistence for workflow event histories, which complements — and in a recovery scenario can reconstruct — the per-step records in `workflow_runs`.

Production

us-east-1

WORKSPACE

Dashboard

Projects

Incidents

Approvals

Audit Log

FLEET

Agents

9

Workflows

Activity Stream

Validator Sandbox

OBSERVABILITY

Performance

Cost Tracker

System Health

INTEGRATIONS

All Integrations

0/6

Webhook Activity

Settings

LIVE

User Login

2m ago

11f15d45-64f7-4ee4-bf9b-82c1e902...

N

Status unknown

Integrations

Connect the third-party tools NEXIS uses to detect incidents, ship fixes, and roll back deployments.

SOURCE CONTROL

Where your code + PRs live.

1 live · 2 roadmap

GitHub

AVAILABLE

Disconnected

PR creation + repo metadata for code changes.

Configure

Check access

GitLab

ROADMAP

COMING SOON

Merge requests + repo metadata.

Unavailable

Bitbucket

ROADMAP

COMING SOON

Pull requests + branch operations.

Unavailable

OBSERVABILITY

Where incidents are detected.

2 live · 5 roadmap

Sentry

AVAILABLE

Disconnected

Error tracking + issue webhooks.

Configure

Check access

Datadog

AVAILABLE

Disconnected

Metrics, APM, and anomaly detection.

Configure

Check access

New Relic

ROADMAP

COMING SOON

Full-stack observability and APM.

Unavailable

Grafana Cloud

ROADMAP

COMING SOON

Hosted Grafana + Mimir/Loki/Tempo.

Unavailable

Prometheus

ROADMAP

COMING SOON

Metrics + Alertmanager webhook ingestion.

Unavailable

Honeycomb

ROADMAP

COMING SOON

Structured-event observability + BubbleUp.

Unavailable

Splunk

ROADMAP

COMING SOON

Log search + Observability Cloud alerts.

Unavailable

INCIDENT MANAGEMENT

On-call routing + paging.

1 live · 2 roadmap

PagerDuty

AVAILABLE

Disconnected

On-call routing + paging.

Configure

Check access

Opsgenie

ROADMAP

COMING SOON

Atlassian on-call + escalation.

Unavailable

Incident.io

ROADMAP

COMING SOON

Modern incident response platform.

Unavailable

DEPLOY & INFRASTRUCTURE

Where rollbacks land.

1 live · 5 roadmap

ArgoCD

AVAILABLE

Disconnected

Deployment + rollback orchestration.

Configure

Check access

Flux CD

ROADMAP

COMING SOON

GitOps deploys + Helm controller.

Unavailable

Kubernetes

ROADMAP

COMING SOON

Direct cluster API: scale, restart, drain.

Unavailable

AWS CloudWatch

ROADMAP

COMING SOON

AWS metrics, alarms, and Lambda recovery.

Unavailable

GCP Monitoring

ROADMAP

COMING SOON

GCP alerts + Cloud Run/GKE recovery.

Unavailable

Azure Monitor

ROADMAP

COMING SOON

Azure alerts + AKS recovery.

Unavailable

CHAT & NOTIFICATIONS

Where approvers get pinged.

1 live · 2 roadmap

Slack

AVAILABLE

Disconnected

Notify channels + DM approvers.

Configure

Check access

Microsoft Teams

ROADMAP

COMING SOON

Channel posts + approval actions.

Unavailable

Discord

ROADMAP

COMING SOON

Webhook notifications.

Unavailable

DATA & PIPELINES

Job and pipeline recovery.

0 live · 6 roadmap

Apache Spark

ROADMAP

COMING SOON

Job failure detection + retry orchestration.

Unavailable

Databricks

ROADMAP

COMING SOON

Cluster + workflow recovery.

Unavailable

Apache Airflow

ROADMAP

COMING SOON

DAG failure recovery + backfill triggers.

Unavailable

Snowflake

ROADMAP

COMING SOON

Query failures + warehouse scaling.

Unavailable

dbt

ROADMAP

COMING SOON

Model failure recovery + lineage-aware retries.

Unavailable

Kafka

ROADMAP

COMING SOON

Consumer lag + dead-letter routing.

Unavailable

SECURITY & FLAGS

Feature flags + secret rotation.

0 live · 2 roadmap

LaunchDarkly

ROADMAP

COMING SOON

Auto-kill flags on incident detection.

Unavailable

HashiCorp Vault

ROADMAP

COMING SOON

Secret rotation on credential compromise.

Unavailable

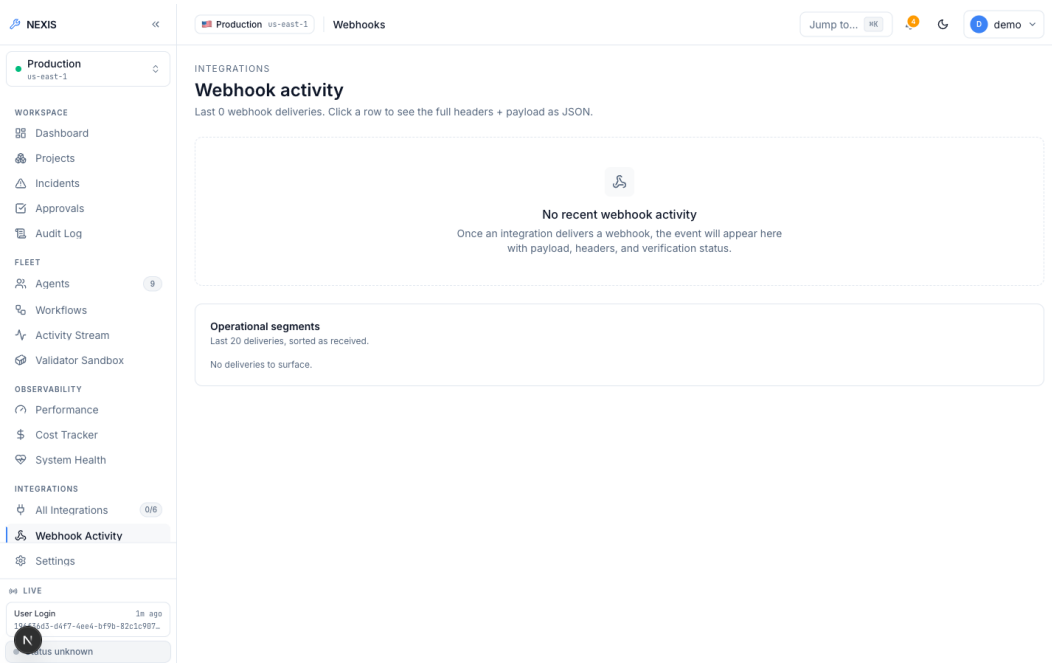


Figure 5.39: Webhook Activity (inbound webhook delivery log).

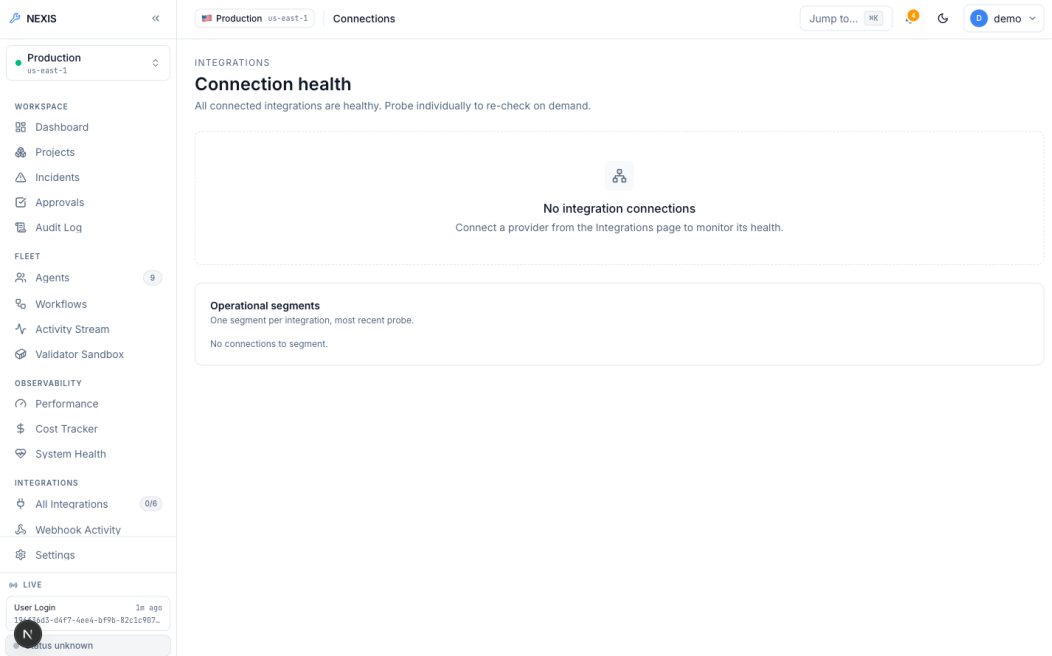


Figure 5.40: Connection Health (per-integration health probes).

Table 5.1: Key tenant-scoped tables in the Nexis PostgreSQL schema.

Table	Purpose	RLS scope
users	Global account identity: credentials, MFA enrolment, profile	Global (identity)
org_members	User-organisation membership and role (owner / admin / member)	Org-scoped
projects	Registered projects, GitHub repository binding, recovery policy	Org-scoped
incidents_raw	Every ingested fault event, from webhooks, fixtures, and the deploy engine	Org-scoped
workflow_runs	One row per Temporal RecoveryPipeline execution, with per-step activity events	Org-scoped
approval_decisions	Terminal approval-gate outcomes: approve, reject, modify, timeout	Org-scoped
deployments	Preview-deploy engine builds/containers and GitOps deployments	Org-scoped
lineage_events	OpenLineage-compatible RunEvents [8] for migration propose/apply	Org-scoped
audit_log	Append-only, hash-chained record of privileged actions	Org-scoped
feedback_examples	RLHF training rows persisted from every terminal approval decision	Org-scoped
role_recommendations	Heuristic role-change recommendations with rationale and cool-down state	Org-scoped
digest_reports	Daily admin digest aggregates (incidents, repairs, approvals, MTTR)	Org-scoped
sessions	Active login sessions: device, IP, user agent, last-seen	User-scoped
api_keys	Scoped bearer keys (nx_live_...) for programmatic access	Org-scoped

Chapter 6 Testing and Results

This chapter reports the verification evidence for NEXIS as it actually occurred during the final engineering cycle. Rather than presenting a constructed matrix of numbered test cases, the chapter states three kinds of real evidence: (i) static verification — the build, vet, lint, type-check, and automated test commands that were run against every service, and their results; (ii) live end-to-end integration runs performed against the real, running system — two complete nine-agent recovery pipeline executions and one complete deploy-engine execution, all driven by a real local large language model rather than stubs or mocks; and (iii) the defects that this testing process itself uncovered, together with their root causes and fixes. The chapter closes with an honest summary of what was and was not tested.

6.1 Static Verification

Every component of the platform was verified with its native toolchain. For the four independent Go modules, the full sequence `go build ./...`, `go vet ./...` (Go’s built-in static analyser), and `go test ./...` was executed and required to complete with zero errors, zero vet findings, and zero test failures. The two Python components were verified with `pytest`; the causal-inference service’s suite comprises 38 tests, and the validator’s Hypothesis-based sidecar [9] has its own passing suite. The Next.js frontend was verified with the project’s three gate commands: TypeScript type checking, ESLint, and a full production build. Table 6.1 summarises the commands and their outcomes.

Two aspects of this table deserve emphasis. First, the result column is not aspirational: every command listed was actually executed against the final codebase and completed cleanly, and the Go test suites include the JWT clock-injection tests discussed in Section 6.4, which were silently failing earlier in the cycle and now genuinely pass. Second, static verification covers every deployable unit of the platform — there is no service that ships without passing its build, its static analysis, and its automated tests.

Table 6.1: Static verification commands and results across all NEXIS components.

Component	Command	Result
services/control-plane (Go)	go build ./... && go vet ./... && go test ./...	Clean — build, vet, and all tests pass
services/validator (Go)	go build ./... && go vet ./... && go test ./...	Clean — build, vet, and all tests pass
services/gitops (Go)	go build ./... && go vet ./... && go test ./...	Clean — build, vet, and all tests pass
services/deploy-egg (Go)	go build ./... && go vet ./... && go test ./...	Clean — build, vet, and all tests pass
services/causal-inference (Python)	pytest	38 tests, all passing
Validator Hypothesis sidecar (Python)	pytest	All tests passing
apps/web (Next.js)	pnpm typecheck && pnpm lint && pnpm build	Clean — type check, lint, and production build all pass

6.2 Live End-to-End Recovery Runs

Static verification establishes that each service is internally sound; it says nothing about whether the nine agents, the Temporal workflow [2], the validator sandbox, the approval gate, and the live console actually cooperate. For that, two complete recovery pipeline runs were executed against the real running system — real Postgres with row-level security active, real Temporal workflows, real Neo4j dependency graph, and a real local LLM (Ollama [20] serving `qwen2.5-coder:14b` and `gpt-oss:20b`) generating the actual plans, diffs, and tests. Nothing in either run was mocked or replayed. Chapter 5 reproduces the console evidence captured during both runs: the pipeline mid-flight with live per-agent token counts, the fully green timeline awaiting approval, the honest timed-out terminal state of the first run, and the complete Approved-to-Succeeded timeline of the second.

6.2.1 Run 1: Approval-Gate Timeout — the Safety Mechanism Under Real Timing Pressure

In the first run, a fault fixture was injected, Sentinel detected it, and the full agent pipeline executed to completion: the Architect produced a structured plan, the Backend agent generated a unified-

diff patch, the QA agent generated Hypothesis property tests, and the Validator executed the patch against those tests inside its locked-down Docker sandbox — every Layer 1 and Layer 2 step reached a green state on the live timeline. The candidate patch then arrived at the human approval gate, where it sat pending. No human decision was made within the 120-second decision window, the window elapsed, and the workflow resolved the incident to a rejected terminal state.

It is important to read this outcome correctly: it is a *positive* test result, not a failure. The central safety invariant of the platform is that no patch is ever deployed without passing validation and clearing the approval gate, and that the system fails closed on ambiguity — an absent decision is treated as a refusal, never as consent. The first run exercised exactly this invariant under real timing pressure, with a real generated patch waiting on the other side of the gate, and the system behaved precisely as designed: it discarded the work rather than shipping an unreviewed change, and it recorded the timeout-rejected outcome honestly and durably in the incident timeline. A demonstration environment could easily have hidden this run and shown only the success; it is reported here deliberately, because the rejected run is the stronger evidence that the gate is real.

6.2.2 Run 2: Approved and Succeeded in 1 Minute 40 Seconds

The second run followed the same detection path but the pending approval was acted on within the window. The incident was classified at Medium severity, the generated patch passed sandbox validation, the approval was granted through the console's decision dialog, and the pipeline proceeded through its deploy stage to a *Succeeded* terminal state. The complete timeline — from fault detection through every agent step to the final succeeded state — spanned 1 minute 40 seconds.

Two properties of this run make it strong evidence rather than a staged demonstration. First, the accounting is real: the timeline records genuine per-agent token consumption and cost figures for every LLM-backed step, as produced by the live Ollama provider during the run. Second, the artifacts are inspectable: the raw JSON payload of the Backend agent's activity event shows the actual unified diff generated by `qwen2.5-coder:14b`, with `provider="ollama"` recorded in the event metadata — the patch that was validated, approved, and deployed is the patch the model wrote, not a canned fixture.

Together the two runs test both branches of the approval gate — refusal by timeout and explicit approval — against the same real pipeline, which is precisely the pair of behaviours a closed-loop recovery system must get right.

6.3 Live Deploy-Engine Verification

The deploy-engine was verified with its own live end-to-end run against a real public GitHub repository, `heroku/node-js-getting-started`, which deliberately contains no `Dockerfile`. This exercises the engine's hardest path: it must clone the repository, detect the stack from marker files, generate a working `Dockerfile` from scratch, build the image, run the container under resource limits, and prove liveness over HTTP. Listing 6.1 reproduces the actual request/response transcript of the run.

```
POST /v1/deploy (repo: heroku/node-js-getting-started, no Dockerfile present)
-> 200 OK
  "status": "running", "dockerfile_source": "generated", "detected_stack": "
    node",
  "url": "http://localhost:54671", "image_tag": "nexis-preview-test-project:22
    d06076c357"
  build_log: "...npm ci... EXPOSE 3000... CMD [\"npm\\\", \"start\\\"]..."
  container_log: "Listening on 3000\nRendering 'pages/index' for route '/'"

GET http://localhost:54671/ -> real rendered HTML of the Node app (verified)

POST /v1/deploy/test-e2e-001/stop -> {"status": "stopped"}; container removed.
```

Listing 6.1: Live deploy-engine end-to-end run: clone, generate Dockerfile, build, run, HTTP-verify, stop.

The transcript demonstrates each stage genuinely occurring: the engine detected a Node.js stack (`detected_stack: "node"`), synthesised a `Dockerfile` (`dockerfile_source: "generated"`) whose build log shows the generated `npm ci`, `EXPOSE 3000`, and `CMD` directives executing; the container log shows the application itself booting and serving its index route; an independent HTTP GET against the returned preview URL returned the application's real rendered HTML; and the stop endpoint cleanly terminated and removed the container. Because the repository is public and the request sequence is three HTTP calls, this is a reproducible integration test, not a one-off observation.

One environmental substitution should be noted for reproducibility: Docker itself was unavailable in the evaluation environment, so Podman [11] was used as the container runtime through the same Docker-compatible API [10] that the deploy-engine and validator target. The engine's code path is identical under both runtimes, and the run above proves the Podman path works end-to-end. The clone in this run used unauthenticated public-repository access; the authenticated GitHub App installation-token path exists in the code but was not exercised in this evaluation environment, as stated in the limitations.

6.4 Defects Found and Fixed During This Engineering Cycle

The verification process was not merely confirmatory: it surfaced two genuine defects, both of which were root-caused and fixed during the cycle. Table 6.2 summarises them.

Table 6.2: Defects discovered by the verification process, with root causes, fixes, and verification.

Defect	Root Cause	Fix	Verification
13 JWT-related tests silently failing	Token <code>exp</code> claims were validated against the wall clock instead of an injected test clock, so time-sensitive assertions depended on when the suite happened to run	One-line change: inject the test clock via <code>jwt.WithTimeFunc</code>	All 13 tests now pass deterministically; production behaviour verified unchanged, because the production code's default clock is already identical to the JWT library's own default
Database migration failed to apply	<code>current_role</code> is a reserved PostgreSQL keyword and was used unquoted as a column name in a migration	Column renamed to <code>existing_role</code>	Migration applies cleanly; the Go test suites over the affected schema remain green

Both defects illustrate why the verification loop earns its cost. The JWT defect was *silent*: the affected tests were failing without blocking anything visible, which is exactly the failure mode that erodes a test suite's value over time; it was found only because the full `go test ./...` output was read rather than skimmed. The migration defect was latent in a reserved-word collision that only manifests at migration-apply time — the kind of bug that static review of the SQL text can easily miss.

For completeness, one further pre-existing bug was found during this cycle but deliberately not fixed, as it is unrelated to the recovery pipeline and out of scope: the workspace-onboarding progress poller requests `/v1/workspaces/{id}/events` and receives a 404, although the workspace itself provisions successfully server-side regardless. It is recorded here, and in the limitations, rather than quietly omitted.

6.5 Testing Summary

The evidence base for NEXIS, stated plainly, is as follows. Static verification is 100% clean across every service in the platform: four Go modules pass build, vet, and their full test suites with zero errors; both Python services pass `pytest`, including the causal-inference service's 38 tests; and the frontend passes type checking, linting, and a full production build. On top of that foundation sit

three live integration runs observed directly against the real running system — two complete nine-agent recovery pipeline executions (one timeout-rejected, demonstrating the fail-closed approval gate; one Approved and Succeeded in 1 minute 40 seconds with real per-agent token and cost accounting) and one complete deploy-engine run against a real public repository (clone, generated Dockerfile, build, run, HTTP-verified, stopped). None of these runs were simulated, and the LLM behind the recovery runs was a real local model, not a stub.

Equally important is what was *not* tested. No formal load or performance testing was performed: the platform has not been exercised under concurrent multi-tenant traffic, no throughput or latency figures under load were measured, and consequently none are claimed in this report. The single measured end-to-end duration (1 minute 40 seconds) is the observed time of one real run, not a statistical performance claim. Load testing under realistic multi-tenant concurrency, along with a larger sample of recovery runs from which meaningful MTTR distributions could be drawn, is named explicitly as future work in Chapter 7.

Chapter 7 Conclusion and Future Work

7.1 Overview

This project set out to build a system that does what existing AI developer tools do not: close the loop between detecting a software failure and deploying a verified fix for it, with a human involved as a single decision point rather than as the pipeline itself. The delivered platform, Nexis, realises that goal as a working, verified system rather than a design proposal.

The central deliverable is the nine-agent closed-loop self-healing pipeline, durably orchestrated by Temporal [2]. A five-agent Execution Team (Architect, Backend, QA, DevOps, Data Engineer) performs the engineering work, while a four-agent Self-Healing Loop (Sentinel, Pathfinder, Synthesiser, Validator) detects faults, localises root causes over the Neo4j dependency graph [3], sandbox-validates every candidate patch, and routes the result through a severity-classified Approval Gate offering Approve, Reject, and Modify decisions. This pipeline was not merely implemented but exercised live: Chapter 6 reports two complete end-to-end recovery runs against a real local language model [20] – one in which the 120-second approval window genuinely elapsed and the run was timeout-rejected exactly as the safety design intends, and one in which a detected fault proceeded through planning, patch generation, property-based test validation, human approval, and deployment to a Succeeded terminal state in approximately one minute and forty seconds.

The second major deliverable is the from-scratch Docker preview-deploy engine [10]. Given a connected repository, it clones the code, detects the technology stack, generates a working Dockerfile when none exists, builds and runs the container under resource limits, and returns a live preview URL; build and health-check failures feed back into the same self-healing pipeline as any other incident source. This too was proven live: a real public repository with no Dockerfile was cloned, containerised, built, served, HTTP-verified, and cleanly stopped in a single recorded run.

Around these two engines sits a complete multi-tenant platform: JWT and SSO authentication with MFA and session management [12], role-based access control enforced down to the database row via PostgreSQL Row-Level Security [7], the heuristic role-recommendation engine that turns audit-log evidence into explainable privilege suggestions, Stripe-based billing [13], webhook-driven integrations with GitHub, Sentry, Datadog, PagerDuty, and Slack [14, 16, 15, 17], and an observability surface (dashboards, cost tracking, system health probes, hash-chained audit log)

built on OpenTelemetry [5]. All Go modules, the Python causal-inference service, and the Next.js console pass their full build, vet, lint, and test suites, as documented in Chapter 6.

7.2 Limitations of the Project

Academic integrity requires that the boundary between what was demonstrated and what was not be stated plainly. The following limitations are real, known, and deliberate; none is hidden behind softer wording.

- **No production credentials were exercised.** No real GitHub App or OpenAI production credentials existed in the evaluation environment used for this report. The live evidence was produced with local Ollama models [20] and an unauthenticated clone of a public repository; a fully credentialed production path – ending in a real merged GitHub pull request – was not exercised end-to-end.
- **The platform is not yet live on its domain.** The `nexis.sh` domain is registered but the platform is not yet deployed to it. All evidence in this report comes from local, native execution. Docker was unavailable in the build environment, so Podman [11] was substituted as a Docker-API-compatible runtime and shown to behave identically.
- **Fine-tuning is a manual step by design.** The RLHF data-export pipeline is real and tested – every terminal approval decision, including engineer-edited diffs, is persisted and exportable as JSONL – but actually submitting a fine-tuning job against that export is a deliberate manual step involving real money and a real external API call, not an automated one.
- **One of twenty-seven demo scenarios is fully wired.** The Live-Demo console lists 27 curated fault scenarios, of which exactly one is wired end-to-end to a real fixture and classification path today; the remaining 26 are UI-complete but backend-pending.
- **The causal ranking is a proxy, not a fitted causal model.** The causal-inference sidecar ranks root-cause candidates using an honest statistical and graph-structural scoring method with a transparent per-component confidence breakdown. It is explicitly not a fitted DoWhy structural causal model [4], because no interventional production data exists against which such a model could be fitted; claiming otherwise would be an overstatement.
- **A known unrelated bug was left unfixed.** A pre-existing bug in the workspace-onboarding progress poller (a 404 from `/v1/workspaces/{id}/events`) was found during eval-

uation and intentionally left unfixed as out of scope; the workspace itself provisions successfully server-side regardless.

- **No formal load testing was performed.** As noted in Chapter 6, no formal concurrent-load performance benchmark was run; the reported latencies come from single-run live executions, not from a controlled load harness.

7.3 Future Work

Each item below is grounded in the existing architecture and represents a concrete next step rather than a generic aspiration.

- **Credentialed production deployment.** Deploy the platform to the registered `nexis.sh` domain behind a real GitHub App installation and a paid OpenAI account, so that the full production path – authenticated clone, real pull-request creation, real merge-triggered redeploy – is exercised with production credentials end-to-end.
- **Complete the scenario library.** Wire the remaining 26 Live-Demo scenario cards to real fixtures and classification paths, extending the existing 11-scenario fault-injection library across the application, database, deploy, observability, and security categories it already enumerates.
- **Finer-grained Sentinel baselines.** The Sentinel’s statistical spike rule currently maintains its EWMA mean and variance baseline per organisation; extending it to per-(organisation, service) baselines would let a noisy service stop masking a quiet one.
- **A fitted causal model.** Once production incident data with genuine interventions accumulates, fit a real DoWhy structural causal model [4] and evaluate it against the current graph-evidence proxy, which was designed from the outset to be replaceable.
- **Execute a fine-tuning run.** Once the accumulated RLHF export contains enough approve/reject/modify examples, submit an actual fine-tuning job against it and measure whether the tuned model reduces rejection and modification rates at the approval gate.
- **Real preview subdomains.** Add a reverse-proxy layer (for example Traefik) in front of the deploy engine so that preview deployments receive stable per-project subdomains instead of `localhost` ports, which is a prerequisite for sharing preview URLs beyond the local machine.

- **Formal load benchmarking.** Run a controlled concurrent-load benchmark against the control plane and the recovery pipeline to characterise throughput, queue behaviour under simultaneous incidents, and Temporal worker saturation.
- **Human-subjects usability study.** Conduct the planned NASA-TLX cognitive-workload study [18] with external participants; the survey instrument is already integrated into the post-incident flow, so the remaining work is recruitment and analysis rather than implementation.

7.4 Conclusion

The central claim of this report is deliberately narrow and deliberately strong: that a software fault can travel from automatic detection, through multi-agent diagnosis, patch generation, sandboxed validation, and a single human approval decision, to a deployed fix – inside one continuous, durable, observable pipeline. That claim is not asserted; it is demonstrated with reproducible, live evidence. One recorded run shows the loop refusing to proceed when its human gate was not answered in time, and another shows the same loop carrying a real fault to an approved, deployed resolution in under two minutes, with every agent step, token count, and decision recorded in a durable timeline. The preview-deploy engine extends the same principle to infrastructure, turning its own build failures into self-healing targets. Equally important is what this report declines to claim. The honest accounting in Section 7.2 – local models rather than production credentials, a registered but not yet live domain, a proxy rather than a fitted causal model, one wired demo scenario out of twenty-seven – is not a list of apologies but part of the project’s rigor: the boundary of the evidence is drawn exactly where the evidence stops. Within that boundary, Nexis stands as a complete, working demonstration that closed-loop fault recovery, gated by a single human decision point, is buildable today.

References

- [1] GitHub Copilot. AI pair-programming assistant for code completion and suggestion. Available at: <https://github.com/features/copilot> (Accessed: 11 August, 2026).
- [2] Temporal. Durable workflow orchestration engine for fault-tolerant distributed applications. Available at: <https://temporal.io/> (Accessed: 11 August, 2026).
- [3] Neo4j. Native graph database for storing and querying highly connected data. Available at: <https://neo4j.com/> (Accessed: 11 August, 2026).
- [4] DoWhy. Causal inference library from Microsoft Research / PyWhy for modelling and estimating causal effects. Available at: <https://github.com/py-why/dowhy> (Accessed: 11 August, 2026).
- [5] OpenTelemetry. Open-source observability framework for traces, metrics, and logs. Available at: <https://opentelemetry.io/> (Accessed: 11 August, 2026).
- [6] Argo CD. Declarative GitOps continuous delivery tool for Kubernetes. Available at: <https://argo-cd.readthedocs.io/> (Accessed: 11 August, 2026).
- [7] PostgreSQL Row Security Policies. Official documentation on row-level security for multi-tenant data isolation. Available at: <https://www.postgresql.org/docs/current/ddl-rowsecurity.html> (Accessed: 11 August, 2026).
- [8] OpenLineage. Open standard for data lineage collection and analysis. Available at: <https://openlineage.io/> (Accessed: 11 August, 2026).
- [9] Hypothesis. Property-based testing library for Python. Available at: <https://hypothesis.readthedocs.io/> (Accessed: 11 August, 2026).
- [10] Docker. Container platform for building, shipping, and running applications. Available at: <https://www.docker.com/> (Accessed: 11 August, 2026).
- [11] Podman. Daemonless, Docker-API-compatible container engine. Available at: <https://podman.io/> (Accessed: 11 August, 2026).
- [12] WorkOS. Enterprise-ready authentication and single sign-on platform. Available at: <https://workos.com/> (Accessed: 11 August, 2026).

- [13] Stripe. Online payment processing and billing infrastructure. Available at: <https://stripe.com/> (Accessed: 11 August, 2026).
- [14] Sentry. Application error tracking and performance monitoring platform. Available at: <https://sentry.io/> (Accessed: 11 August, 2026).
- [15] PagerDuty. Incident response and on-call management platform. Available at: <https://www.pagerduty.com/> (Accessed: 11 August, 2026).
- [16] Datadog. Cloud-scale monitoring and observability service. Available at: <https://www.datadoghq.com/> (Accessed: 11 August, 2026).
- [17] Slack API. Platform APIs for building Slack integrations and interactive messages. Available at: <https://api.slack.com/> (Accessed: 11 August, 2026).
- [18] NASA Task Load Index (NASA-TLX). Subjective workload assessment instrument developed by NASA Ames Research Center. Available at: <https://humansystems.arc.nasa.gov/groups/tlx/> (Accessed: 11 August, 2026).
- [19] pgvector. Open-source vector similarity search extension for PostgreSQL. Available at: <https://github.com/pgvector/pgvector> (Accessed: 11 August, 2026).
- [20] Ollama. Tool for running large language models locally. Available at: <https://ollama.com/> (Accessed: 11 August, 2026).